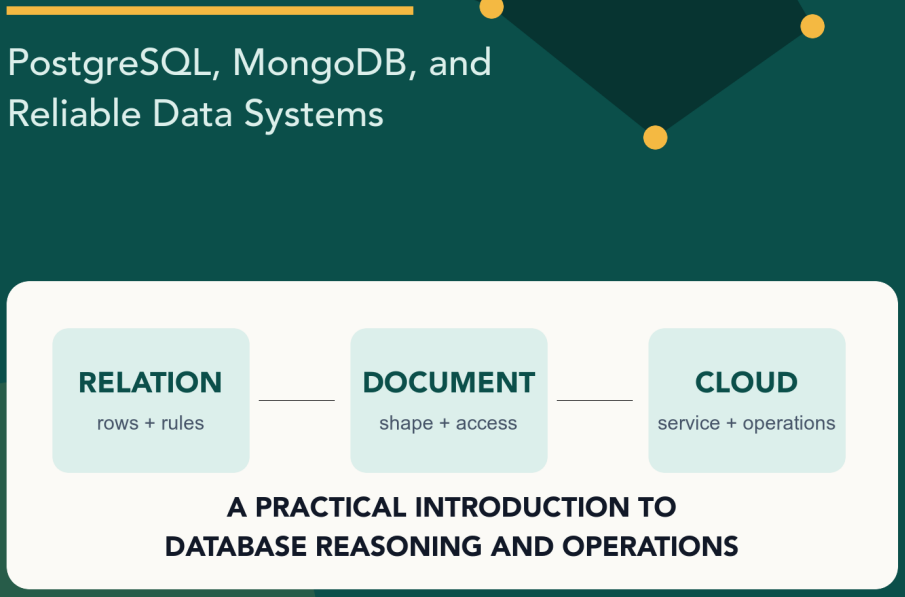
AN OPEN TEXTBOOK

# OPERATING CLOUD DATABASES

PostgreSQL, MongoDB, and Reliable Data Systems




**A PRACTICAL INTRODUCTION TO DATABASE REASONING AND OPERATIONS**

**ATILIO BARREDA**

Second edition draft | CC BY-NC-SA 4.0

# Contents


| Section | Page |
|---|---|
| About This Book | 3 |
| **Part I: Relational Foundations** | **5** |
| 1. How Database-Backed Applications Work | 6 |
| 2. Relational Operations Become Testable SQL | 16 |
| 3. A Schema Protects Meaning | 28 |
| **Part II: Operating PostgreSQL Safely** | **38** |
| 4. Safe Changes Are Planned and Verified | 39 |
| 5. Transactions Coordinate Competing Work | 48 |
| 6. Access Should Be Useful and Limited | 63 |
| 7. Performance Work Begins With Measurement | 73 |
| 8. A Backup Matters Only When Recovery Works | 81 |
| **Part III: Documents and MongoDB** | **90** |
| 9. Documents Emerged From Changing Workloads | 91 |
| 10. MongoDB Models the Way an Application Reads | 106 |
| 11. MongoDB Operations Connect Pipelines, Rules, and Indexes | 121 |
| 12. Reliability Is a Set of Explicit Promises | 130 |
| **Part IV: Scale and Integration** | **144** |
| 13. Scale Changes the Questions a System Must Answer | 145 |
| 14. Multiple Databases Multiply Options and Obligations | 158 |
| **Part V: Professional Communication** | **169** |
| 15. Professional Communication Makes Technical Skill Visible | 170 |
| Appendix A: Notation and Technical Language | 178 |
| Appendix B: Reusable Technical Templates | 182 |
| Appendix C: Running the Examples | 186 |
| License, Attribution, and Reuse | 195 |

# About This Book

*Operating Cloud Databases* is a beginner-friendly, lab-first textbook for students who need both a substantial relational/SQL review and an introduction to modern managed data systems. It develops one operating method across every topic: define the question, model the mechanism, run a controlled check, interpret the result, and state the tradeoff.

The book begins with relations, relational algebra, SQL, schema, and constraints. It then teaches safe change, transactions, access control, performance, and recovery in PostgreSQL and Supabase. The second half introduces NoSQL history, JSON, MongoDB query and document modeling, aggregation, validation, indexing, replication, reliability, scale, and multi-database systems. The final chapter turns technical work into clear portfolio and interview explanations.

This second edition adds mathematical notation, explicitly labeled code listings, editable conceptual diagrams, dated and redacted cloud interface figures, expanded relational review, and appendices for notation, glossary, and operational templates. The original edition remains preserved separately.

## Who This Is For

The primary reader has written some SQL before but may not remember how relational operations explain a query. No prior cloud administration or MongoDB experience is assumed. Examples stay small enough to reason about, while the questions mirror real operational work: What does this query mean? What can fail? Who owns the control? Which result would change the decision? What remains uncertain?

## How to Read a Chapter

Every chapter follows a stable learning pattern:

1. **Operating question:** the problem the chapter teaches you to reason about.
2. **Learning outcomes:** actions you should be able to perform and explain.
3. **Concepts and models:** durable ideas before product menus.
4. **Numbered code listings:** executable or inspectable examples with a named language and an explanation of the expected result.
5. **Worked example:** one decision carried from question through verification.
6. **Misconceptions:** plausible shortcuts that fail under closer inspection.
7. **Practice, retrieval, and transfer:** individual work that asks you to reuse the idea in a new situation.

## The Recurring Case

Metro Support is a small synthetic service-request system with users, tickets, and ticket events. Its scale is intentionally modest. A beginner can inspect every row, while the same facts support joins, constraints, locks, access policies, indexes, backups, document models, aggregation, replication, integration, and incident communication.

The dataset is not presented as a production city system. It is a controlled environment for learning how an operator moves from a concrete question to a tested decision.

## Cloud Interfaces and Change

Interface figures were captured from live free-tier teaching projects on August 25, 2026 and redacted before inclusion. They show what was observed, not a promise that a menu or plan will remain identical. Each screenshot is paired with the durable concept it demonstrates and a reminder to verify current official documentation before using a feature.

The worked examples do not require a paid textbook, subscription, certification, or database plan. Local, static, and simulated examples distinguish the concepts they demonstrate from behavior that requires a live managed service.

## Accessibility and Formats

The canonical source uses semantic headings, descriptive links, text alternatives, language-labeled code blocks, and source equations that convert to Word equations and MathML. The same book is built as editable Word, PDF, standalone HTML, and EPUB so readers can select the format that works best with their device and assistive technology.

Each chapter includes self-contained practice for revision and transfer. Companion notebooks provide executable explorations, not prerequisites for reading the explanations. A chapter's shell listing and its associated Python notebook teach related ideas in different languages. Use the stated environment and fixture rather than pasting shell code into a Python cell or assuming that two teaching datasets have identical rows.

## Independent Study

No class meeting, instructor handout, learning-management system, or completed assignment is assumed. Start with Chapter 1 if database terminology is new; Chapter 2 rebuilds SQL and relational reasoning before later chapters use them. Appendix C explains the execution environments and prints the complete relational dataset and indexing workload. Use it before running your first database example. Code fragments identify the surrounding state they require; complete examples include setup and cleanup. Companion notebooks offer another way to explore the examples, not a prerequisite for understanding the book.

## Edition Status

This is a **second-edition textbook draft**. Editable source and the other formats are maintained in the authoring package. Distribution is not a formal institutional deposit or approval. Version and review records distinguish this draft from a future formally released edition.

# Part I: Relational Foundations

Database administration begins with meaning. Before tuning, backing up, or distributing a database, an operator must know what one row represents, which facts belong together, which rules make a state valid, and how a query transforms relations.

Chapters 1-3 establish a question-model-test cycle, rebuild relational algebra and SQL from first principles, and connect schemas and constraints to business meaning. The goal is not memorizing syntax in isolation. It is predicting a result, expressing the same operation precisely, and verifying whether the actual database state supports the claim.

# 1 How Database-Backed Applications Work

## 1.1 Opening Question

You tap **Submit** in an application. A moment later, a new support ticket, playlist, order, or game item appears on screen. What happened between the click and the stored result?

That question is the starting point for database administration. Before tuning, backing up, securing, or scaling a database, you need a useful mental model of the complete system around it.

We will follow one fictional service, Metro Support, from a resident's request to the reports used by staff. Keeping one case lets us see how an early decision has consequences later. An optional assignee affects a join. A status spelling affects a workload count. A second copy affects what a resident sees after an update. Database administration begins with understanding those connections, not with memorizing a dashboard.

## 1.2 Learning Outcomes

After this chapter, you can:

- distinguish data, a database, a database management system, and a managed cloud platform;
- trace a request from an application to stored data and back;
- explain how PostgreSQL relates to Supabase and how MongoDB relates to Atlas;
- describe the technical work performed by database administrators, backend developers, data engineers, and cloud support staff;
- recognize where schemas, queries, identities, networks, and storage can affect one request; and
- create a simple Markdown document with GitHub's browser editor.

## 1.3 Begin With a Familiar Application Action

Suppose a resident submits a streetlight repair ticket. The application may send data resembling this request:

**Listing 1. JSON**

```
{
  "requester_id": 101,
  "category": "streetlight",
  "priority": "high",
  "description": "Lamp is dark at Harbor Avenue"
}
```

The browser does not usually write directly to a database. A common path is:

1. the browser or mobile client collects input;
2. an application server or API checks the request;
3. the application authenticates the user and determines what the user may do;
4. the application sends a SQL statement or MongoDB operation to a DBMS;
5. the DBMS checks types, constraints, permissions, and transaction state;
6. the DBMS reads or changes stored data; and

7. the result travels back through the application to the user.

A delay or error can occur at any stage. The database is important, but it is not the entire application.

The request body above is input, not proof of identity. A caller could change `requester_id` from 101 to somebody else's number. The server must establish who the caller is and either derive the requester ID from that identity or check that the caller is permitted to act for the named person. A valid JSON object and a valid database foreign key would not, by themselves, make that request authorized. We will separate those responsibilities carefully in Chapter 6.

### 1.3.1 Persistence Is Different From What the Screen Shows

The browser may hold unsaved text in memory. The API may hold a request while it is being processed. The DBMS manages the stored record. These are different states, even when they display the same words. Closing a browser tab should not delete a ticket that the service has already accepted into durable storage. Conversely, showing a friendly confirmation before a database write completes can promise more than the system has done.

A database also contains more than what one screen chooses to retrieve. Consider this small example:

| Ticket | Status | Assigned agent |
|---:|---|---|
| 1001 | open | 201 |
| 1003 | resolved | 201 |
| 1004 | new | none yet |
| 1009 | new | none yet |

Suppose the staff screen first keeps active requests, where active means `new` or `open` in this small example. That leaves 1001, 1004, and 1009. It then matches each request to a known agent and retains only successful matches. Now only 1001 appears. Nothing in that sequence deleted either unassigned request. The screen has answered "which active requests already have a matching agent?" rather than "which requests still need attention?"

The appropriate repair is to preserve an active request even when its agent is missing, then label the missing assignment clearly. Adding a larger server, restoring a backup, or recreating the missing requests would not repair this query's meaning. This is why we review the relational model before studying performance and recovery. A fast, highly available system can still give the wrong answer.

### 1.3.2 A Confirmation Can Be Lost After a Write Succeeds

Now consider a different failure. The DBMS stores a new request, but the network connection breaks before the API's reply reaches the browser. The resident sees an error even though the ticket exists. Pressing Submit again might create a second ticket unless the application can recognize the repeated operation.

At this point we do not need a particular implementation. We need the right distinction: "the client did not receive success" is not identical to "the database did not make the change." Later chapters introduce transactions, operation identifiers, and retries. Each is a way to make this uncertainty manageable rather than pretending it cannot occur.

## 1.4 Four Terms That Should Not Be Blurred Together

### 1.4.1 Data

**Data** represents facts or observations. In the ticket example, `101`, `streetlight`, and `high` are values. Their column or field names provide meaning.

### 1.4.2 Database

A **database** is an organized collection of data. It may contain tables, documents, indexes, views, constraints, and metadata.

### 1.4.3 Database Management System

A **database management system**, or **DBMS**, is the software that stores and retrieves data while coordinating users and programs. PostgreSQL and MongoDB are DBMSs. They provide query languages, access controls, indexes, transactions, concurrency behavior, and recovery mechanisms.

### 1.4.4 Managed Database Platform

A **managed platform** runs a DBMS on cloud infrastructure and adds management services. The platform may handle servers, storage hardware, software updates, monitoring, and parts of high availability. The customer still creates data models, queries, indexes, users, network rules, and application code.

"Managed" describes an allocation of work. It is not a guarantee that every application rule, query, permission, or recovery procedure is correct. If a provider replaces a failed disk, that addresses one infrastructure problem. It does not determine whether our analyst should be allowed to read residents' email addresses. A useful administrator can identify the particular control responsible for a result instead of treating the provider as one indivisible box.

Two comparisons will recur throughout this book:

| DBMS | Managed platform in the examples | Main representation |
|---|---|---|
| PostgreSQL | Supabase | relations, usually shown as tables |
| MongoDB | MongoDB Atlas | BSON documents, usually written in JSON-like syntax |

Supabase is not a replacement name for PostgreSQL. It is a platform built around PostgreSQL and additional services. Atlas similarly operates MongoDB deployments and provides cloud management tools.

### 1.4.5 Why Database Systems Separate Logical and Physical Decisions

In the relational model, a user asks for facts by naming relations, attributes, and conditions rather than by naming disk locations. This distinction is part of *data independence*: programs should not need to change merely because the system changes how it stores or finds the same facts. Codd's 1970 paper made protection from representation changes a central motivation for relational databases. The original paper's IBM research record provides historical context; the chapter explains the concept without requiring a paid article download.

For example, the question “which requests are still open?” remains the same if the DBMS adds an index. The index is a physical access choice, and the query can retain its logical meaning. If we instead rename a field or redefine what open means, we have changed an interface or a data contract. A consumer may need revision even though the files remain on the same disk. Chapter 4 studies how to manage that second kind of change.

This separation is an aim, not perfect insulation. Physical choices can change latency and resource cost, and those changes matter to an application. The important advantage is being able to discuss logical correctness and physical execution as related but different questions.

## 1.5 Read a Cloud Dashboard Without Treating It as the Database

Figure 1.1 shows a Supabase project overview. The screen combines several kinds of information: project health, database location, compute size, request counts, and shortcuts to other platform services. The **Primary Database** card refers to PostgreSQL; the surrounding dashboard belongs to the Supabase platform.

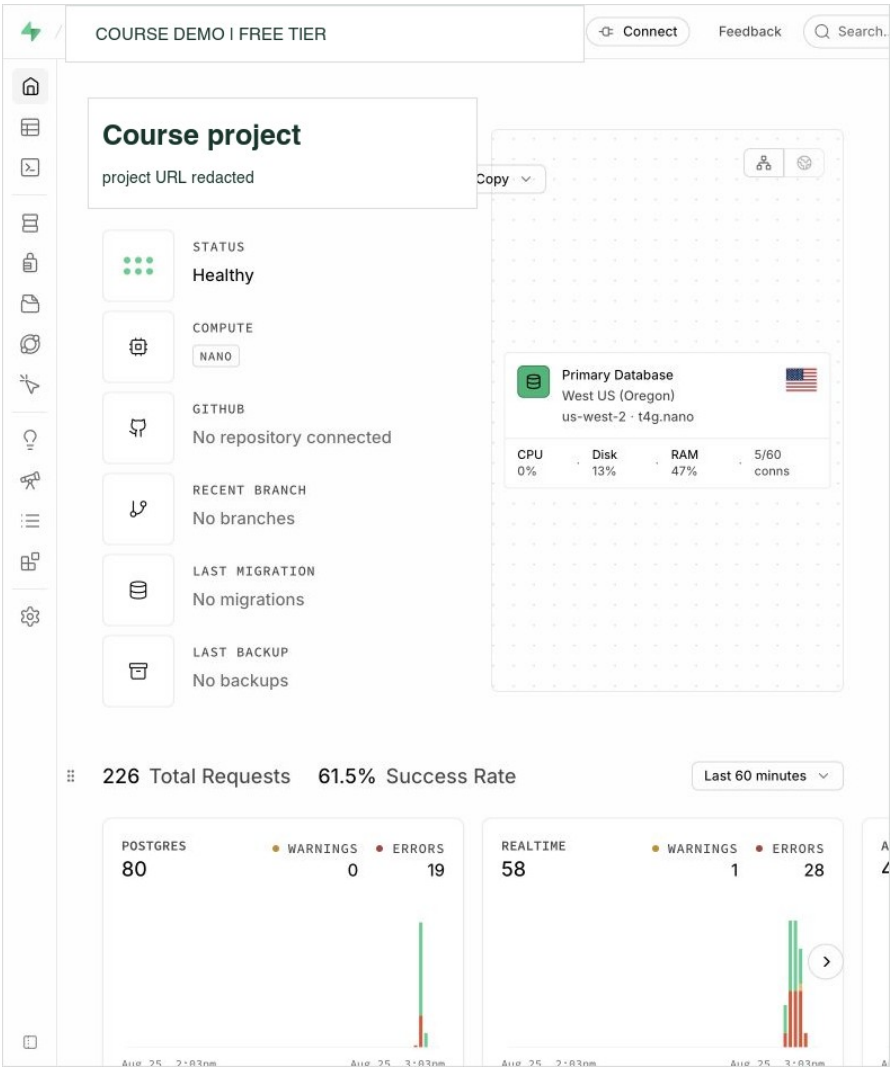



*Figure 1.1: A redacted Supabase Free project overview. The database is one component inside a larger managed platform. Interface captured August 25, 2026.*

Figure 1.2 shows an Atlas project overview. The project contains a MongoDB deployment and links to Data Explorer, network access, database users, backup, and other services. The yellow network notice matters because a running database can still be unreachable from the current client.

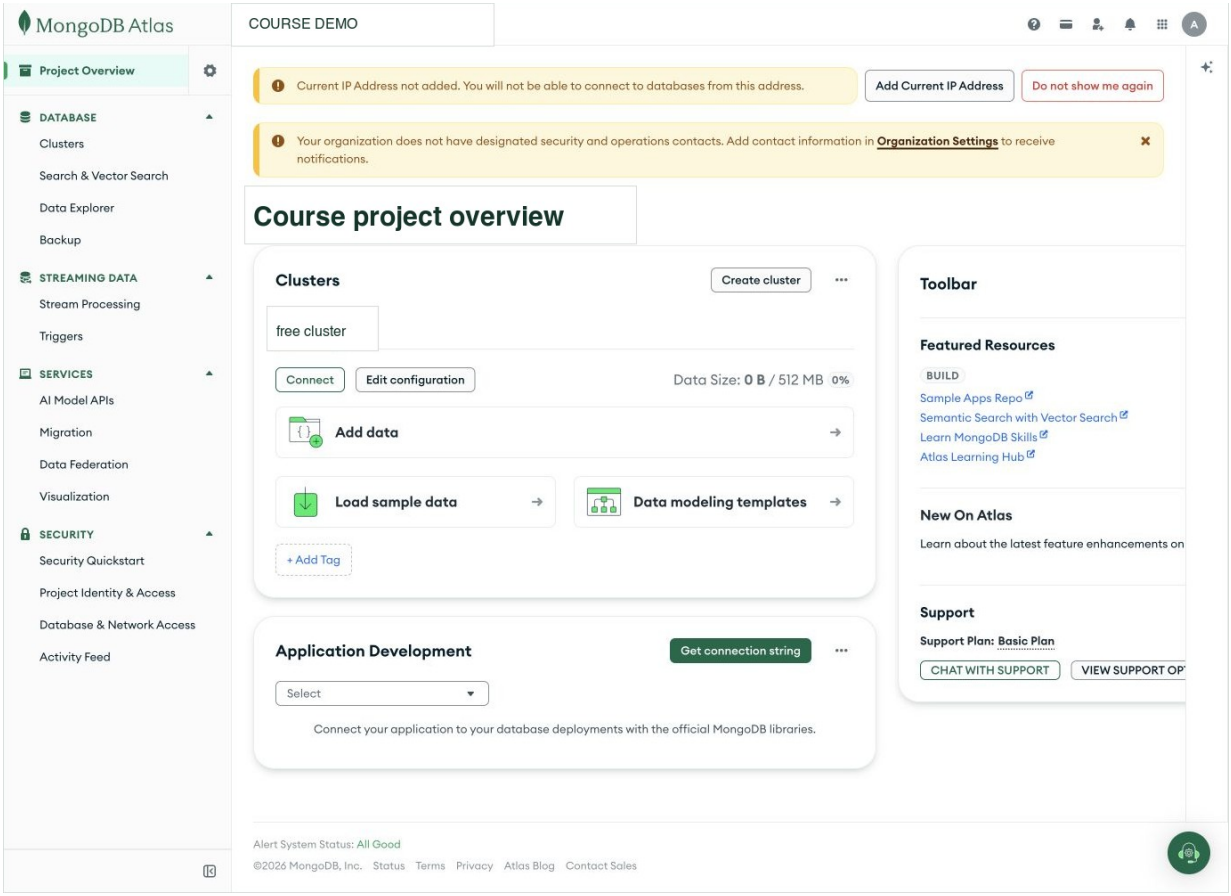



*Figure 1.2: A redacted MongoDB Atlas project overview showing a free deployment and the surrounding platform controls. Interface captured August 25, 2026.*

The labels and menus may change, but the underlying distinctions remain useful:

- **project or organization access** controls who can manage the cloud account;
- **database identity** controls who can connect to the DBMS;
- **network access** controls which clients can reach the service;
- **schema or document design** controls how data is represented; and
- **queries and indexes** control what work the database performs.

## 1.6 What Database Professionals Actually Do

Database work appears under several job titles. A database administrator may work beside backend developers, data engineers, site reliability engineers, security analysts, and cloud support staff. The boundaries differ by workplace, but the technical work commonly includes the following.

### 1.6.1 Model Data

Choose tables, documents, keys, relationships, types, and constraints that match the facts the application needs to store.

### 1.6.2 Manage Access

Create roles and users, grant only necessary privileges, protect credentials, and review who can read or change sensitive data.

### 1.6.3 Change Schemas and Data Safely

Plan migrations, preserve compatibility, use transactions where appropriate, and avoid losing or misinterpreting existing data.

### 1.6.4 Investigate Performance

Read query plans, measure expensive work, design indexes for real queries, and consider the cost of maintaining those indexes.

### 1.6.5 Coordinate Concurrent Work

Understand what happens when many sessions read and write at the same time. Diagnose blocking, deadlocks, and transaction behavior without guessing.

### 1.6.6 Prepare for Failure

Create backups or exports, rehearse restores, understand replication and failover, and decide how much data loss or downtime an application can tolerate.

### 1.6.7 Automate and Communicate

Use scripts, notebooks, version control, logs, and runbooks so recurring work is consistent and understandable to the next person.

This book develops all of these areas at an introductory level.

## 1.7 The Same Ticket in Two Database Models

A relational design might separate users and tickets into tables:

The SQL below illustrates definitions. It is a small inspectable example, not the full Metro Support setup used in later chapters.

Listing 2. SQL

```sql
CREATE TABLE users (
    user_id      integer PRIMARY KEY,
    display_name text NOT NULL
);

CREATE TABLE tickets (
    ticket_id    integer PRIMARY KEY,
    requester_id integer NOT NULL REFERENCES users(user_id),
    category     text NOT NULL,
    priority     text NOT NULL,
    status       text NOT NULL
);
```

The foreign key states that each `requester_id` must identify an existing user. A query can join the two relations when it needs both ticket and requester data.

A document design might store a bounded requester summary inside a ticket:

Listing 3. JSON

```json
{
  "ticket_id": 1001,
  "requester": {
    "user_id": 101,
    "display_name": "Requester 101"
  },
  "category": "streetlight",
  "priority": "high",
  "status": "open"
}
```

Neither shape is automatically better. The relational design gives the user one central row and joins when needed. The document design can read a ticket and its requester summary together but must decide what happens if the display name changes. Later chapters connect these choices to access patterns, consistency, and growth.

## 1.8 A Practical Troubleshooting Order

Consider the report: “The ticket page spins and then says it could not load.” Do not immediately rewrite a query or create an index. First locate the failing part of the request path.

| Area | Useful first question | Example observation |
|---|---|---|
| Client | Did the browser send a request? | request is absent, pending, or returned an HTTP status |
| Application/API | Did the route run and accept the identity? | application log or authentication error |
| Network | Can this client reach the database endpoint? | DNS, timeout, or allow-list error |
| DBMS | Did the database receive work? | active session, database error, or lock wait |
| Query and data | Did the operation ask for too much or meet an unexpected data shape? | query text, plan, row count, or rejected value |

The purpose of this table is not to produce paperwork. It prevents random changes to several layers at once.

## 1.9 Protect Credentials From the Start

A password, connection string, API key, or private token can grant real access. Never place one in a public repository, shared slide, shared notebook output, or chat message.

Use these habits whenever you work with a database:

- enter secrets only when the program is running;
- use environment variables or the notebook's password prompt;
- keep `.env` files out of Git;
- remove secrets from outputs before sharing work; and
- rotate a credential immediately if it is exposed.

The Metro Support examples use fictional people and reserved example domains, not real student or customer records.

## 1.10 GitHub as a Code Notebook

GitHub stores versions of text files and notebooks. It can hold SQL, JSON, Markdown, and small scripts. Its browser editor lets you create and revise these files without using a command line.

To create a Markdown file in the browser:

1. open a repository you own or have permission to edit;
2. choose **Add file** and **Create new file**;
3. enter a path such as `notes/app_database_map.md`;
4. write in the editor and check the **Preview** tab; and
5. choose **Commit changes** with a short description.

Markdown uses simple punctuation for structure:

**Listing 4. Markdown**

```
# Music App Database Map

## Data the app stores

- users
- playlists
- songs

## Questions the database must answer

1. Which songs are in this playlist?
2. Which playlists belong to this user?
```

The commit records a version of the file, so you can inspect later changes or return to an earlier explanation.

## 1.11 Worked Example: Trace One Request

**Action:** An agent changes ticket 1001 from `open` to `in_progress`.

**Possible request path:**

1. the browser sends the ticket identifier and new status;
2. the API identifies the agent;
3. an authorization rule permits agents to update assigned tickets;
4. PostgreSQL begins a transaction;
5. a constraint checks that `in_progress` is an allowed status;
6. PostgreSQL updates the row and commits; and
7. the API returns the new ticket state.

This one action touches identity, authorization, a transaction, a constraint, stored data, and application behavior. The remaining chapters study those pieces one at a time and then reconnects them.

## 1.12 Study and Practice

Use the missing-request case to trace a read path and a write path. Distinguish the information stored by the database from the information selected by a query and displayed by an application. Chapter 2 develops the relational operations behind that distinction.

## 1.13 Chapter Summary

- A database is stored information; a DBMS is the software that manages it; a cloud platform operates the DBMS with additional services.
- PostgreSQL is the relational DBMS used by Supabase. MongoDB is the document DBMS used by Atlas.
- A user action travels through several layers before it becomes stored data.
- Database work includes modeling, access control, schema change, concurrency, performance, recovery, automation, and communication.
- SQL and document models organize related facts differently; the workload determines which tradeoffs matter.
- Credentials must never be committed or shared.

## 1.14 Check Your Understanding

1. Explain the difference between PostgreSQL and Supabase.
2. Explain the difference between MongoDB and Atlas.
3. Name three places a request can fail before the DBMS executes a query.
4. Why might a foreign key be useful in the ticket example?
5. What tradeoff appears when a user name is embedded in a ticket document?
6. Name four kinds of work performed by database professionals.

## 1.15 Further Reading

- PostgreSQL, “About PostgreSQL”
- Supabase architecture

- MongoDB Atlas documentation
- GitHub Skills, “Introduction to GitHub”
- O*NET, Database Administrators

# 2 Relational Operations Become Testable SQL

## 2.1 Operating Question

How can a query answer a real question while making it possible to check whether the answer is trustworthy?

A database can execute exactly the query we wrote while answering a different question from the one we intended. This is common when SQL has become a collection of remembered keywords rather than a model of how rows combine. We will rebuild that model, starting with a few visible facts and ending with queries whose behavior we can explain before and after execution. The aim is not speed at typing SQL. It is being able to find a mistake that the SQL parser cannot find.

## 2.2 Learning Outcomes

After this chapter, you can:

- explain rows, columns, keys, and relationships in a relational model;
- translate selection, projection, rename, product, join, union, and difference between relational-algebra notation and plain language;
- use `SELECT`, `WHERE`, `ORDER BY`, `JOIN`, `GROUP BY`, and aggregate functions;
- review subqueries, common table expressions, and safe `INSERT`, `UPDATE`, and `DELETE` patterns;
- reason about `NULL` without treating it as zero or an empty string;
- distinguish the written order of a query from its logical processing order; and
- verify a query with counts, boundary cases, and comparison queries.

## 2.3 A Relation Represents One Kind of Fact

### 2.3.1 The Instance Used in This Chapter

The worked SQL uses the complete Metro Support baseline: **8 users, 12 tickets, and 21 ticket events**. Appendix C prints the complete schema and seed data and explains how to start a practice database. Run that setup before these queries. A new web-editor tab may use a new database session, so the examples use names such as `metro_support.tickets` rather than relying on an earlier `SET search_path` command.

The companion SQL review notebook starts with a smaller 4-user/6-ticket/9-event instance to make the first demonstrations easy to inspect, then provides the full baseline used here. A different instance can produce different answers to the same correct query. Always identify which data you are reasoning about before comparing counts.

The relational model organizes data into relations commonly presented as tables. A row represents one occurrence, and each column represents one attribute of that kind of occurrence. A key identifies a row. A foreign key records a relationship by referring to a key in another table.

Metro Support separates:

- `users`: one row per person or staff account;
- `tickets`: one row per support request; and
- `ticket_events`: one row per recorded event in a ticket's history.

This separation avoids repeating a user's name and email in every ticket event. It also creates work: a query must join related rows when the answer needs facts from more than one table.

### 2.3.2 Schema, Instance, Domain, and Key

A *schema* describes the structure: for example, tickets have an identifier, requester, status, and opening time. An *instance* is the collection of rows that exists at a particular moment. The instance changes when a ticket arrives; the schema usually does not. Mixing up the two leads to rules that happen to fit today's data but do not describe tomorrow's valid data.

A *domain* describes the permitted kind of value for an attribute. Some domain information comes from a type such as integer or timestamp, and some comes from a rule such as the set of allowed statuses. A ticket identifier and an agent identifier may both use integers without meaning the same thing. Their names, keys, and relationships give them different roles.

A key is not simply a convenient-looking column. It must distinguish permitted rows. A person's current display name is a poor identifier if two people can share it or one person can change it. An assigned `user_id` allows relationships to remain stable through a name change. Chapter 3 will separate the mathematical idea of a candidate key from the particular key chosen as the SQL primary key.

## 2.4 Relational Algebra Gives Us a Reasoning Vocabulary

Relational algebra describes how one or more input relations become an output relation. The notation is less important than the habit of decomposing a question into operations.

Assume $T$ is the tickets relation and $U$ is the users relation. Let $\varphi$ and $\theta$ stand for predicates: expressions that evaluate to true or false for a tuple or tuple pair.

| Operation | Conventional notation | Plain-language job |
|---|---|---|
| selection | $\sigma_{\varphi}(T)$ | keep tuples from $T$ that satisfy predicate $\varphi$ |
| projection | $\pi_{a_1,\ldots,a_k}(T)$ | keep attributes $a_1$ through $a_k$ |
| rename | $\rho_S(T)$ | give relation $T$ the name $S$ |
| Cartesian product | $T \times U$ | pair every tuple in $T$ with every tuple in $U$ |
| theta join | $T \bowtie_{\theta} U$ | pair tuples that satisfy relationship predicate $\theta$ |
| union | $R \cup S$ | include tuples that occur in either compatible relation |

| Operation | Conventional notation | Plain-language job |
|---|---|---|
| intersection | $R \cap S$ | include tuples that occur in both compatible relations |
| difference | $R - S$ | include tuples in $R$ but not in compatible relation $S$ |

The subscripts carry the details of an operation. In $\sigma_{\varphi}(T)$, $\varphi$ is the row condition. In $\pi_{a_1,\ldots,a_k}(T)$, the subscript is the attribute list. The letters $R$, $S$, $T$, and $U$ name relations; they are not SQL keywords.

Classical projection selects attributes. SQL's expression list can also compute new values, which is often called generalized projection. Likewise, grouping and outer joins are useful extensions beyond the small classical algebra listed here. Knowing the boundary prevents us from forcing every SQL feature into a notation that was introduced for a simpler mathematical model.

| Relational idea | Common SQL expression |
|---|---|
| selection | `WHERE` |
| projection | the expression list after `SELECT` |
| rename | `AS` |
| Cartesian product | `CROSS JOIN` |
| theta/equi-join | `JOIN ... ON` |
| union | `UNION` |
| intersection | `INTERSECT` |
| difference | `EXCEPT` |

Union, intersection, and difference require *union-compatible* inputs: the same number of attributes with corresponding domains that can be compared. SQL adds practical details such as column names, data-type conversion, duplicate handling, and `NULL` semantics.

A theta join can be understood as a selection applied to a Cartesian product:

$$T \bowtie_{\theta} U = \sigma_{\theta}(T \times U)$$

This identity explains why a missing join predicate is dangerous. Without $\theta$, the system retains the full product rather than only meaningful pairs.

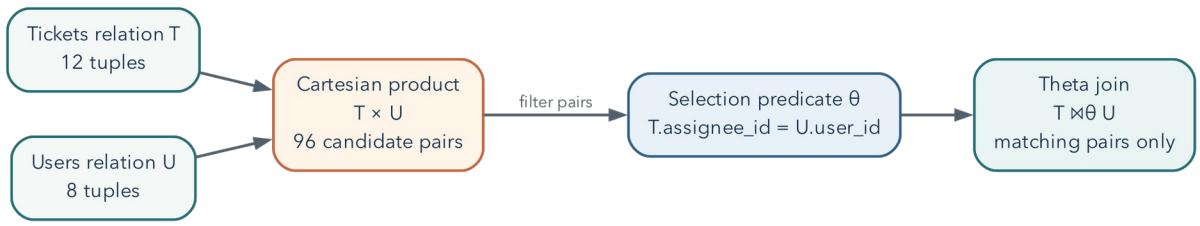



*Figure 2.1: A theta join can be reasoned about as a Cartesian product followed by a selection predicate.*

### 2.4.1 Example Translation

**Question:** Which high-priority tickets are still active, and what are their subjects?

Reasoning:

1. select tickets where priority is high and status is active;
2. project the ticket identifier, subject, and status; and
3. sort for presentation, which is useful SQL behavior but not a core relational- algebra operation.

Let $A$ be the set of active status values:

$$A = \{\text{'new'}, \text{'open'}, \text{'in_progress'}\}.$$

Define the predicate $\varphi$ and then apply selection followed by projection:

$$\varphi = (\text{priority} = \text{'high'}) \wedge (\text{status} \in A)$$

$$Q = \pi_{\text{ticket_id}, \text{subject}, \text{status}}\left(\sigma_{\varphi}(\text{tickets})\right)$$

Listing 5. SQL

```sql
SELECT ticket_id, subject, status
FROM metro_support.tickets
WHERE priority = 'high'
  AND status IN ('new', 'open', 'in_progress')
ORDER BY ticket_id;
```

### 2.4.2 Algebra and SQL Are Related, Not Identical

In the full baseline, the query above returns ticket IDs **1001 and 1011**. Ticket 1003 is urgent but already resolved, so it does not belong. Ticket 1005 is high priority but resolved, so it does not belong either. The excluded examples are as important as the included ones: they test that both parts of the question have survived the translation.

Classical relational algebra uses sets: duplicate tuples do not occur, and ordering is not part of a relation. SQL normally uses *bag* or multiset semantics, so duplicate result rows can occur unless a key, grouping operation, `DISTINCT`, or set operator changes that. SQL also includes `NULL` and three-valued logic, which require reasoning beyond classical algebra.

This difference explains two important habits: always state the expected result grain, and never assume row order without `ORDER BY`.

## 2.5 Start With a Precise Question

"Show tickets" is vague. "List open or in-progress high-priority tickets, newest first, with the assigned agent's name" is testable. It identifies:

- the unit: one ticket per result row;
- the filter: selected statuses and high priority;
- the order: newest first;
- the relationship and attributes: ticket to assigned user, including the agent's name.

Before writing SQL, state the expected grain: what does one result row mean? Unexpected duplicates often reveal that the query changed grain.

## 2.6 Select, Filter, and Sort

Listing 6. SQL

```sql
SELECT ticket_id, subject, status, opened_at
FROM metro_support.tickets
WHERE status IN ('open', 'in_progress')
ORDER BY opened_at DESC;
```

`SELECT` names the output expressions. `FROM` identifies the source. `WHERE` removes rows that do not meet the predicate. `ORDER BY` controls presentation. Without `ORDER BY`, row order is not guaranteed, even when repeated runs appear consistent.

When two tickets share an opening time, that one-column sort does not determine their relative order. Add `ticket_id` as a tie-breaker if repeatable ordering matters: `ORDER BY opened_at DESC, ticket_id`. A stable order is particularly important when an application retrieves one page at a time.

### 2.6.1 Parentheses Preserve the Question

Suppose the question changes to active tickets with **high or urgent** priority. Write the two priority alternatives together:

Listing 7. SQL

```sql
SELECT ticket_id, priority, status
FROM metro_support.tickets
WHERE priority IN ('high', 'urgent')
  AND status IN ('new', 'open', 'in_progress')
ORDER BY ticket_id;
```

Without the grouping, `priority = 'high' OR priority = 'urgent' AND ...` means high priority *regardless of status*, or urgent priority with the additional status condition. SQL evaluates `AND` more tightly than `OR`. On the baseline, that mistake admits resolved high-priority tickets 1005 and 1008. The statement is legal SQL; the error is in the logical claim we encoded.

Use parentheses even when you know the precedence rule if they make that claim easier for the next reader to see. Choosing a compact expression is useful only when it preserves both meaning and readability.

Avoid `SELECT *` in durable work. Explicit columns document intent, reduce unnecessary transfer, and make a query less vulnerable to later schema changes.

## 2.7 SQL's Logical Order

SQL is written in a human-readable order but reasoned about approximately as:

1. `FROM` and `JOIN` build the source rows.
2. `WHERE` filters individual rows.
3. `GROUP BY` forms groups.
4. aggregate calculations summarize each group.
5. `HAVING` filters groups.
6. `SELECT` produces output expressions.

7. `DISTINCT`, if requested, removes duplicate output rows.
8. `ORDER BY` sorts the result.
9. `LIMIT` restricts the returned rows.

This explains why a `SELECT` alias is often unavailable in `WHERE`: the filter is logically evaluated before the output alias exists.

This is a reasoning order, not a literal instruction to the storage engine to build every intermediate table. An optimizer may push a safe filter earlier, choose a different join algorithm, or avoid sorting by reading an index in order. Those choices must preserve the query's semantics. Chapters 7 and 11 return to this distinction between the question and its physical execution.

## 2.8 Join Related Facts

Listing 8. SQL

```sql
SELECT
    t.ticket_id,
    t.subject,
    u.display_name AS assignee_name
FROM metro_support.tickets AS t
LEFT JOIN metro_support.users AS u
    ON u.user_id = t.assignee_id
ORDER BY t.ticket_id;
```

An `INNER JOIN` returns only matching pairs. A `LEFT JOIN` preserves every row from the left side and supplies `NULL` for missing right-side values. Because two tickets are unassigned, an inner join would silently remove them. The join type is therefore a statement about the question, not just syntax.

### 2.8.1 A Join Is a Filtered Product

A Cartesian product pairs every row in one relation with every row in another. With 12 tickets and 8 users, the product has 96 pairs. An equi-join keeps only pairs whose key values match. Thinking of a join as product plus selection makes a missing join condition easier to recognize.

Formally, if $|T|$ denotes the number of tuples in $T$, then:

$$|T \times U| = |T||U|.$$

For the baseline data, $|T| = 12$ and $|U| = 8$, so $|T \times U| = 96$. A selective join predicate should discard most of those candidate pairs.

### 2.8.2 Diagnose Duplicate Rows

Joining one ticket to many events changes the grain from one row per ticket to one row per matching event:

Listing 9. SQL

```sql
SELECT t.ticket_id, t.subject, e.event_type, e.event_at
FROM metro_support.tickets AS t
JOIN metro_support.ticket_events AS e
    ON e.ticket_id = t.ticket_id
WHERE t.ticket_id = 1003
ORDER BY e.event_at;
```

Three rows for ticket 1003 are correct because it has three events. The repeated ticket identifier refers to distinct events, not three separate tickets.

`DISTINCT` removes only rows identical across the selected expressions. If event type or time differs, those three rows remain distinct. If we project only the ticket ID, `DISTINCT` can deliberately answer "which tickets had a matching event?" But it does not turn an event-level sum into a ticket-level sum. We must choose the unit of analysis before choosing the aggregate, especially when the result could influence workload or service decisions.

## 2.9 `NULL` Means Missing or Inapplicable

`NULL` is not zero, blank text, or a normal value. Ordinary equality comparisons with `NULL` produce unknown rather than true. In a `SELECT` query, compare these two filter clauses:

**Listing 10. SQL**

```sql
-- Incorrect: this does not find unassigned tickets.
WHERE assignee_id = NULL

-- Correct.
WHERE assignee_id IS NULL
```

Three-valued logic includes true, false, and unknown. This matters for filters, constraints, joins, and aggregates. `count(*)` counts rows. `count(assignee_id)` counts only rows where that expression is not null.

The essential truth table for `AND` is:

| $p$ | $q$ | $p \wedge q$ |
|---|---|---|
| TRUE | TRUE | TRUE |
| TRUE | UNKNOWN | UNKNOWN |
| FALSE | UNKNOWN | FALSE |
| UNKNOWN | UNKNOWN | UNKNOWN |

A `WHERE` clause keeps only rows for which its predicate is `TRUE`. Both `FALSE` and `UNKNOWN` are filtered out.

### 2.9.1 Why `NOT IN` Can Surprise You

The assigned-agent column contains nulls. Consequently, asking for users whose ID is `NOT IN (SELECT assignee_id FROM metro_support.tickets)` is not the same as asking whether no ticket has that user's ID. A comparison against the null member is unknown, and negating unknown does not make it true. This can eliminate every otherwise unmatched user from the result.

An existence test states the intended question more directly:

Listing 11. SQL

```sql
SELECT u.user_id, u.display_name
FROM metro_support.users AS u
WHERE NOT EXISTS (
    SELECT 1
    FROM metro_support.tickets AS t
    WHERE t.assignee_id = u.user_id
)
ORDER BY u.user_id;
```

For each candidate user, the inner query asks whether a matching assignment exists. The selected constant `1` has no business meaning; `EXISTS` cares whether there is a row. Null assignments simply do not match an ordinary user ID. This query includes residents and other non-agent accounts because it asks about **all users**. Add a role condition only if the intended population is narrower.

## 2.10 Group and Aggregate

Listing 12. SQL

```sql
SELECT
    category,
    count(*) AS ticket_count,
    count(closed_at) AS closed_timestamp_count
FROM metro_support.tickets
GROUP BY category
ORDER BY ticket_count DESC, category;
```

After grouping, every selected expression must either identify the group or summarize it. The result grain is now one row per category.

Use `HAVING` for a condition on a group:

Listing 13. SQL

```sql
SELECT category, count(*) AS ticket_count
FROM metro_support.tickets
GROUP BY category
HAVING count(*) >= 3;
```

## 2.11 Worked Example: Staff Workload Including Zero Counts

**Question:** For every agent or supervisor, how many active tickets are assigned?

First define “active” as `new`, `open`, or `in_progress`. The result should have one row per staff member in those two roles, including supervisor Agent 203, who currently has none. Analysts and residents are outside this report’s stated population. This explicit definition matters: the baseline’s two agents both have active work, so an agents-only example would not show the zero-count case.

Listing 14. SQL

```sql
SELECT
    u.user_id,
    u.display_name,
    count(t.ticket_id) AS active_ticket_count
FROM metro_support.users AS u
LEFT JOIN metro_support.tickets AS t
    ON t.assignee_id = u.user_id
   AND t.status IN ('new', 'open', 'in_progress')
WHERE u.role IN ('agent', 'supervisor')
GROUP BY u.user_id, u.display_name
ORDER BY active_ticket_count DESC, u.display_name;
```

Why is the ticket-status condition in `ON` rather than `WHERE`? A `WHERE` condition on `t.status` would remove the null-extended row for an agent with no matching active ticket, making the left join behave like an inner join for this purpose.

Why `count(t.ticket_id)` instead of `count(*)`? The left join produces one row for an agent with no match, but `t.ticket_id` is null in that row. Counting the ticket identifier correctly yields zero.

The expected result is Agent 201 with 3, Agent 202 with 2, and Agent 203 with 0. Those counts sum to 5, while the baseline has 7 active tickets. The two unassigned requests, 1004 and 1009, are not owned by any staff member and therefore do not appear in this *staff* grouping. Nothing was lost from the database. A complete operational dashboard could present the five assigned requests and a separate two-request unassigned queue.

This is a useful limit on the phrase “preserves missing rows.” A left join preserves the chosen left population, not every record in every participating table. In the earlier query, the left population was tickets. Here it is staff. Changing which side we preserve changes the question.

## 2.12 Subqueries and CTEs Name Intermediate Relations

A table subquery produces a result used as an input relation. A scalar subquery produces one value and must not return multiple rows. An `EXISTS` subquery answers a yes/no existence question. A common table expression, or CTE, gives an intermediate result a name. These are related tools, not interchangeable syntax.

Listing 15. SQL

```sql
WITH active_tickets AS (
    SELECT ticket_id, assignee_id, priority
    FROM metro_support.tickets
    WHERE status IN ('new', 'open', 'in_progress')
),
agent_counts AS (
    SELECT assignee_id, count(*) AS active_count
    FROM active_tickets
    WHERE assignee_id IS NOT NULL
    GROUP BY assignee_id
)
SELECT u.display_name, a.active_count
FROM agent_counts AS a
JOIN metro_support.users AS u
    ON u.user_id = a.assignee_id
ORDER BY a.active_count DESC, u.display_name;
```

Read each CTE as a named step in relational reasoning. A CTE can improve clarity, but it is not automatically faster. PostgreSQL’s optimizer and version determine how it is planned.

The CTE example above starts from assigned active tickets. It therefore returns Agent 201 and Agent 202, but not Agent 203. That is intentional for this particular query; it is not equivalent to the preceding all-staff report. To include zero-workload staff, start with the staff relation and left join the CTE counts, using `COALESCE(active_count, 0)` for an absent group. Naming an intermediate result improves readability, but it does not excuse checking who is included.

Set operators express union and difference when inputs have compatible columns:

Listing 16. SQL

```sql
-- Users who requested a ticket but are not assigned to any ticket.
SELECT requester_id AS user_id
FROM metro_support.tickets
EXCEPT
SELECT assignee_id
FROM metro_support.tickets
WHERE assignee_id IS NOT NULL;
```

`UNION` removes duplicate rows. `UNION ALL` preserves them and avoids duplicate- elimination work when set semantics are not required.

## 2.13 Review Safe Data Changes

Queries read state; data-manipulation statements change it.

Use the following insertion as a rehearsal. The transaction rolls back the new event, so rerunning the example does not accumulate records or collide with the same event key. Begin from the baseline, where event 5099 is unused.

Listing 17. SQL

```sql
BEGIN;
INSERT INTO metro_support.ticket_events (
    event_id, ticket_id, actor_id, event_type, old_status, new_status, note, event_at
) VALUES (
    5099, 1001, 201, 'note_added', 'open', 'open',
    'Replacement lamp requested', now()
)
RETURNING event_id, ticket_id, event_type;
ROLLBACK;
```

Before an `UPDATE` or `DELETE`, write the same predicate as a `SELECT` and inspect the target keys:

Listing 18. SQL

```sql
SELECT ticket_id, priority
FROM metro_support.tickets
WHERE category = 'streetlight'
  AND priority = 'medium';

BEGIN;

UPDATE metro_support.tickets
SET priority = 'high'
WHERE category = 'streetlight'
  AND priority = 'medium'
RETURNING ticket_id, priority;

ROLLBACK;
```

`RETURNING` shows affected rows. A transaction creates a decision point. Neither replaces a correct predicate or independent verification.

Use the same discipline for deletion:

This deletion example is a predicate-and-transaction template. If you just ran the rolled-back insertion above, event 5099 does not exist, so it correctly deletes zero rows. To observe one deletion, create that disposable event inside this same transaction before the `DELETE`. Never change the predicate to a broad one merely to make the affected-row count nonzero.

**Listing 19. SQL**

```sql
BEGIN;

DELETE FROM metro_support.ticket_events
WHERE event_id = 5099
RETURNING event_id, ticket_id;

-- Verify the intended row and then choose COMMIT or ROLLBACK.
ROLLBACK;
```

Never practice broad destructive statements in a shared or production database.

## 2.14 Verification Is a Second Query or Reasoning Path

Use at least one of these methods:

- **Known total:** compare grouped counts with the total number of source rows.
- **Boundary case:** inspect an unassigned ticket, a null value, or a user with no match.
- **Simpler query:** verify one result category with a direct filtered count.
- **Uniqueness check:** compare total rows with distinct keys at the expected grain.
- **Manual sample:** trace one identifier across source tables.

For the workload query, verify Agent 201 directly:

**Listing 20. SQL**

```sql
SELECT ticket_id, status
FROM metro_support.tickets
WHERE assignee_id = 201
  AND status IN ('new', 'open', 'in_progress');
```

The verification query is intentionally less abstract. Independent reasoning is more valuable than repeating the same logic in a different format.

## 2.15 Common Misconceptions

### 2.15.1 "No rows means the query worked"

No rows could mean no data matches, the filter is wrong, a join removed records, or the setup failed. Verify source counts and test one known identifier.

### 2.15.2 "A join connects tables automatically"

SQL joins use the condition you write. A missing or incorrect condition can create a Cartesian product or pair unrelated records.

### 2.15.3 "`DISTINCT` fixes duplicates"

`DISTINCT` removes identical output rows. It does not fix an incorrect grain or relationship and may hide a modeling or join error.

## 2.16 Practice

The following problem brings filtering, null reasoning, joins, and grouping together. Use the full Metro Support baseline described at the start of the chapter and verify the result against simpler counts.

First express the required operations in plain language or relational-algebra notation. Then write a query that returns one row per neighborhood with:

- the number of resident users;
- the number of tickets they requested; and
- the most recent ticket opening time.

Before running it, predict which joins are required and whether a left join is necessary. After running it, verify one neighborhood with a simpler query.

## 2.17 Retrieval and Transfer

1. How do selection and projection differ?
2. Why is a join related to a Cartesian product?
3. What does one row in a query result represent?
4. Why can a left join return more rows than the left table?
5. What is the difference between `WHERE` and `HAVING`?
6. Why does `column = NULL` not work as expected?
7. When does `UNION ALL` express the intended result better than `UNION`?
8. What should you do before running an `UPDATE` or `DELETE`?
9. A report suddenly doubles its row count after an events table is joined. What should you inspect before adding `DISTINCT`?

## 2.18 Further Reading

- PostgreSQL `SELECT`
- PostgreSQL table expressions and joins
- PostgreSQL aggregate functions
- *Database Design - 2nd Edition*, Chapter 8

# 3 A Schema Protects Meaning

## 3.1 Operating Question

Which invalid states should the database refuse, even when an application has a bug or a user imports a bad file?

Imagine receiving three CSV files that produce the right report today. That does not yet tell us whether tomorrow's import can contain two records for the same ticket, an impossible date, or a request assigned to a person who does not exist. A schema turns selected assumptions into rules the DBMS can enforce. Choosing those rules requires understanding the facts, not simply selecting a type from a menu.

## 3.2 Learning Outcomes

After this chapter, you can:

- use PostgreSQL schemas as namespaces and permission boundaries;
- choose basic data types, primary keys, and foreign keys;
- explain how dependencies and normalization reduce inconsistent copies of facts;
- apply `NOT NULL`, `UNIQUE`, `CHECK`, and referential actions intentionally;
- distinguish an integrity constraint from an index; and
- inspect database metadata instead of relying on memory.

## 3.3 "Schema" Has Two Related Meanings

A database schema can mean the overall structure of data: tables, columns, relationships, constraints, and indexes. In PostgreSQL, a *schema* is also a named namespace inside a database. `metro_support.tickets` identifies the `tickets` table in the `metro_support` namespace.

Namespaces help:

- separate application objects from extensions or shared utilities;
- avoid name collisions;
- organize permissions; and
- make object ownership explicit.

PostgreSQL resolves unqualified names through `search_path`. In durable scripts, schema-qualified names reduce ambiguity:

**Listing 21. SQL**

```sql
SELECT ticket_id, status
FROM metro_support.tickets;
```

## 3.4 Dependencies Explain Why We Separate Tables

Suppose a ticket table repeats the requester's current name and email on every row. Requester 101 has three tickets. When her email changes, one row is updated and two are not. The database now offers conflicting answers to the question "what is Requester 101's current email?" The problem is not that repetition is visually untidy. The same fact has several independently writable representations.

A *functional dependency* expresses a rule about which attributes determine others. Under our account model, one `user_id` determines one current display name and email:

$$\text{user_id} \rightarrow \{\text{display_name}, \text{email}\}.$$

This means that two valid rows with the same user ID cannot disagree about those attributes. It is a rule about every permitted instance, not a pattern proved by today's eight rows. If every ticket in a tiny sample happens to have a different category, that coincidence does not make category a valid ticket identifier.

A *superkey* determines the entire row. A *candidate key* is a minimal superkey: removing any attribute from it would lose that property. We choose one candidate as the primary key. Additional business identities may be protected with unique constraints. The word minimal concerns the included attributes, not whether the numeric values are small.

### 3.4.1 Insertion, Update, and Deletion Anomalies

Repeating current account details in a ticket table creates several problems. An update can change one copy and leave others behind. Inserting an account may require inventing a ticket merely to have somewhere to store the account's attributes. Deleting the person's last ticket may delete the only remaining record of the account. These are update, insertion, and deletion anomalies.

We separate `users` from `tickets` so each current account fact has one authoritative location. Tickets store the identifying reference. We can join the relations when a report needs both. That join has a computational cost, but it also reconstructs a combined view without making every combined view a new independently writable copy.

### 3.4.2 A Brief Normalization Review

First normal form is conventionally taught as using one value from the declared domain in each attribute rather than repeating groups such as `event1`, `event2`, and `event3`. A variable-length history should not require adding another column every time an event occurs. PostgreSQL can legitimately store arrays or `jsonb`; their availability does not decide whether a nested value is the best way to represent a particular relationship.

Second normal form addresses partial dependencies on a composite candidate key. For a table keyed by `(ticket_id, tag_id)`, a tag's current label depends on `tag_id`, not on the whole assignment pair. Store the label in a tag relation rather than repeating it for every ticket that uses the tag.

Third normal form further limits dependencies that let non-key facts determine other non-key facts. In the repeated-requester example, `ticket_id` determines `requester_id`, and requester identity determines the current email. Keeping the email in `users` avoids storing that dependency repeatedly in `tickets`. The formal 3NF condition is: for every nontrivial dependency $X \rightarrow A$, either $X$ is a superkey or $A$ belongs to a

candidate key. This qualification matters when a relation has several overlapping candidate keys. Boyce-Codd normal form uses the stricter requirement that each nontrivial determinant be a superkey.

The practical goal is to recognize where a current fact has multiple writable copies and explain a sensible decomposition. Decompositions should be *lossless*: joining the pieces through the intended keys must recover the original facts without inventing combinations. In Metro Support, each ticket's non-null requester points to one uniquely identified user, so joining that reference retrieves one requester record per ticket.

Normalization does not forbid every purposeful copy. A ticket may need the address reported **at the time of submission**, even after a resident moves. That historical address is a different fact from the account's current address. A document summary used for faster reads may also copy current data, provided we identify its owner and update policy. Chapters 10 and 14 revisit those deliberate choices. A label such as "denormalized" is not an explanation by itself.

## 3.5 Data Types Express Allowed Representation

A data type is the first integrity decision. Choose a type that represents the domain and supports required operations.

| Need | Common PostgreSQL type | Reasoning |
|---|---|---|
| whole-number identifier | `integer` or `bigint` | numeric identity, efficient equality |
| arbitrary text | `text` | no invented length limit |
| exact money/measurement | `numeric(p,s)` | decimal precision rather than binary approximation |
| yes/no state | `boolean` | avoids multiple spellings |
| instant in time | `timestamptz` | stores an instant and converts display zones |
| calendar date | `date` | no implied time of day |
| structured flexible attribute | `jsonb` | queryable JSON when a relational column is not appropriate |

Do not choose a type only because the current sample fits. Ask what values should exist and what operations must be correct.

An instant and a calendar label are different facts. `timestamptz` lets us compare the instant at which two events occurred even if clients display different time zones. It does not retain the user's original zone name. If the application must schedule "9 a.m. in New York" on future dates, keep the relevant zone or scheduling rule too. A ZIP code may look numeric but is often better stored as text because leading zeroes are significant and arithmetic on the value has no meaning.

## 3.6 Keys Identify and Connect Facts

A **primary key** uniquely identifies a row and is not null. A **foreign key** requires a value to match a key in another table, unless the foreign-key column is allowed to be null.

The following reduced definition illustrates those rules. The complete baseline already has a `tickets` table, so do not execute this `CREATE TABLE` again after loading it.

Listing 22. SQL

```sql
CREATE TABLE metro_support.tickets (
    ticket_id integer PRIMARY KEY,
    requester_id integer NOT NULL
        REFERENCES metro_support.users(user_id),
    assignee_id integer
        REFERENCES metro_support.users(user_id)
);
```

The nullable `assignee_id` represents a real state: a ticket may be unassigned. The non-null `requester_id` states that every ticket must have a known requester.

The assignee foreign key establishes that a user exists, not that the user has role `agent`. Assigning resident 101 would satisfy this particular reference. That is a useful example of a valid database state that may still violate a business rule we have not encoded. A richer design could separate eligible staff into a referenced relation. Application authorization and Chapter 6's permissions address other parts of the problem; none should be assumed from the foreign key alone.

### 3.6.1 Referential Actions Are Business Decisions

What should happen if a user row is deleted?

- `RESTRICT` or `NO ACTION`: refuse deletion while dependent rows exist.
- `CASCADE`: delete or update dependent rows automatically.
- `SET NULL`: preserve the dependent row but remove the reference.

There is no universally correct action. Cascading a requester deletion into all historical tickets would probably destroy important records. An anonymization workflow or restricted deletion may be safer.

## 3.7 Constraints Reject Invalid States

### 3.7.1 `NOT NULL`

Use when the fact must exist at row creation. Do not mark a field optional only because imports are inconvenient.

### 3.7.2 `UNIQUE`

Use when duplicate values would violate identity or a business rule:

Listing 23. SQL

```sql
-- The supplied baseline already declares email NOT NULL UNIQUE.
-- Inspect the existing rule instead of adding a second copy.
SELECT conname, pg_get_constraintdef(oid) AS definition
FROM pg_constraint
WHERE conrelid = 'metro_support.users'::regclass
  AND contype = 'u';
```

By default, PostgreSQL uniqueness does not treat two nulls as the same value. The baseline avoids that issue for email by also requiring a value. If a different design needs null to participate in uniqueness as one shared

missing value, PostgreSQL 15 and later support `UNIQUE NULLS NOT DISTINCT`. Choose the intended meaning rather than assuming every SQL system makes the same choice.

### 3.7.3 `CHECK`

Use a Boolean rule about one row:

Listing 24. SQL

```sql
ALTER TABLE metro_support.tickets
ADD CONSTRAINT tickets_priority_allowed
CHECK (priority IN ('low', 'medium', 'high', 'urgent'));
```

Listing 25. SQL

```sql
-- This equivalent date rule already exists in the supplied baseline.
-- The statement illustrates its definition; do not add a redundant copy.
ALTER TABLE metro_support.tickets
ADD CONSTRAINT tickets_close_after_open
CHECK (closed_at IS NULL OR closed_at >= opened_at);
```

Constraints should have meaningful names. An error mentioning `tickets_priority_allowed` is more useful than one mentioning a generated name.

A `CHECK` rejects false, but accepts true **or unknown**. Thus `CHECK (score >= 0)` does not prohibit a null score. Add `NOT NULL` if the value itself is required. This differs from `WHERE`, which retains only true. The same three-valued logic has different consequences at a filter and at a constraint boundary.

### 3.7.4 Constraints Are Not a Complete Workflow Engine

A row-level `CHECK` can keep `closed_at` after `opened_at`. It cannot easily prove that every status transition followed a multi-row workflow or external approval. Use the database for durable invariants and choose application or procedural logic for process rules that cross rows, time, or services.

PostgreSQL does not support cross-table queries inside a `CHECK` as a general integrity mechanism. A function that happens to look at another table does not make the dependency safely maintained when that other table changes. Use foreign keys or another explicitly coordinated mechanism for cross-row requirements. The PostgreSQL constraints reference documents these null and cross-row distinctions.

## 3.8 An Index Is an Access Structure, Not the Rule Itself

An index stores an organized path to rows. It can accelerate matching, joining, sorting, or uniqueness checks. It also consumes storage and adds work to inserts, updates, deletes, backups, and maintenance.

Listing 26. SQL

```sql
CREATE INDEX tickets_status_opened_idx
ON metro_support.tickets (status, opened_at DESC);
```

This index may support a workload that filters status and requests recent rows. It is not automatically useful for every query involving either column. Column order, selectivity, table size, and the query plan matter. Chapter 7 develops the facts and measurements needed to decide.

A primary key or `UNIQUE` constraint normally creates a supporting unique index, but the concepts differ. The constraint states a rule. The index is a mechanism PostgreSQL can use to enforce or access data.

## 3.9 Managed PostgreSQL Still Leaves Schema Design to You

Supabase operates PostgreSQL infrastructure and adds services, but it cannot know what `priority` values your system should accept or whether deleting a user should remove tickets. The platform may expose a table editor, yet the SQL definition is the durable artifact.

Use the dashboard to inspect and learn. Preserve schema changes in ordered SQL files so another environment can be rebuilt and reviewed.

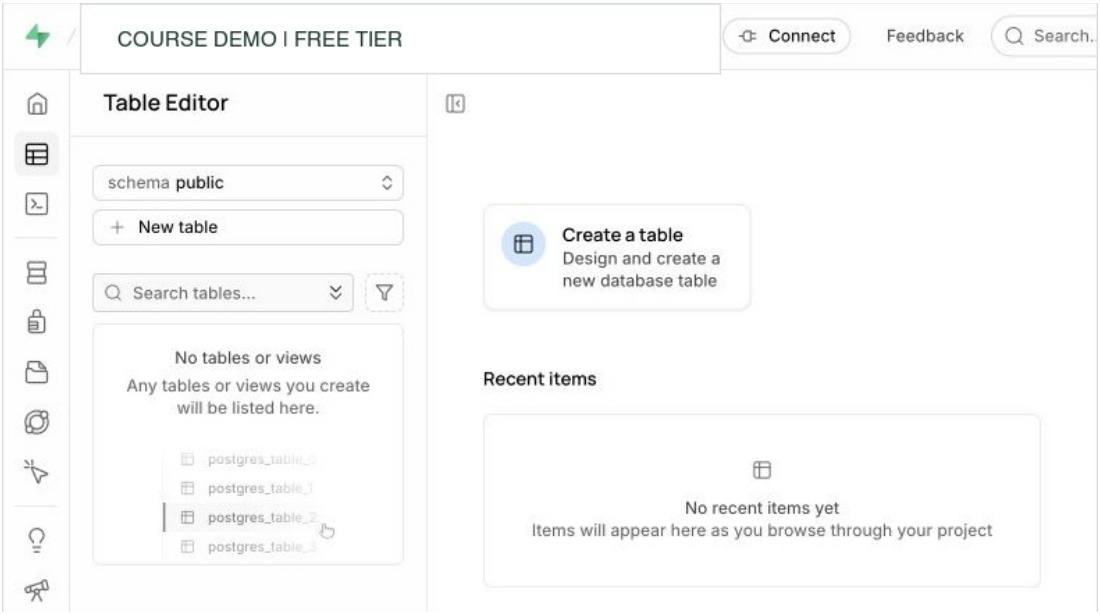



*Figure 3.1: Supabase Table Editor detail with no tables displayed in the selected public namespace. Custom schemas may contain other tables. Account identifiers are redacted. Interface captured August 25, 2026.*

The selector above the table list currently says `public`. It limits what this part of the dashboard displays. After loading the Metro Support setup script, select `metro_support` to find its tables. An empty list in `public` cannot establish that the database has no tables elsewhere. A schema name is part of an object's address, so it belongs in the explanation of how to find the data.

### 3.9.1 The First SQL Session

A SQL editor is a client: it sends a statement to a database and displays a response. In Supabase, the PostgreSQL server runs remotely in the selected project. Opening a script displays text; clicking Run executes it. Saving that text preserves instructions, while committing database changes preserves the resulting state. Closing an editor does not undo committed changes.

Start with a calculation that does not depend on any existing table:

**Listing 27. SQL**

```
SELECT 2 + 3 AS sum, current_database() AS database_name;
```

One returned row contains `5` under `sum`. The `AS` clause labels an output column; it does not create a stored attribute. `current_database()` identifies the database that answered. This small query separates a working client from a missing table: if it succeeds but a later table query fails, the next useful question concerns the table name, schema, project, or permission.

Appendix C contains the complete Metro Support setup, including a browser-only path and local PostgreSQL instructions. Its SQL defines users before tickets and tickets before events because the foreign keys require referenced rows to exist. Its final query should report eight users, twelve tickets, and twenty-one events. Opening the CSV files alone does not install that schema.

Treat the setup as a reset. Its `DROP SCHEMA ... CASCADE` can remove both the practice schema and dependent objects, including dependencies in other schemas. Use a dedicated practice environment with no data you need to retain. Once loaded, keep everyday queries separate from that file; a repeated reset would erase constraints and views you added in later exercises.

### 3.9.2 Reading the First Table Result

**Listing 28. SQL**

```sql
SELECT ticket_id, status, assignee_id
FROM metro_support.tickets
ORDER BY ticket_id
LIMIT 4;
```

The result contains tickets 1001 through 1004. Ticket 1004 has status `new` and a missing assignee. `LIMIT` controls how many rows the client receives, not how many the table stores. `ORDER BY` establishes which four appear. The qualified name `metro_support.tickets` tells PostgreSQL exactly which namespace to use. This matters because a setting such as `search_path` in one editor session may not carry over to a later request.

Read an error as information about the attempted operation. A missing-relation error suggests checking the name and target database. A named constraint error means the server found the table but rejected a proposed state. An empty result from a successful `SELECT` means no rows satisfied that query; it does not by itself mean the connection failed. If an explicit transaction has failed, run `ROLLBACK;` before trying the corrected transaction.

### 3.9.3 PostgreSQL Practice Without a Cloud Account

The PGlite playground runs these schema and migration examples without an account. Paste a complete statement or block at its SQL prompt and press Enter. PGlite runs PostgreSQL in the browser using WebAssembly, a format browsers can execute, and the playground retains its practice database in browser storage. That makes it useful for schemas, constraints, views, and the migration rehearsal. Keep your SQL in a separate file so losing browser storage does not lose your work.

The location of execution still matters. A browser database does not establish a remote connection to Supabase, exercise Supabase Auth, or recreate Chapter 5's two independent server sessions. Use the environment specified for each example. PGlite's documentation explains its embedded implementation; Supabase's database overview describes the managed PostgreSQL project and its editors.

## 3.10 Worked Example: Audit the Metro Support Baseline

The setup script accepts any text for `status`. That creates plausible but inconsistent states:

The next steps are a worked migration on a fresh practice baseline, before the status constraint shown below has been added. Do not run the entire chapter as one script; some listings are definitions or deliberate failures. In particular, keep expected-error statements separate from a normal successful run.

Listing 29. SQL

```sql
UPDATE metro_support.tickets
SET status = 'IN PROGRESS'
WHERE ticket_id = 1002;
```

The update succeeds even though existing data uses `in_progress`. Reports that filter the expected value may silently miss the row.

### 3.10.1 Step 1: Inspect Existing Values

Listing 30. SQL

```sql
SELECT status, count(*)
FROM metro_support.tickets
GROUP BY status
ORDER BY status;
```

### 3.10.2 Step 2: Normalize Before Constraining

For this known spelling error, map the identified invalid value explicitly. Do not apply a broad text transformation and assume all resulting words represent approved states. An unknown value should be investigated rather than silently reinterpreted.

Listing 31. SQL

```sql
UPDATE metro_support.tickets
SET status = 'in_progress'
WHERE ticket_id = 1002 AND status = 'IN PROGRESS'
RETURNING ticket_id, status;
```

### 3.10.3 Step 3: Add the Invariant

Listing 32. SQL

```sql
ALTER TABLE metro_support.tickets
ADD CONSTRAINT tickets_status_allowed
CHECK (status IN ('new', 'open', 'in_progress', 'resolved', 'closed'));
```

### 3.10.4 Step 4: Verify With an Expected Failure

Listing 33. SQL

```sql
INSERT INTO metro_support.tickets (
    ticket_id, requester_id, category, priority,
    status, subject, opened_at
) VALUES (
    1099, 101, 'parks', 'low', 'almost_done', 'Constraint test', now()
);
```

The expected constraint error confirms that the database rejects the bad state. Run the test inside a transaction and roll it back if any part succeeds.

## 3.11 Inspect Metadata Instead of Guessing

The catalog is data about the database. `information_schema` offers portable views; PostgreSQL's `pg_catalog` exposes deeper implementation details.

Listing 34. SQL

```sql
SELECT
    column_name,
    data_type,
    is_nullable,
    column_default
FROM information_schema.columns
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets'
ORDER BY ordinal_position;
```

Listing 35. SQL

```sql
SELECT indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'metro_support'
  AND tablename = 'tickets';
```

Metadata queries are more trustworthy than remembering what a dashboard displayed or assuming a script ran.

## 3.12 Common Misconceptions

### 3.12.1 "The application validates it, so the database does not need to"

Imports, scripts, future services, bugs, and administrator actions can bypass one application. Enforced database constraints apply regardless of which ordinary client issues a write. An administrator who can alter or remove the rules is a different trust boundary; constraint design does not replace access control.

### 3.12.2 "More constraints are always better"

An incorrect rule can reject valid business states and make change difficult. Add constraints for stable invariants, name them, test existing data, and plan change.

### 3.12.3 "Every foreign key should cascade"

Cascade is convenient but may erase more than intended. Choose based on lifecycle and audit requirements.

## 3.13 Practice

Apply the key, dependency, and constraint concepts to the actual baseline before proposing rules. Test both valid and invalid values: a rule should reject a specific invalid state without excluding a legitimate one.

Audit `metro_support.tickets` and propose:

1. one value-set constraint;
2. one nullability decision;
3. one referential-action decision; and
4. one candidate index tied to a stated query.

For each proposal, name the bad state or workload it addresses and one tradeoff.

## 3.14 Retrieval and Transfer

1. What is the difference between a PostgreSQL schema and a table definition?

2. Why does a foreign key permit null unless the column is also `NOT NULL`?
3. How is a `UNIQUE` constraint conceptually different from an index?
4. What test confirms that a new constraint is working?
5. Which Metro Support rule is too complex for a simple row-level `CHECK`, and why?

## 3.15 Further Reading

- Andy Pavlo, Carnegie Mellon University, **Functional Dependencies** and **Normal Forms** (2017 lecture notes). These free notes extend the dependency and decomposition discussion; they are external readings rather than this book's original material.
- PostgreSQL data definition
- PostgreSQL constraints
- PostgreSQL schemas
- PostgreSQL indexes
- Supabase database overview

# Part II: Operating PostgreSQL Safely

Once data meaning is explicit, administration becomes controlled change under concurrency, access rules, performance constraints, and failure. Chapters 4-8 move from views and migrations through transactions, row-level security, query plans, indexes, backup, and restore.

Each topic uses the same discipline: frame a testable promise, observe a baseline, make one deliberate change, verify the result and side effects, and record what the test does not cover. Supabase appears as managed PostgreSQL, not as a new SQL dialect, so provider responsibility remains separate from customer-owned schema, permissions, queries, secrets, and recovery.

# 4 Safe Changes Are Planned and Verified

## 4.1 Operating Question

How can a team change a database while limiting broken queries, bad data, long locks, and changes that cannot be undone?

An application rarely stops existing when a new database requirement arrives. Old clients continue to send requests, reports continue to run, and yesterday's records must still mean what they meant yesterday. A migration is therefore a transition between usable states, not merely the final `ALTER TABLE` statement. We will use views and a newly required field to examine that transition, then connect generated identifiers to what a transaction can and cannot roll back.

## 4.2 Learning Outcomes

After this chapter, you can:

- use views to create a stable and limited query interface;
- explain identity columns and sequences without confusing them with row counts;
- inspect columns, constraints, views, and dependencies through metadata;
- plan a small migration using precheck, change, verification, and rollback; and
- apply an expand-migrate-contract pattern to a breaking change.

## 4.3 A Database Change Has More Than One Audience

Adding a column may affect:

- existing rows that do not have a value;
- application code that selects or inserts columns;
- reports and views that depend on names or types;
- locks and query latency while the change runs;
- backup and recovery procedures;
- permissions and row-level policies; and
- people trying to understand the current schema.

The SQL statement is only one part of a migration. A professional change includes intent, preconditions, execution, verification, and a response if the result is wrong.

## 4.4 Views Create a Query Interface

A view stores a query definition, not a separate copy of its result in the common case.

This example uses the distinct name `chapter4_active_queue` to avoid replacing an existing application view with a different column layout. Chapter 6 reuses this view.

Listing 36. SQL

```sql
CREATE OR REPLACE VIEW metro_support.chapter4_active_queue AS
SELECT
    t.ticket_id,
    t.category,
    t.priority,
    t.status,
    t.subject,
    t.opened_at,
    u.display_name AS assignee_name
FROM metro_support.tickets AS t
LEFT JOIN metro_support.users AS u
    ON u.user_id = t.assignee_id
WHERE t.status IN ('new', 'open', 'in_progress');
```

The view can:

- hide an underlying join;
- expose only approved columns;
- give a stable name to a common query; and
- separate a consumer's interface from some storage details.

A view is not automatically faster. PostgreSQL usually expands its query into the outer query and plans the whole statement. A materialized view stores results and requires refresh, which creates a freshness tradeoff.

### 4.4.1 Follow a Read Through the View

On the full baseline, the active queue returns seven tickets, including the two that do not yet have an assigned agent. The left join preserves those requests; the `assignee_name` expression becomes null for them. The view does not turn that null into a nonexistent user. An application can display "Unassigned" while retaining the ticket's identity.

If we update a ticket's status to `resolved`, a subsequent ordinary view query uses the current visible database state and stops including that ticket. There is no separate queue table for us to synchronize. A materialized view is different: it holds previously computed results, so an application needs a refresh policy and an acceptable staleness limit. Both mechanisms can be useful, but they have different meanings when someone asks whether the screen is current.

A view also does not automatically isolate secrets. Omitting email from a reporting view helps define a narrower interface, but a role that can select the underlying `users` table can still retrieve email directly. Chapter 6 tests the combination of the view and the grants as the intended actor.

### 4.4.2 Avoid `SELECT *` in Durable Views

Explicit columns make the contract visible. PostgreSQL expands `SELECT *` when it defines a view: adding a base-table column later does **not** automatically add that column to the existing view. However, recreating or replacing a view from a `SELECT *` definition after the table changes can produce a different interface. Naming the intended columns makes that review deliberate, especially when the new column contains private information.

In PostgreSQL, `CREATE OR REPLACE VIEW` must preserve existing output column names, order, and types, although it can append columns. Inserting a new column in the middle of an established definition is not equivalent to appending it. A consumer that depends on a name or position may break even if the new

query returns sensible values. For a genuinely incompatible interface, a separately named version can let old and new consumers coexist while they migrate.

Ordering is another part of the contract to state explicitly. A view's rows are not inherently in a guaranteed presentation order. A consumer that needs the newest requests should request `ORDER BY opened_at DESC, ticket_id` in its own query rather than infer an ordering from earlier observations.

## 4.5 Identity Columns and Sequences Generate Values

An identity column asks PostgreSQL to generate a value, usually from a sequence:

**Listing 37. SQL**

```
CREATE TABLE metro_support.chapter4_change_notes (
    note_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    change_name text NOT NULL,
    recorded_at timestamptz NOT NULL DEFAULT now()
);
```

Generated identifiers are not row counts. Gaps are normal because sequence values may be consumed by rolled-back transactions, failed inserts, caching, or manual allocation. Do not promise consecutive numbers or infer how many rows exist from the largest identifier.

`chapter4_change_notes` is a separate practice table for this example. Create it once in your disposable practice database. If it already exists, inspect its rows rather than expecting a new sequence to start at 1.

`GENERATED ALWAYS` resists explicit values unless overridden. `GENERATED BY DEFAULT` permits an explicit value. Choose according to import and ownership needs.

### 4.5.1 Why a Rollback Leaves a Gap

**Client display:** the examples below include results inside transactions. For each complete SQL block, use a persistent `psql` session (Appendix C) to see all intermediate results. Some web editors show only the final result or command status. Do not split a transaction across separate web-editor requests; a new request may use another connection. You can still run each complete block in SQL Editor and inspect the committed state afterward.

Suppose the table above has just been created and its identity sequence has not been used. Run this sequence against that practice table:

Listing 38. SQL

```sql
BEGIN;
INSERT INTO metro_support.chapter4_change_notes (change_name)
VALUES ('Initial view') RETURNING note_id;
COMMIT;

BEGIN;
INSERT INTO metro_support.chapter4_change_notes (change_name)
VALUES ('Rehearsal that will not be retained') RETURNING note_id;
ROLLBACK;

BEGIN;
INSERT INTO metro_support.chapter4_change_notes (change_name)
VALUES ('Approved revision') RETURNING note_id;
COMMIT;

SELECT note_id, change_name
FROM metro_support.chapter4_change_notes ORDER BY note_id;
```

The inserts return 1, 2, and 3, but the final table contains rows 1 and 3. The second row was rolled back. Its sequence allocation was not. With multiple sessions, reclaiming a number every time one transaction aborts would complicate independent allocation. A sequence provides generated values without promising an uninterrupted count of committed rows. The PostgreSQL sequence documentation describes this non-rollback behavior.

The explicit commit boundaries matter when an editor sends this entire listing as one batch. The first row must be committed before the rehearsal begins; otherwise a client-side batch can place earlier statements in the transaction that is later rolled back. Transaction boundaries are part of the example, not just formatting.

On a table already used in a previous run, the exact numbers will be higher. The interpretation remains the same: use `count(*)` to count rows, and use keys to identify them. Requirements for consecutive invoice or receipt numbers need a separately designed business process; an identity column is not that process. Likewise, explicitly importing a large ID does not automatically advance the underlying sequence to that value. Imports need a deliberate sequence check.

## 4.6 Introspection Reveals the Actual State

Before changing an object, query the system that will be changed.

### 4.6.1 Find Columns and Defaults

Listing 39. SQL

```sql
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets'
ORDER BY ordinal_position;
```

### 4.6.2 Find Constraints

Listing 40. SQL

```sql
SELECT constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets'
ORDER BY constraint_type, constraint_name;
```

### 4.6.3 Find Views That Mention a Table

Listing 41. SQL

```sql
SELECT table_schema, table_name, view_definition
FROM information_schema.views
WHERE view_definition ILIKE '%metro_support.tickets%';
```

Text search is a useful first pass, not a complete dependency model. PostgreSQL catalogs and tools can expose more precise dependencies. The key habit is to inspect rather than assume.

## 4.7 What a Migration Must Establish

For a small migration, be able to explain:

1. **Intent:** what user or operating need justifies the change?
2. **Precheck:** what must be true before execution?
3. **Change:** what ordered SQL creates the new state?
4. **Verification:** what proves structure and data are correct?
5. **Rollback or forward fix:** how will the team respond if verification fails?

Not every production migration can be rolled back instantly. A forward fix may be safer after data has been transformed. State the boundary honestly.

## 4.8 Worked Example: Add a Public Status Label Safely

Metro Support wants user-facing labels such as “In progress” while preserving the machine value `in_progress`.

### 4.8.1 Risky Approach

Rename or replace the stored values immediately. Existing queries, constraints, and integrations may expect the old values.

### 4.8.2 Expand

Add a view with a derived label while preserving the source column:

Listing 42. SQL

```sql
CREATE OR REPLACE VIEW metro_support.public_ticket_status AS
SELECT
    ticket_id,
    subject,
    status AS status_code,
    CASE status
        WHEN 'new' THEN 'New'
        WHEN 'open' THEN 'Open'
        WHEN 'in_progress' THEN 'In progress'
        WHEN 'resolved' THEN 'Resolved'
        WHEN 'closed' THEN 'Closed'
        ELSE 'Unknown'
    END AS status_label,
    opened_at,
    closed_at
FROM metro_support.tickets;
```

### 4.8.3 Verify

Listing 43. SQL

```sql
SELECT status_code, status_label, count(*)
FROM metro_support.public_ticket_status
GROUP BY status_code, status_label
ORDER BY status_code;
```

Check that every known code has one intended label and that the total count equals the base table count.

### 4.8.4 Migrate Consumers

Update a report or application to read the view. Observe before removing or renaming any old interface.

### 4.8.5 Contract Later

Only after all consumers are confirmed should an obsolete interface be removed. In this case, keeping both code and label is useful, so contraction may not be needed.

## 4.9 Transactional DDL Helps, but Locks Still Matter

Many PostgreSQL data-definition statements can run inside a transaction:

The next example rehearses adding a required field to existing data. Begin with the practice baseline before `source_channel` exists. Historical tickets have no structured channel field. Some event notes mention mobile submission, so it would be false to label every old ticket `web`. We use `unknown` to mean that the structured value was not captured. More detailed backfilling would require a reviewed, record-specific source.

Listing 44. SQL

```sql
BEGIN;
SET LOCAL lock_timeout = '2s';
SET LOCAL statement_timeout = '10s';

ALTER TABLE metro_support.tickets
ADD COLUMN source_channel text;

UPDATE metro_support.tickets
SET source_channel = 'unknown'
WHERE source_channel IS NULL;

ALTER TABLE metro_support.tickets
ADD CONSTRAINT tickets_source_channel_allowed
CHECK (source_channel IN ('web', 'phone', 'mobile', 'unknown'));

ALTER TABLE metro_support.tickets
ALTER COLUMN source_channel SET DEFAULT 'unknown',
ALTER COLUMN source_channel SET NOT NULL;

-- Inspect before choosing COMMIT or ROLLBACK.
SELECT source_channel, count(*)
FROM metro_support.tickets
GROUP BY source_channel;

ROLLBACK;
```

The grouped result inside this rehearsal is `unknown`, 12. After rollback, the new column and its constraint no longer exist. To retain the migration, repeat the verified block and deliberately choose `COMMIT` instead. Do not run the creation block repeatedly against an already migrated table.

The default has a particular purpose: an older client that inserts an explicit list of the old columns can still create a row. Its omitted channel receives `unknown`. An explicit null is different from an omitted field and fails the new not-null rule. A new client should provide the actual channel. After all writers are upgraded, we can reconsider whether the default still serves a useful purpose. Adding a default is an application-compatibility decision, not just a way to make an error disappear.

Rollback protects atomicity, but an `ALTER TABLE` may still acquire a strong lock and block other work while the transaction remains open. Production-safe change planning considers table size, lock duration, statement timeout, maintenance windows, and compatibility with live clients.

### 4.9.1 A Safer Pattern for Large Tables

For a required new value:

1. add a nullable column;
2. deploy writers that populate it;
3. backfill old rows in controlled batches;
4. verify no nulls remain;
5. add or validate a constraint; and
6. remove old behavior only after consumers migrate.

The exact feature support and lock behavior depend on the PostgreSQL version. Consult current official documentation for a production plan.

### 4.9.2 Before Commit and After Commit Are Different Repair Boundaries

During our small rehearsal, rollback restores the prior database state. After a deployment commits and new mobile requests arrive, dropping `source_channel` would erase newly collected facts. That may be technically possible but is not an adequate recovery plan. A forward repair could correct the view definition or writer behavior while retaining channel data.

The same distinction applies to a destructive transformation. If a script replaces detailed values with a coarse category, reversing its SQL does not reconstruct the discarded detail. Preserve a verified source or choose a non-destructive transition when that information matters. A rollback plan must describe the state and data it recovers, rather than merely name an opposite command.

The timeouts in the worked example are guardrails for this small experiment. `lock_timeout` limits waiting to acquire a lock; `statement_timeout` limits a statement's elapsed execution. A timeout can leave the current transaction aborted, requiring rollback. Their appropriate production values depend on the workload and operational policy. They do not make a long migration intrinsically safe or eliminate the need to understand its locks.

## 4.10 Verification Should Cover Structure and Meaning

Run the following checks while the migrated column exists, either inside the rehearsal before its rollback or after an intentionally committed migration. After rollback, the column's absence is the expected result.

Structural check:

Listing 45. SQL

```sql
SELECT column_name, is_nullable
FROM information_schema.columns
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets'
  AND column_name = 'source_channel';
```

Data check:

Listing 46. SQL

```sql
SELECT count(*) AS missing_source_count
FROM metro_support.tickets
WHERE source_channel IS NULL;
```

Behavior check:

Listing 47. SQL

```sql
BEGIN;
INSERT INTO metro_support.tickets (
    ticket_id, requester_id, category, priority, status, subject, opened_at
) VALUES (
    1098, 101, 'parks', 'low', 'new', 'Old-client compatibility test', now()
)
RETURNING ticket_id, source_channel;
ROLLBACK;
```

With our compatibility default, this insert succeeds and returns `unknown` for the channel. The rollback removes the test ticket. A second test that explicitly supplies a null channel should fail the not-null rule. If we later remove the default, this omitted-field insertion will fail too, so that change requires knowing that old writers have been upgraded.

## 4.11 Common Misconceptions

### 4.11.1 “A successful migration statement proves the application is safe”

The structure may have changed while existing queries, permissions, or data are wrong. Verify at multiple layers.

### 4.11.2 “Sequences should never have gaps”

Sequences allocate values efficiently, not gapless business numbering. A primary key or unique constraint enforces uniqueness in the stored table, including against explicit values or later sequence resets.

### 4.11.3 “A view is a backup”

A normal view stores a query definition. It depends on underlying objects and does not preserve deleted data.

## 4.12 Practice

The following problem combines schema evolution with historical data and client compatibility. Distinguish the rule for new writes from the treatment of rows that existed before the field was introduced.

Plan a change that adds `resolution_summary` for resolved or closed tickets. Write:

1. the intent;
2. one data precheck;
3. an expand step;
4. two verification queries; and
5. a rollback or forward-fix decision.

Do not make the field required in the same step that creates it. Explain why.

## 4.13 Retrieval and Transfer

1. What problem does a view solve that a base table does not?
2. Why can a generated identity value contain gaps?
3. Name the five parts of a small change record.
4. Why does transactional DDL not eliminate operational risk?
5. A new required column will be added to ten million rows. Which expand-migrate- contract steps reduce risk?

## 4.14 Further Reading

- PostgreSQL views
- PostgreSQL identity columns
- PostgreSQL information schema
- PostgreSQL `ALTER TABLE`
- Supabase database migrations

# 5 Transactions Coordinate Competing Work

## 5.1 Operating Question

When two sessions read and change the same data, how can we predict what each session sees, identify who is waiting, and resolve the situation without guessing?

## 5.2 Learning Outcomes

After this chapter, you can:

- define a transaction boundary and demonstrate `COMMIT` and `ROLLBACK`;
- connect atomicity, consistency, isolation, and durability to observable behavior;
- explain snapshots and row versions at a beginner level;
- explain why vacuum is required under MVCC and what transaction-ID wraparound protection does;
- distinguish normal waiting, blocking, and deadlock;
- explain a backend PID and distinguish it from a user or record identifier;
- use PostgreSQL activity and blocking state to identify session relationships; and
- resolve a controlled blocking incident safely and verify the result.

## 5.3 A Transaction Is a Unit of Decision

A transaction groups database statements into one unit that either commits or rolls back.

Consider a repair desk assigning an unassigned request to Agent 202. The application needs to change the ticket's current assignee and add an event saying who made that change. If the ticket update succeeds but the history insert fails, the current state and the history disagree. Each statement can be valid SQL, and each table can satisfy its own constraints, while the application operation is still incomplete. A transaction lets the application define that larger unit.

Start with the smaller boundary below. It uses the supplied Metro Support data and ends with rollback so reading the example does not permanently change the fixture. Run the block as a whole in your own practice database.

**Listing 48. SQL**

```sql
BEGIN;

UPDATE metro_support.tickets
SET assignee_id = 202,
    status = 'in_progress'
WHERE ticket_id = 1004
RETURNING ticket_id, assignee_id, status;

ROLLBACK;
```

Many clients use **autocommit**: each statement becomes its own transaction unless you explicitly begin one. An explicit transaction creates a review point, but it also keeps locks and snapshots alive until the transaction ends. "I forgot to commit" is operationally meaningful.

The `RETURNING` row is visible to the transaction that made the change. It is not a commit receipt. After this block, a fresh query should still show ticket 1004 as unassigned and `new`. Replacing the last statement with `COMMIT` keeps the change. Test a commit only in a separate disposable copy so that the baseline for the remaining examples stays unchanged.

### 5.3.1 Keeping State and History Together

This second example adds the history event. It assumes the original fixture: ticket 1004 is `new` and event 5998 does not exist.

**Listing 49. SQL**

```sql
BEGIN;

UPDATE metro_support.tickets
SET assignee_id = 202, status = 'in_progress'
WHERE ticket_id = 1004 AND status = 'new'
RETURNING ticket_id, assignee_id, status;

INSERT INTO metro_support.ticket_events
    (event_id, ticket_id, actor_id, event_type, old_status,
     new_status, note, event_at)
VALUES
    (5998, 1004, 202, 'status_changed', 'new',
     'in_progress', 'Assigned to Agent 202 for follow-up', now());

SELECT event_id, ticket_id, new_status
FROM metro_support.ticket_events WHERE event_id = 5998;
ROLLBACK;
```

Inside the transaction, both changes can be inspected. Outside it after rollback, neither remains. If the insert violates a constraint, PostgreSQL reports an error and the transaction enters a failed state. End it with `ROLLBACK`; do not keep issuing ordinary statements and assume that the earlier update committed.

There is an important boundary to this protection. An `UPDATE` matching zero rows is **not** an SQL error. A real application must check that the intended ticket was changed before accepting the history event and committing. The `status = 'new'` condition protects the proposed transition, but it does not automatically tell later application code to stop when the condition matches nothing. The database cannot infer the meaning of "assign this previously unassigned request" from an arbitrary list of successful statements.

### 5.3.2 An Outdated Assignment Request

An application may display a ticket as `new`, then receive an assignment request after somebody has already changed that status. The original screen is now outdated. The following one-session rehearsal makes the condition visible without requiring concurrent clients. Start with ticket 1004 unassigned and `new`, and execute the complete block. This probe changes no history events.

Listing 50. SQL

```sql
BEGIN;

-- The first request finds the state it expected.
UPDATE metro_support.tickets
SET assignee_id = 202, status = 'in_progress'
WHERE ticket_id = 1004 AND status = 'new'
RETURNING ticket_id, assignee_id, status;

-- The next request still assumes that the ticket is new.
UPDATE metro_support.tickets
SET assignee_id = 201, status = 'in_progress'
WHERE ticket_id = 1004 AND status = 'new'
RETURNING ticket_id, assignee_id, status;

SELECT ticket_id, assignee_id, status
FROM metro_support.tickets WHERE ticket_id = 1004;
ROLLBACK;
```

The first UPDATE returns `1004, 202, in_progress`. The second returns no rows. Its condition no longer matches, so it never assigns Agent 201. The SELECT still shows Agent 202's assignment inside the transaction. After rollback, the source ticket returns to its original state. This is a test of the condition, not a demonstration of two concurrent sessions; the later lock experiment supplies those connections.

For this one-ticket action, application code should require one returned ticket before inserting the matching history event. If there is no returned row, it should end the attempt without recording a fictitious assignment, inspect the current record, and give the caller a useful response. The caller may need to reload an outdated screen. A missing ticket can also produce zero rows, so the count alone does not explain why the request failed to match.

The database result and the application message answer different questions. "The SQL executed" is too weak to justify "your ticket was assigned." The PostgreSQL UPDATE reference defines the zero-row result and the rows returned by `RETURNING`.

### 5.3.3 What Rollback Does Not Undo

Rollback covers transactional database changes. It does not retract an email already sent, undo an external API call, or reclaim every sequence value. Chapter 4's identity example showed a sequence gap after rollback. Chapter 14 returns to external messages: an application must coordinate them with database commits rather than assume that a transaction spans every service it calls.

A rollback should be verified, not merely assumed. Figure 5.1 shows a reversible browser exercise that creates a temporary table, inserts one row, rolls the transaction back, and then asks PostgreSQL whether the temporary relation still exists. The returned `true` means `to_regclass(...) IS NULL`: the relation is no longer present.




*Figure 5.1: The Supabase SQL Editor query and result show `rollback_verified = true`: rollback removed the temporary relation without changing existing practice data. Navigation and empty lower panels are outside this frame. Account identifiers are redacted. Captured August 25, 2026.*

This is stronger than a green "success" message because the final `SELECT` checks the postcondition independently. Supabase may still display a heuristic RLS warning for a `CREATE TABLE` statement; in this controlled example the table is temporary and the transaction is rolled back. A persistent application table would require an explicit access-policy decision.

## 5.4 ACID Describes Guarantees, Not a Product Label

### 5.4.1 Atomicity

All changes in a transaction commit together or none remain. If assigning a ticket and recording its event must be one operation, put them in one transaction.

### 5.4.2 Consistency

A transaction moves the database between states that satisfy enforced rules. Consistency is not magic correctness: the database can enforce only the constraints and transaction logic it has been given.

In this use of the word, consistency means maintaining application invariants, such as a valid ticket status and a matching history event. Later chapters use "consistency" when comparing observations across replicas. The same word refers to different questions; a valid state is not automatically an immediately visible state on every machine.

### 5.4.3 Isolation

Concurrent transactions behave according to an isolation model. Isolation does not always mean that every transaction runs as though completely alone. Different levels permit different observations and may require retries.

### 5.4.4 Durability

After a successful commit, the DBMS promises the change survives failures within its documented durability model. Durability is not the same as indefinite retention or protection from an authorized later deletion. Backups and recovery still matter.

## 5.5 Concurrency Creates Useful Work and New Questions

Without concurrency, a database would waste time while one client thinks, communicates, or waits for input. With concurrency, two sessions may:

- read the same row;
- update different rows;
- compete to update the same row;
- make decisions from different snapshots; or
- acquire resources in opposite orders.

Common anomaly names include dirty read, nonrepeatable read, phantom, lost update, and serialization anomaly. PostgreSQL's implementation and isolation levels determine which can occur. The practical questions are:

1. what did each transaction read?
2. what did each transaction change?
3. which transaction is waiting, and on whom?
4. which outcome can the application accept?

The names become useful when attached to a situation. A **dirty read** would show another transaction's uncommitted assignment. A **nonrepeatable read** shows an old assignment on one query and a newly committed assignment on a later query. A **phantom** concerns the membership of a result: the next active-ticket query includes a newly committed request. These are not all the same error. For a live queue, seeing a newly committed ticket may be exactly what the application wants. For two parts of one financial report, changing the visible data between queries may be unacceptable.

## 5.6 MVCC Lets Readers and Writers Coexist

PostgreSQL uses multi-version concurrency control, or MVCC. An update creates a new row version rather than overwriting the only copy in place. A transaction reads versions visible to its snapshot.

This design often allows readers and writers to proceed without blocking one another. It does not mean there are no locks. Writers competing for the same row can wait, schema changes can take strong locks, and long transactions can prevent old row versions from being cleaned up.

`VACUUM` helps reclaim or mark reusable space from obsolete versions and supports transaction-ID safety. Routine vacuuming is normal database maintenance; it does not by itself mean that PostgreSQL is broken.

Think of a row version as a value with visibility conditions, not a second ticket the user should count. While Agent 202's assignment is uncommitted, an ordinary reader can still see the previously committed ticket. After commit, a later snapshot can see the new version. Old snapshots may still need the old version, which is one reason a long-running transaction can interfere with cleanup even if it is not actively changing rows. MVCC is not a built-in historical reporting API: obsolete versions are eventually reclaimed rather than kept as a permanent event history.

## 5.7 Vacuum Keeps MVCC Honest

MVCC leaves old row versions in place until nobody can see them. Something has to remove them afterward, and something has to keep the transaction-ID arithmetic that visibility depends on from running out of room. Both jobs belong to `VACUUM`. This section explains what it does, why operators tune it, and what happens when it falls behind, because the failure mode is an outage rather than a slow query.

### 5.7.1 A Row Version Carries Two Transaction IDs

Every PostgreSQL table has hidden system columns that can be selected by name. `xmin` is the ID of the transaction that created a row version, and `xmax` is the ID of the transaction that expired it; `0` means no transaction has.

Listing 51. SQL

```
SELECT xmin, xmax, ticket_id, priority
FROM metro_support.tickets
WHERE ticket_id = 1004;
```

On a fresh PostgreSQL 16 server, the row read `xmin` 733, `xmax` 0, priority `medium` before an update, and `xmin` 735, `xmax` 0, priority `high` afterward. The old version did not disappear. It remains in the table file with its `xmax` set to 735, because a snapshot taken before transaction 735 committed is still entitled to read it. Once no snapshot can see it, the old version is **dead**: it occupies space in the table and in every index that points at it, and no `UPDATE` or `DELETE` ever removes it.

### 5.7.2 What VACUUM Does

`VACUUM` is the routine maintenance that cleans up after MVCC. It has four jobs:

- remove dead row versions and mark their space reusable within the table and its indexes;
- freeze old transaction IDs so the ID counter can continue safely, which the next subsection explains;
- maintain the visibility map, which lets later vacuums skip clean pages and lets index-only scans avoid the table; and
- with `ANALYZE`, refresh the row counts and value distributions the planner uses.

Plain `VACUUM` runs alongside ordinary reads and writes and does not shrink the table file; it makes space available for reuse. `VACUUM FULL` rewrites the table into a new file with no dead space, needs that much extra

disk, and holds an `ACCESS EXCLUSIVE` lock for the duration, so nothing else can use the table. On the twelve-ticket table above, seven updates left `n_dead_tup` at 7 in `pg_stat_user_tables`; `VACUUM (VERBOSE)` reported seven tuples removed and twelve remaining, and the dead-row count returned to zero.

### 5.7.3 Transaction IDs Wrap Around

A transaction ID is a 32-bit integer, so there are $2^{32}$, roughly four billion, possible values. PostgreSQL compares them on a circle: from any current ID, about two billion IDs count as the past and about two billion as the future. Visibility depends on that comparison. A row version created by a committed past transaction is visible; one created by a future transaction is not.

If the counter advanced far enough to lap an old row, that row's creating transaction would appear to be in the future and the row would vanish from every query. An old deletion would appear not to have happened yet, and the deleted row would return. Nothing on disk changes; the arithmetic simply stops meaning what it meant. The documentation calls this catastrophic data loss.

The defense is **freezing**. When `VACUUM` finds a row version whose creating transaction is older than `vacuum_freeze_min_age` (50 million transactions by default), it marks the version frozen: visible to every transaction forever, with no comparison needed. PostgreSQL records the oldest unfrozen ID for each table in `pg_class.relfrozenxid` and for each database in `pg_database.datfrozenxid`. The *age* of those IDs, their distance from the current counter, is the number an operator watches.

Listing 52. SQL

```sql
SELECT datname, age(datfrozenxid) AS xid_age
FROM pg_database
ORDER BY xid_age DESC;
```

On an idle practice server this returned 14 for every database. A healthy production database stays well under 200 million, for the reason given next.

### 5.7.4 Autovacuum Decides When to Run

Nobody runs `VACUUM` by hand on a busy system. The autovacuum launcher wakes every `autovacuum_naptime` (one minute by default) and starts up to `autovacuum_max_workers` (three) workers. A worker vacuums a table once its dead rows exceed `autovacuum_vacuum_threshold` plus `autovacuum_vacuum_scale_factor` times the table's estimated row count, `reltuples`. The defaults are 50 rows and 0.2, so the trigger is 50 dead rows plus 20 percent of the table.

| Table size (`reltuples`) | Dead rows before autovacuum starts |
|---:|---:|
| 12 | 53 |
| 1,000 | 250 |
| 1,000,000,000 | 200,000,050 |

The defaults suit ordinary tables. A twelve-row table is vacuumed rarely, which is harmless. A billion-row table carries two hundred million dead rows before anything happens, and the vacuum that finally starts takes hours. Operators therefore lower `autovacuum_vacuum_scale_factor` for their largest tables with `ALTER TABLE ... SET (autovacuum_vacuum_scale_factor = 0.01)` or similar.

A second trigger ignores dead rows entirely. When a table's oldest unfrozen ID is older than `autovacuum_freeze_max_age` (200 million by default), autovacuum vacuums it to freeze old rows, even if autovacuum has been disabled for that table or for the server. This forced vacuum is the system's own insurance against wraparound.

### 5.7.5 Why Teams Tune Autovacuum, and the Trap

Vacuum reads and writes a great deal of disk, and every autovacuum setting exists for a legitimate reason.

- `autovacuum_vacuum_cost_delay` (default 2 ms): raised so vacuum uses less I/O at peak. Cost: vacuum finishes later, and dead rows and unfrozen IDs accumulate.
- `autovacuum_max_workers` (default 3): raised when many large tables fall due together. Cost: more maintenance I/O competing with the application.
- `autovacuum_vacuum_scale_factor` (default 0.2): lowered per table so a huge table is vacuumed sooner. Cost: more frequent, shorter vacuums.
- `autovacuum_freeze_max_age` (default 200 million): raised to delay forced wraparound vacuums. Cost: tables approach the wraparound horizon before anyone is forced to act.
- `maintenance_work_mem` (default 64 MB): raised so one pass remembers more dead rows and makes fewer index passes. Cost: memory per worker.

Every change that makes vacuum cheaper today moves work to later. Autovacuum tuning looks like a performance decision, and usually it is one. But vacuum is also the only mechanism that keeps the transaction-ID arithmetic valid, so a setting that lets maintenance fall behind is a correctness risk with a fixed deadline. The counter does not wait.

### 5.7.6 Wraparound Protection Chooses Unavailability Over Corruption

PostgreSQL will not let the counter lap an unfrozen row. It watches the distance between each database's oldest unfrozen ID and the wraparound point, and it escalates.

| Transactions left before wraparound | PostgreSQL 16 behavior |
|---|---|
| More than 40 million | Normal operation; forced freezing vacuums begin at age 200 million |
| 40 million | Every command logs `WARNING: database "mydb" must be vacuumed within N transactions` |
| 3 million | New transaction IDs are refused: `ERROR: database is not accepting commands to avoid wraparound data loss in database "mydb"` |

At the last line, read-only transactions still run, but every `INSERT`, `UPDATE`, `DELETE`, and `TRUNCATE` fails until a `VACUUM` in that database freezes the old rows. Versions before 14 drew the line at one million transactions. The error's hint still suggests single-user mode; the current documentation says instead to end long-running and prepared transactions, drop stale replication slots, and run `VACUUM` normally. The database is choosing to be unavailable rather than wrong. That is the right design, and it also means the outage belongs to the operator, and the only exit is a vacuum that may take hours on a very large table.

### 5.7.7 Case Study: Sentry, July 20, 2015

Sentry receives error events from other organizations' applications, so its primary PostgreSQL database was very write-heavy and held some enormous tables. In the team's own account, they had been running with too high a value for `autovacuum_freeze_max_age`, the default three workers, and far too much vacuum cost delay. Each choice is defensible alone. Together they kept the maintenance on the largest tables slightly behind until, on a Monday morning, wraparound protection fired and the database stopped accepting writes.

The team let the running vacuums finish rather than restart them, moved the primary to new hardware with more memory and CPU, and waited nearly three hours. Querying PostgreSQL's statistics showed every table finished except one: a very large mapping from each event to its rollup group. With no way to know whether that vacuum needed one more hour or ten, they decided they were willing to lose the table's contents to recover immediately, because it served non-critical features and replicas held a copy. They truncated it, and five minutes later the service was fully restored. Afterward they raised the worker count to six, cut the naptime to fifteen seconds, set the cost delay to zero, gave maintenance far more memory, lowered `vacuum_freeze_min_age`, and planned to split the data across several databases so no single table could grow that large again.

Two lessons follow. The first is operational: the condition had been visible for weeks in `age(datfrozenxid)`, and an alert on that number would have turned an outage into a scheduled vacuum. The second concerns power. The recovery was a `TRUNCATE` run under pressure with the most privileged credentials in the system, a decision only someone with that access and the authority to accept the loss could make. Chapter 6 is about keeping that power away from application roles; this incident shows why the people who hold it must be ready to use it and to explain it.

### 5.7.8 Monitor the Two Numbers That Predict It

**Listing 53. SQL**

```sql
SELECT datname, age(datfrozenxid) AS xid_age
FROM pg_database
ORDER BY xid_age DESC;

SELECT relname, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 5;
```

| Reading | Healthy | Worrying |
|---|---|---|
| Database transaction-ID age | Well under 200 million | Climbing steadily toward 2 billion |
| Dead rows per table | Rises, then falls after each vacuum | Only rises; `last_autovacuum` empty or days old |

A rising age usually means something is preventing freezing: a vacuum that cannot keep up, a transaction open for days, or an abandoned replication slot. A concrete prevention plan names the number, the threshold, and who is paged.

## 5.8 Isolation Levels Change Visibility and Retry Behavior

PostgreSQL supports:

- **Read Committed**, the default: each statement receives a snapshot at statement start. Two selects in one transaction may see different committed data.
- **Repeatable Read:** the transaction keeps a stable snapshot for ordinary reads; conflicting patterns may produce an error rather than an inconsistent result.
- **Serializable:** PostgreSQL attempts behavior equivalent to some serial order; applications must be prepared to retry serialization failures.

PostgreSQL treats Read Uncommitted as Read Committed. Do not infer behavior only from a generic isolation-level chart; consult the DBMS documentation.

Stronger isolation is not simply "better." It can increase coordination, abort/retry behavior, and application complexity. Choose based on the invariant.

### 5.8.1 Read the Same Row Twice

Here is a reasoning example, not an instruction to leave browser sessions open. Both sessions start from ticket 1004 with priority `low`.

| Order | Session A | Session B |
|---|---|---|
| 1 | begins a transaction and reads `low` | not yet changing the row |
| 2 | remains open | changes priority to `high` and commits |
| 3 | reads the same ticket again | has finished |
| 4 | ends its transaction | has finished |

With PostgreSQL Read Committed, A's second read can show `high`: the second statement gets a new snapshot. With Repeatable Read, A's ordinary second read still shows `low`: its transaction uses the snapshot established by its first non-control statement. B's commit was not lost; it is simply outside A's snapshot. A new transaction can see it. If A now tries to update a row changed since that Repeatable Read snapshot, a serialization failure may require A to retry. PostgreSQL isolation behavior

### 5.8.2 A Stable Snapshot Is Not Every Business Rule

Suppose Agent 201 and Agent 202 are both on call, and the rule is "at least one agent must remain on call." Each transaction reads that the other agent is available and then changes only its own agent to off call. Under snapshot isolation, the writes can target different rows and both commit, leaving nobody on call. There was no dirty read, and each transaction's view was stable, but the shared rule failed. This pattern is called **write skew**.

The point is not to memorize another name. Identify whether the rule concerns one row or a relationship among several decisions. A design can coordinate on a shared locked record or use serializable transactions with a retry path. The right choice depends on the actual invariant. Merely adding `BEGIN` and `COMMIT` around a read-then-write program does not establish serial execution.

## 5.9 Sessions, Server Processes, and PIDs

A **process** is a running instance of a program. A database client sends work over a connection to PostgreSQL. A **session** is the database conversation associated with that connection. PostgreSQL uses a server process, called a **backend**, to serve it. A session can execute many statements and more than one transaction before disconnecting.

**PID** means **process identifier**. In the activity output used here, it is the number identifying a PostgreSQL backend on the database server. It is not the PID of a browser or a Python notebook on the client computer. Two sessions can use the same database account and work on the same ticket while having different backend PIDs.

### 5.9.1 Find the Current Session's PID

Run this read-only query in an existing PostgreSQL connection. It needs no application tables or setup and leaves nothing to clean up.

Listing 54. SQL

```sql
SELECT pg_backend_pid() AS my_pid;
```

The result is one positive integer under the heading `my_pid`. An illustrative result is `7310`; the actual value comes from the running server. Run the query again on the same persistent backend connection and the PID remains the same, including across commits. A commit ends a transaction, not the session.

Keep three kinds of identifier separate. A ticket ID identifies a stored business record. A database user identifies an account. A backend PID identifies a running server process. The PID is temporary: a later connection may use a different process, and a stopped process's number can eventually be reused. Record PIDs together with the time and server being inspected rather than treating them as permanent names.

In a pooled environment, the client connection and server backend need not stay paired between transactions. For the controlled experiment below, use persistent connections or a session-mode pooler, and obtain PIDs by asking PostgreSQL. Separate browser tabs alone do not establish those connections. The diagnostic connection has its own PID; calling `pg_backend_pid()` there identifies the observer, not automatically Session A or B.

The PostgreSQL client/server architecture and session information functions describe the server process and the identity returned by `pg_backend_pid()`.

## 5.10 Waiting Is a Relationship Between Sessions

If Session A updates ticket 1004 and remains open, Session B's attempt to update the same row waits. Session B is blocked; Session A is the blocker.

Waiting can be correct: competing row writes must coordinate. It does not by itself prevent a stale application value from overwriting a newer value after the wait ends. The problem may be an unexpectedly long wait, an unknown transaction, or a write that no longer satisfies the application's condition.

The two-session SQL below explains the mechanism. Use it only with two persistent connections to a disposable practice database and release Session A promptly. Separate web-editor tabs are not guaranteed to

represent persistent independent database sessions. The optional transaction notebook creates the wait, captures it, and releases it within one cell. Reading its output does not leave another query blocked.

### 5.10.1 Session A

Listing 55. SQL

```sql
BEGIN;

UPDATE metro_support.tickets
SET priority = 'high'
WHERE ticket_id = 1004;

-- Leave this transaction open only during the controlled experiment.
```

### 5.10.2 Session B

Listing 56. SQL

```sql
BEGIN;
SET LOCAL statement_timeout = '15s';
UPDATE metro_support.tickets
SET status = 'in_progress'
WHERE ticket_id = 1004;
COMMIT;
```

Session B waits because the target row is already being changed by the uncommitted transaction in Session A.

If Session B times out, issue `ROLLBACK` in B and release A before retrying the exercise. A timeout is a bound on waiting, not evidence that the blocking transaction was automatically removed.

## 5.11 Diagnose From a Third Session

`pg_stat_activity` exposes server processes and current activity. Access may be limited by role and managed-service policy.

Listing 57. SQL

```sql
SELECT
    pid,
    usename,
    state,
    wait_event_type,
    wait_event,
    xact_start,
    query_start,
    left(query, 100) AS query_sample
FROM pg_stat_activity
WHERE datname = current_database()
ORDER BY xact_start NULLS LAST, query_start;
```

### 5.11.1 Read the Waiting PID and the Blocking PIDs

The `pid` column identifies the process represented by that activity row. `pg_blocking_pids(pid)` takes the PID of the process being inspected and returns an integer array naming its lock blockers. Read the input as “whose wait am I investigating?” and the output as “which processes is it waiting for?”

Consider this illustrative snapshot, not live identifiers to copy into commands:

| Session | PID | pg_blocking_pids(pid) |
|---|---|---|
| A | 7310 | {} |
| B | 7332 | {7310} |

Start with B's row: process `7332` is waiting for process `7310`. B is the blocked session, and A is the blocker. The braces are PostgreSQL's array notation. `{7310}` contains one PID; `{}` is empty. An empty list on A means no blocker was returned for A at that observation. It does not mean A holds no locks or cannot block B. A Python client may display those arrays as `[7310]` and `[]`.

**Check your reading:** which session must finish its transaction to release this one-way wait? A, PID `7310`. B's PID identifies the waiter, not the owner of the conflicting change. In a real incident, first verify that the relationship still exists and establish whether the owner's work should commit or roll back.

Use this read-only query to find current relationships. When there are no blocked sessions, it returns no rows; that is not an error.

Listing 58. SQL

```sql
SELECT
    blocked.pid AS blocked_pid,
    blocked.wait_event_type,
    blocked.wait_event,
    pg_blocking_pids(blocked.pid) AS blocking_pids,
    left(blocked.query, 100) AS blocked_query
FROM pg_stat_activity AS blocked
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0;
```

The PostgreSQL blocking-function reference defines the returned array. The process identifier is a diagnostic clue, not permission to terminate a session. First identify the user, transaction age, query, application, impact, and owner.

## 5.12 Worked Example: Resolve the Controlled Block

1. Confirm that Session B is waiting and record its PID.
2. Record the blocking PID returned by `pg_blocking_pids`.
3. Return to Session A and decide whether its change is valid.
4. Use `ROLLBACK` in the controlled experiment to release the row lock without keeping the priority change.
5. Observe Session B finish.
6. Query ticket 1004 from a fresh session and record the final priority and status.

The final state proves which change remained. The activity view proves the blocking relationship. Neither alone tells the complete incident story.

## 5.13 Deadlock Is Different From One-Way Blocking

A deadlock forms a cycle: Session A holds something Session B needs while Session B holds something Session A needs. Neither can proceed. PostgreSQL detects the cycle and aborts one transaction so the other can continue.

Applications should handle the error and retry the entire transaction when safe. Design can reduce deadlocks by touching resources in a consistent order and keeping transactions short.

Do not create uncontrolled deadlocks in a shared environment. A controlled two-row exercise belongs in a dedicated practice schema with explicit cleanup.

## 5.14 Safe Incident Communication

A useful update separates observation from inference:

> Ticket updates are waiting. At 14:12 UTC, PID 8124 was waiting on a transaction lock and `pg_blocking_pids` identified PID 8071. The blocking transaction began at 14:06 UTC from the SQL editor. We rolled back the controlled Session A change, Session B completed, and a fresh query confirmed ticket 1004 is `in_progress` with its original priority. We will add a transaction timeout and shorten the workflow before repeating the test.

Avoid "the database froze" when the system views show one lock relationship.

## 5.15 Common Misconceptions

### 5.15.1 "Readers never block and writers never wait under MVCC"

MVCC reduces many read/write conflicts. Row writes, explicit locks, and schema changes can still block.

### 5.15.2 "The oldest query is always the blocker"

Age is a clue. Use blocking relationships and lock state, not age alone.

### 5.15.3 "Killing the blocked session fixes the cause"

The blocked session is the waiter. Terminating it may remove the symptom while the blocking transaction remains open.

### 5.15.4 "Serializable means no errors"

Serializable execution can abort a transaction to preserve the guarantee. Correct applications expect and safely retry those failures.

### 5.15.5 "Autovacuum tuning is only a performance decision"

Slowing or delaying vacuum trades disk activity now for dead rows and unfrozen transaction IDs later. If freezing falls far enough behind, PostgreSQL stops accepting writes to prevent wraparound corruption. Treat the age of the oldest unfrozen transaction ID as a correctness metric, not only a performance one.

## 5.16 Study and Practice

Use a timeline to reason about visibility and waiting. Label each connection separately, and distinguish a statement finishing from its transaction committing.

Draw a timeline for Session A and Session B in the worked example. Mark:

- transaction start;
- snapshot or statement reads;
- row update;
- wait start;
- rollback; and
- final verification.

Then write one query you would run in the third session and one conclusion it can support.

Using the Sentry case study, write a short argument in paragraphs: what PostgreSQL was protecting when it stopped accepting writes, what let the maintenance fall behind, whether truncating the table was an acceptable risk, and two concrete prevention steps with the more important one named. Keep availability (is the service up) separate from correctness (is the data right), and ground every claim in the incident report.

## 5.17 Retrieval and Transfer

1. Why can an explicit transaction be both safer and riskier than autocommit?
2. What does MVCC mean for row updates?
3. How does one-way blocking differ from deadlock?
4. Which function directly identifies PostgreSQL blocking PIDs?
5. Why must a serializable application be able to retry?
6. What final check proves which ticket state remained after the incident?
7. Why does PostgreSQL refuse new writes near transaction-ID wraparound, and which number predicts that refusal weeks in advance?

## 5.18 Further Reading

- PostgreSQL transactions
- PostgreSQL transaction isolation
- PostgreSQL explicit locking
- PostgreSQL monitoring statistics
- PostgreSQL routine vacuuming
- PostgreSQL autovacuum configuration
- PostgreSQL system columns
- Sentry: Transaction ID Wraparound in Postgres (2015)

# 6 Access Should Be Useful and Limited

## 6.1 Operating Question

How can we prove that the right actor can perform a required action while the wrong actor is denied, without exposing credentials during the test?

## 6.2 Learning Outcomes

After this chapter, you can:

- distinguish identity, authentication, authorization, and auditing;
- represent access as actor-action-resource conditions;
- create PostgreSQL roles and apply grants using least privilege;
- explain the relationship among Supabase Auth, PostgreSQL roles, and row-level security;
- write and test a beginner RLS policy; and
- handle connection strings and service credentials safely.

## 6.3 Security Questions Need Separate Terms

Imagine two residents using the same repair application. Both have valid accounts; both need to read their requests. If one resident changes a ticket number in a browser URL, the server must still decide whether that person may read the requested row. Hiding a button or using a long identifier does not make the row private. The access decision must hold when the caller sends a different request.

- **Identity:** who or what claims to be acting?
- **Authentication:** how is that claim verified?
- **Authorization:** what may the authenticated actor do?
- **Auditing:** which logs record relevant actions and decisions?

A valid password proves authentication only within its mechanism. It does not mean the account should read every row. A failed query may reflect authentication, authorization, network access, an object name, or SQL syntax. Classify the failure before changing permissions.

These stages explain several different symptoms. “Network unreachable” occurs before the server can decide table permissions. A bad password prevents the intended login. “Permission denied for table” means a reachable database rejected an operation. A query returning no rows may be valid because no row satisfies an access policy. Granting every privilege is not an appropriate response to all four situations.

## 6.4 Model Access as Actor, Action, Resource, and Condition

Replace “give the app access” with a testable statement:

| Actor | Action | Resource | Condition |
|---|---|---|---|
| support agent | `SELECT`, `UPDATE` | assigned tickets | only rows assigned to that agent |
| resident | `SELECT` | tickets | only tickets requested by that resident |
| analyst | `SELECT` | reporting view | no private internal notes |
| migration process | schema changes | application schema | only during controlled deployment |

This matrix reveals different layers. A table grant may permit `SELECT` on the table, while RLS limits which rows the statement can return.

A **privilege** authorizes an operation on an object, such as reading a table. A **policy** can add conditions on which rows that operation may affect. For an ordinary SELECT in the simple resident design, think of the result as the rows satisfying both the application’s question and the resident’s ownership rule.

$$R_{\text{visible}} = \sigma_{\text{requested condition} \wedge \text{owner is current resident}}(R).$$

This is a model of the result, not an instruction to trust an ownership condition supplied by the browser. The database policy is applied even when the caller’s query omits that condition. Object privileges must also permit SELECT; a row policy does not grant table access on its own.

## 6.5 PostgreSQL Roles Represent Users and Groups

PostgreSQL uses roles for both login identities and privilege groups.

**Listing 59. SQL**

```sql
CREATE ROLE metro_analyst NOLOGIN;
GRANT metro_analyst TO CURRENT_USER;
```

This creates a permission group and lets your current administrative login assume it during the controlled test. It does not create a new password or an application account. Run it only in your personal practice environment with the permitted role-management privileges. If the role already exists from your earlier run, inspect or clean up that exercise rather than treating an existing shared role as disposable.

A group role with `NOLOGIN` collects privileges. Login roles become members. This separates identity lifecycle from permission definition.

`session_user` identifies the original database login; `current_user` is the effective database role used for checks such as the test below. They can differ after `SET ROLE`. A person in the Supabase dashboard is another identity again. Keeping these names distinct makes an access test interpretable.

## 6.6 Grant the Minimum Required Privileges

PostgreSQL privileges are object-specific. Accessing a table in a schema can require both schema `USAGE` and table privileges.

Listing 60. SQL

```sql
GRANT USAGE ON SCHEMA metro_support TO metro_analyst;
GRANT SELECT ON metro_support.chapter4_active_queue TO metro_analyst;
```

If the analyst should read only the view, do not also grant broad access to every base table. The view above is introduced in Chapter 4; create it there first, or use the complete limited-view example later in this chapter.

A normal PostgreSQL view generally checks underlying relation access using the view owner's permissions. This can deliberately expose selected columns without granting the caller direct access to the base table. It also means that a view owned by an elevated role is not automatically an RLS-safe resident interface. On PostgreSQL 15 and later, a `security_invoker` view uses the caller's underlying permissions instead. The reporting-view exercise and the resident-policy exercise test different designs, not interchangeable shortcuts.

Revoke privileges that were granted too broadly:

Listing 61. SQL

```sql
REVOKE UPDATE, DELETE
ON metro_support.tickets
FROM metro_analyst;
```

Privileges on future tables are separate from current objects. Default privileges can automate future grants, but they are defined by the object-creating role and must be tested.

Privileges can also come from memberships and `PUBLIC`, the implicit group of all roles. Revoking a direct grant from one role does not erase a privilege it still receives through another path. Therefore, read the effective result of a test instead of treating the presence of a REVOKE statement as a security proof.

## 6.7 Test Both an Allowed and a Denied Action

An allow-only test is incomplete. In a dedicated practice environment:

Listing 62. SQL

```sql
BEGIN;
SET LOCAL ROLE metro_analyst;

SELECT current_user, ticket_id, status
FROM metro_support.chapter4_active_queue;
ROLLBACK;
```

Then test the denied operation in its own transaction on the same connection:

Listing 63. SQL

```sql
BEGIN;
SET LOCAL ROLE metro_analyst;

-- This should fail if no base-table update privilege was granted.
UPDATE metro_support.tickets
SET priority = 'urgent'
WHERE ticket_id = 1001;

-- Execute this even if the client stops after the expected error.
ROLLBACK;
```

The expected error is an insufficient-privilege error, SQLSTATE `42501`, not a misspelled table or a broken connection. Rollback ends the transaction and its local role setting. Keep actor, query, and transaction on one connection: separate editor executions may use different pooled sessions. If an editor hides intermediate results, use a client that can display each result on the same connection; do not split a transaction across unrelated pooled sessions.

## 6.8 A Complete Row-Level Security Experiment

A table grant answers whether an actor may read a table. Row-level security (RLS) adds a condition on the rows that actor may see. This experiment uses three synthetic requests and two residents to make the distinction visible. It needs no application, cloud account, notebook, or Metro Support fixture.

Use the local PostgreSQL server and `psql` connection described in Appendix C. Connect as your practice database administrator, who can create roles, and keep this one connection open throughout. Do not use a shared database or split the transaction across pooled SQL-editor requests. The two practice roles are not login accounts: we assume them with `SET LOCAL ROLE` to test authorization, not password authentication.

### 6.8.1 Create the Isolated Practice Data

Run the blocks in order. `BEGIN` keeps all changes uncommitted until the final rollback, including the new roles. If a practice name already exists, stop and run `ROLLBACK;` rather than deleting an object you did not create. Do not replace the final rollback with a commit.

Listing 64. SQL

```sql
BEGIN;
CREATE ROLE book_resident_maya NOLOGIN NOSUPERUSER NOBYPASSRLS;
CREATE ROLE book_resident_jordan NOLOGIN NOSUPERUSER NOBYPASSRLS;
GRANT book_resident_maya, book_resident_jordan TO CURRENT_USER;

CREATE SCHEMA book_rls_demo;
CREATE TABLE book_rls_demo.requests (
    request_id integer PRIMARY KEY,
    owner_role text NOT NULL,
    summary text NOT NULL
);
INSERT INTO book_rls_demo.requests VALUES
    (1, 'book_resident_maya', 'Kitchen tap leaks'),
    (2, 'book_resident_jordan', 'Hallway light is out'),
    (3, 'book_resident_maya', 'Window will not close');
```

The administrative login owns the table. Neither resident owns it or has the privilege to bypass RLS. Testing only as the owner would miss the access boundary we want to examine.

Listing 65. SQL

```sql
GRANT USAGE ON SCHEMA book_rls_demo
TO book_resident_maya, book_resident_jordan;
GRANT SELECT, INSERT ON book_rls_demo.requests
TO book_resident_maya, book_resident_jordan;

ALTER TABLE book_rls_demo.requests ENABLE ROW LEVEL SECURITY;
CREATE POLICY read_own_requests ON book_rls_demo.requests
FOR SELECT TO book_resident_maya, book_resident_jordan
USING (owner_role = current_user);

CREATE POLICY insert_own_requests ON book_rls_demo.requests
FOR INSERT TO book_resident_maya, book_resident_jordan
WITH CHECK (owner_role = current_user);
```

The grants permit reading and inserting; they do not permit updating or deleting. `USING` checks existing rows for SELECT. `WITH CHECK` checks a proposed new row for INSERT. Both rules compare the stored owner with the effective database identity. A policy is not a substitute for the object grants. PostgreSQL row-security rules

### 6.8.2 Read as Each Resident

**Listing 66. SQL**

```sql
SET LOCAL ROLE book_resident_maya;
SELECT current_user, request_id, summary
FROM book_rls_demo.requests ORDER BY request_id;

SELECT request_id FROM book_rls_demo.requests
WHERE request_id = 2;
```

The first query returns requests 1 and 3, both with `book_resident_maya` as `current_user`. It contains no ownership filter: the policy supplies that restriction. The second query returns zero rows. Requester 101 cannot retrieve Requester 104's request simply by guessing its identifier; the database does not expose that row and then report a permission error.

**Listing 67. SQL**

```sql
RESET ROLE;
SET LOCAL ROLE book_resident_jordan;
SELECT current_user, request_id, summary
FROM book_rls_demo.requests ORDER BY request_id;
```

Requester 104 sees only request 2. `RESET ROLE` returns to the administrative identity before the next test. `SET LOCAL ROLE` changes the effective role for this transaction, not the original authenticated login. Here the administrator is deliberately acting as each resident; this does not grant Requester 101 membership in Requester 104's role. PostgreSQL SET ROLE

### 6.8.3 Allow One Insert and Reject Another

Still acting as Requester 104, insert a request owned by that same role:

**Listing 68. SQL**

```sql
INSERT INTO book_rls_demo.requests VALUES
    (4, 'book_resident_jordan', 'Bedroom radiator is cold');
SELECT request_id FROM book_rls_demo.requests ORDER BY request_id;
SAVEPOINT before_wrong_owner;
```

The query now returns 2 and 4. The savepoint marks a position to which we can return after the next, intentionally rejected statement. Run the next block **by itself**; it is supposed to produce an error.

**Listing 69. SQL**

```sql
INSERT INTO book_rls_demo.requests VALUES
    (5, 'book_resident_maya', 'A request with a false owner');
```

Expect an error stating that the new row violates the row-level security policy, with SQLSTATE `42501`. That is different from a missing table or a connection failure. A PostgreSQL error leaves the transaction in a failed state; ordinary queries cannot continue until you roll back. In the same connection, run:

**Listing 70. SQL**

```sql
ROLLBACK TO SAVEPOINT before_wrong_owner;
SELECT request_id FROM book_rls_demo.requests ORDER BY request_id;
```

The result is still 2 and 4. Rolling back to the savepoint preserves the allowed insert performed before it, while recovering from the rejected operation. If the transaction was already ended or the connection closed, the savepoint no longer exists; start the experiment again rather than skipping setup. PostgreSQL savepoint recovery

### 6.8.4 Verify and Remove the Practice Objects

Listing 71. SQL

```
RESET ROLE;
SELECT request_id, owner_role
FROM book_rls_demo.requests ORDER BY request_id;
ROLLBACK;

SELECT to_regnamespace('book_rls_demo') IS NULL AS schema_removed;
SELECT count(*) AS remaining_practice_roles FROM pg_roles
WHERE rolname IN ('book_resident_maya', 'book_resident_jordan');
```

As the table owner, the administrator sees 1, 2, 3 and 4, but never the rejected request 5. The final rollback removes this transaction's schema, data, policies, grants and roles. The two cleanup results should be `true` and `0`. If you stop early, `ROLLBACK;` also abandons the uncommitted experiment; disconnecting without committing does the same.

This design intentionally uses one database role per resident to isolate how RLS works. It is not a prescription for a web application's login architecture. The next section explains how Supabase distinguishes many application users who share one database role.

## 6.9 Supabase Adds an Identity Layer to PostgreSQL

Supabase Auth can issue JSON Web Tokens for application users. Requests through Supabase's data APIs are mapped to PostgreSQL roles such as `anon` or `authenticated`, and claims can be read by policy helpers such as `auth.uid()`.

A **token** is a value the client presents with a request. A JSON Web Token, or JWT, carries claims such as a user identifier; the receiving service verifies the token before using those claims. A signed token is not necessarily encrypted. Treat a usable login token as a credential, even when its payload can be decoded. The browser must not be allowed to establish identity merely by writing an arbitrary user ID in the request body.

This creates distinct concepts:

- a Supabase dashboard account administers a project;
- a PostgreSQL role controls database privileges;
- an application user exists in the authentication system;
- a token carries claims about a request; and
- an RLS policy evaluates row access inside PostgreSQL.

Do not treat these identities as interchangeable.

In a small role-based demonstration, each resident could have a separate PostgreSQL role, with an `owner_role` column checked against `current_user`. That compares database identities, not application login identities. A normal Supabase application instead serves many people through the shared `authenticated` role and uses verified user claims to distinguish them. It does not need one database role for every resident.

The Supabase SQL editor commonly runs with elevated ownership privileges. Table owners and roles with `BYPASSRLS` can bypass row-level security. A successful query in the editor therefore does not prove what an anonymous or authenticated client can see.

## 6.10 Row-Level Security Adds a Row Predicate

The following is an **illustrative Supabase adaptation**, not a command to paste into the unchanged Metro Support fixture. That fixture has integer requester IDs, not Supabase Auth UUIDs. A UUID is a 128-bit identifier, commonly displayed as groups of hexadecimal characters; it is not a password.

For this adaptation, assume that a reviewed migration has created and populated `requester_auth_id uuid` with actual application-user identities, the appropriate schema is exposed to the intended API, and SELECT has been explicitly granted to `authenticated`. Adding a nullable column without mapping existing residents would not establish correct ownership.

Listing 72. SQL

```sql
ALTER TABLE metro_support.tickets ENABLE ROW LEVEL SECURITY;

CREATE POLICY residents_read_own_tickets
ON metro_support.tickets
FOR SELECT
TO authenticated
USING (requester_auth_id = (SELECT auth.uid()));
```

`USING` controls which existing rows are visible for the operation. `WITH CHECK` controls which new or changed row states may be created for relevant operations.

With no authenticated user, `auth.uid()` can be NULL. Equality with NULL is not true, so this ownership condition does not select a row. That is the same three-valued logic introduced in Chapter 2, now serving an access boundary.

Listing 73. SQL

```sql
CREATE POLICY residents_insert_own_tickets
ON metro_support.tickets
FOR INSERT
TO authenticated
WITH CHECK (requester_auth_id = (SELECT auth.uid()));
```

Policy design is default-deny after RLS is enabled: if no applicable policy allows the operation, ordinary roles cannot perform it. Verify current Supabase guidance and test through the same role and request path the application uses.

An INSERT also needs an INSERT grant. If it supplies a different user's ID, the `WITH CHECK` condition rejects the new row. A SELECT of another user's existing row ordinarily returns no matching row rather than revealing it and then raising an error. This is why access testing must compare visible identities as well as error messages. Supabase RLS guide

Do not solve a failed ownership test by adding `USING (true)`. That rule allows every row for its covered operation. Multiple permissive policies combine with OR, so one broad policy can undo the restriction expected from another. Owners normally bypass RLS; superusers and roles with `BYPASSRLS` bypass it. RLS is not a defense against an unrestricted database administrator. PostgreSQL row-security rules

## 6.11 Worked Example: Build an Access Test Matrix

**Requirement:** an analyst may read ticket identifiers, category, priority, status, and opening time, but not resident email addresses or internal event notes.

### 6.11.1 Design

Create a limited view:

Listing 74. SQL

```sql
CREATE OR REPLACE VIEW metro_support.analyst_ticket_summary AS
SELECT ticket_id, category, priority, status, opened_at, closed_at
FROM metro_support.tickets;

GRANT USAGE ON SCHEMA metro_support TO metro_analyst;
GRANT SELECT ON metro_support.analyst_ticket_summary TO metro_analyst;
REVOKE ALL ON metro_support.users FROM metro_analyst;
REVOKE ALL ON metro_support.ticket_events FROM metro_analyst;
```

### 6.11.2 Test

| Test | Expected |
|---|---|
| select from `analyst_ticket_summary` | allowed |
| select email from `users` | denied |
| select note from `ticket_events` | denied |
| update a ticket | denied |

### 6.11.3 Interpret

If the expected allow succeeds and every expected deny fails for permission reasons, the results support the stated boundary. They do not demonstrate protection against a privileged administrator, a leaked credential, a vulnerable function, or an untested API path.

## 6.12 Secret Handling Is a Database Skill

A connection URI may include host, database, user, and password. Treat the whole string as a secret when it contains credentials.

Safe notebook pattern:

Listing 75. Python

```python
from getpass import getpass

database_url = getpass("Paste the temporary database URL: ")
```

Do not hardcode a secret, print it, save it in notebook output, or place it in a GitHub issue. Use environment variables or platform secret storage in applications.

Supabase distinguishes publishable client keys from secret server keys; older projects may also show legacy `anon` and `service_role` keys. A publishable key identifies an application component, not a particular resident. A user's signed login token supplies the user context. Secret keys and legacy service-role keys

provide elevated access and must not be shipped in browser code. Check the key type, not just whether a string is called an "API key." Supabase API-key guide

If a secret reaches Git history, revoke or rotate it first. Removing the visible line does not make the old credential safe.

## 6.13 Network Controls Complement Authorization

A remote connection crosses several gates. The hostname must resolve; the network must reach the service; TLS must establish the intended protected connection; authentication must succeed; and authorization must permit the operation. Passing one gate does not establish that the remaining gates pass.

Atlas IP access lists, database users, TLS, PostgreSQL connection controls, and platform network restrictions reduce who can reach a service. They do not replace least-privilege database authorization.

For temporary practice allow-list rules, use the narrowest practical scope and remove them after the activity. An allow-from-anywhere rule may be convenient, but it increases exposure and still requires strong credentials and database permissions.

## 6.14 Common Misconceptions

### 6.14.1 "RLS is enabled, so the policy works"

Enablement and a policy definition are only configuration. Test the intended client role, token claims, allowed rows, and denied rows.

### 6.14.2 "The `authenticated` role identifies one person"

It represents a class of requests. Policies commonly use token claims such as the user ID to distinguish rows.

### 6.14.3 "A public client key is the same as a service-role key"

They have different powers and intended locations. Treat service credentials as secrets and never expose them in client code.

### 6.14.4 "More privileges will fix the error"

Broad grants may hide the actual problem and create a security defect. Identify the actor, object, action, and expected policy first.

## 6.15 Study and Practice

An access policy needs positive and negative tests. State which identity is being tested, which resource it addresses, and whether the expected decision comes from an object privilege or a row policy.

Choose one Metro Support actor. Write an access matrix with two allowed actions and two denied actions. For each action, identify:

- the PostgreSQL object or API resource;
- the grant or policy layer involved;

- the exact test; and
- what the result would and would not prove.

Chapter 5's case study of Sentry's 2015 wraparound outage ends with engineers running `TRUNCATE` under pressure with the most privileged credentials available. Add one row to your matrix for that action: which actor holds it, which ownership or grant makes it possible, and what evidence that actor would want before using it.

## 6.16 Retrieval and Transfer

1. How do authentication and authorization differ?
2. Why can a PostgreSQL group role use `NOLOGIN`?
3. Why should an access test include a denied action?
4. What is the difference between `USING` and `WITH CHECK` in RLS?
5. Why can the Supabase SQL editor be a misleading RLS test path?
6. What should happen first after a secret is committed to Git?

## 6.17 Further Reading

- PostgreSQL roles
- PostgreSQL privileges
- PostgreSQL row security
- Supabase row-level security
- Supabase API security

# 7 Performance Work Begins With Measurement

## 7.1 Operating Question

When a query feels slow, what observations can distinguish a scan, a bad estimate, an expensive join, an avoidable sort, blocking, or simply more work than expected?

## 7.2 Learning Outcomes

After this chapter, you can:

- turn a vague complaint into a reproducible query and workload statement;
- distinguish estimated plans from plans with actual execution measurements;
- recognize common scan, join, sort, and aggregate plan nodes;
- connect selectivity and statistics to planner choices;
- design and test a basic single-column, composite, or partial index; and
- decide whether an improvement justifies its write, storage, and maintenance cost.

## 7.3 "Slow" Is a Symptom, Not a Diagnosis

Begin with a reproducible statement:

- Which exact query and parameter values?
- Which database, schema, and data volume?
- What result grain and row count?
- What elapsed time or service objective?
- Is the time spent executing, waiting for a lock, transferring many rows, or rendering in a client?
- Is the behavior repeatable, and what changed?

A query returning a million rows may be fast for the database and slow for the network or browser. A query waiting on a lock may have a simple plan. Measure the right layer.

The repair desk wants its twenty newest open requests, not a faster version of a different answer. That distinction defines the experiment: keep the query's meaning and data fixed, then change how the DBMS can find the rows. The small twelve-ticket fixture is useful for checking the answer. The separate indexing workload in Appendix C supplies 100,000 synthetic rows, including 2,000 open tickets, to make differences in search work visible. It is a teaching workload, not a benchmark of city services or a cloud vendor.

## 7.4 SQL Describes an Answer; a Plan Describes Work

Chapter 2 used relational algebra to explain what a query means. A physical plan chooses algorithms and access paths that produce that answer. A join could compare rows repeatedly, probe an index, build a hash table, or merge ordered inputs. The correct result should not depend on which suitable algorithm wins. This

separation is part of data independence: an administrator can add an index without requiring every application to rewrite its question.

Cost-based optimization compares estimated alternatives rather than applying "always use an index." The classic System R work by Selinger and colleagues described access-path selection for relational queries in 1979. Its relevance here is the separation of a declarative request from the physical strategy that executes it, not a claim that a modern PostgreSQL plan is identical to System R. IBM's research record

## 7.5 `EXPLAIN` Shows the Planner's Strategy

Listing 76. SQL

```sql
EXPLAIN
SELECT ticket_id, subject, opened_at
FROM metro_support.tickets
WHERE status = 'open'
ORDER BY opened_at DESC;
```

Plain `EXPLAIN` estimates a plan without executing the planned statement. Use the book's trusted SQL: planning can still require locks and evaluate eligible constant expressions, so it is not a general sandbox for arbitrary code.

Listing 77. SQL

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_id, subject, opened_at
FROM metro_support.tickets
WHERE status = 'open'
ORDER BY opened_at DESC;
```

`ANALYZE` executes the statement and reports actual timing and rows. On `UPDATE`, `DELETE`, or function calls, that means real side effects unless protected and rolled back. Use it only when execution is safe. `BUFFERS` reports cache and I/O activity available to the statement.

A transaction and rollback can undo ordinary transactional data changes made by an analyzed write, but they do not make external side effects or sequence allocation disappear. In these experiments, measure SELECT queries in disposable data. Do not benchmark a production DELETE merely because its plan is interesting.

## 7.6 Read a Plan From the Inside Out

A plan is a tree. Child nodes produce rows for parent nodes.

Common nodes include:

- **Seq Scan:** inspect table pages and test rows. Often correct for a small table or a query returning much of the table.
- **Index Scan:** use an index to find row locations, then access table rows.
- **Index Only Scan:** answer from index entries when required columns and visibility information permit it.
- **Bitmap Index/Heap Scan:** collect many index matches and visit table pages in batches.
- **Nested Loop:** for each row from one input, scan or probe the other input.
- **Hash Join:** build a hash table for one input and match the other, commonly for equality joins.

- **Merge Join:** consume sorted inputs and merge matching keys.
- **Sort:** order rows; may use memory or spill to temporary storage.
- **Aggregate/GroupAggregate/HashAggregate:** reduce input rows into summaries.

Node names are not grades. A sequential scan is not automatically bad, and an index scan is not automatically good.

### 7.6.1 Follow the Rows Through One Plan

The following excerpt comes from the book's fixture on an isolated PostgreSQL 15 server during the September 7, 2026 audit. Timings and estimates will vary on another run. The text is abbreviated to expose the rows and algorithm, not to hide an unfavorable measurement.

**Listing 78. Text**

```
Limit (actual rows=20 loops=1)
  -> Sort (actual rows=20 loops=1)
       Sort Key: opened_at DESC
       Sort Method: top-N heapsort
       -> Seq Scan on tickets (actual rows=2000 loops=1)
            Filter: status = 'open'
            Rows Removed by Filter: 98000
```

The scan tests 100,000 rows, keeps 2,000, and rejects 98,000. The sort receives 2,000 candidates. A top-N sort can retain a small set of best candidates rather than fully sort every row, but it still has to inspect the input to know which are newest. The parent asks for twenty rows, so only twenty emerge at the top. Seeing `rows=20` at the sort does not mean only twenty candidates were considered.

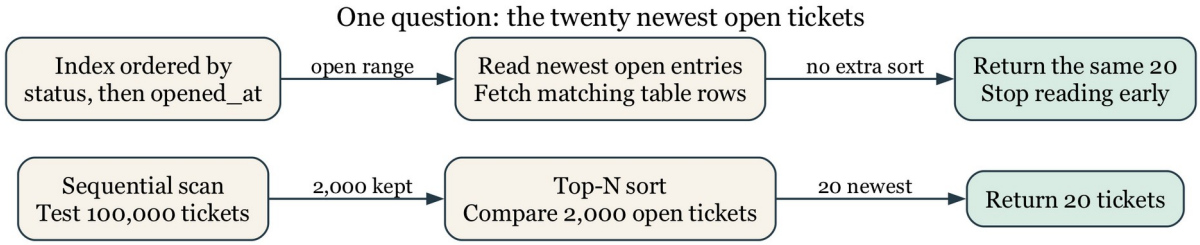



*Figure 7.1: The same twenty-row answer can require scanning and comparing many candidates or walking an index in the needed order. Arrows show row flow, not network traffic.*

For a node repeated by a nested loop, actual rows and time are reported per loop on average. A child with `rows=3 loops=100` produces about 300 rows across those invocations, not three total. Parent time includes work in its children, so adding every node's time double-counts work. Likewise, a shared buffer **hit** means PostgreSQL found a page in its buffer cache; a **read** does not necessarily mean the physical disk was accessed, because the operating system may cache it. PostgreSQL EXPLAIN guide

## 7.7 Estimates Drive Choices

The planner estimates row counts and costs using table statistics, data type information, constraints, and configurable assumptions. Compare estimated rows with actual rows in an analyzed plan.

Large mismatches may come from:

- stale statistics;

- correlated columns that basic statistics treat as independent;
- unusual parameter values;
- skewed data;
- expressions without useful statistics; or
- a model that hides important relationships.

`ANALYZE` refreshes statistics:

**Listing 79. SQL**

```
ANALYZE metro_support.tickets;
```

Do not run maintenance blindly as a ritual. Identify the mismatch and verify the effect.

The fixture's first scan estimated 2,017 open rows and produced 2,000. That is a reasonable estimate, even though the plan still had to read the whole table. After refreshing statistics, another estimate was 1,953. Sampling makes small differences normal. A useful question is whether an error is large enough to favor an unsuitable join or access path, not whether every estimate is exact.

The displayed `cost=startup..total` uses the planner's cost units, not milliseconds. Startup cost estimates work before the first row; total cost estimates work to produce the full node result. LIMIT can favor a path that starts quickly and stops early even if that path's full-result cost is higher.

## 7.8 Selectivity Explains Many Scan Decisions

Selectivity is the fraction of rows expected to satisfy a condition. An index is often attractive when a query needs a small, identifiable fraction of a large table. If nearly every row has `status = 'open'`, using an index may still require visiting most table pages, making a sequential scan reasonable.

For predicate $\varphi$ over relation $R$, define selectivity as:

$$s(\varphi) = \frac{|\sigma_{\varphi}(R)|}{|R|}, 0 \le s(\varphi) \le 1.$$

A predicate matching 500 rows in a 1,000,000-row table has $s(\varphi) = 0.0005$, or $0.05\%$. Selectivity alone does not choose a plan, but it helps explain why an index can be attractive for one predicate and wasteful for another.

Here $|R|$ means the number of rows and $\sigma$ means selection, or keeping rows that match the predicate. The ratio is defined for a nonempty relation; there is no need to divide by zero for an empty table.

The twelve-row baseline dataset is too small to demonstrate realistic planner tradeoffs. The indexing workload in Appendix C creates a larger, disposable table with a skewed status distribution. Small examples teach correctness; larger fixtures reveal access paths.

## 7.9 B-Tree Indexes Support Ordered Comparisons

PostgreSQL's default B-tree index supports equality and range comparisons and can also help ordered retrieval.

A B-tree is a balanced search structure organized into pages. At an internal page, comparisons choose a smaller range of possible keys. Leaf entries lead to matching table rows and support movement through neighboring ordered keys. The tree stays relatively shallow because a page can direct searches to many child pages. This is different from reading every ticket and testing its status.

For a simplified model with roughly $b$ children per internal page and $N$ keys, the number of levels grows on the order of $\log_b N$, rather than $N$. The notation says how growth behaves; it is not a prediction of milliseconds or an exact PostgreSQL page count. Retrieving many matches still requires processing those matches and possibly visiting many table pages.

**Listing 80. SQL**

```
CREATE INDEX perf_tickets_status_opened_idx
ON performance_lab.tickets (status, opened_at DESC);
```

A composite index is ordered first by `status`, then by `opened_at` within each status. It may efficiently support this query fragment, which is not a standalone SQL statement:

**Listing 81. SQL**

```
WHERE status = 'open'
ORDER BY opened_at DESC
LIMIT 20
```

It is less directly suited to a query that filters only `opened_at` without the leading column. Column order should follow real predicates and ordering needs, not a rule such as “most unique first” applied without context.

It is also not correct to say a later column can *never* help without the first. Planner choices depend on the version and distribution; PostgreSQL 18 documents B-tree skip-scan cases. This chapter’s comparison does not depend on that feature. Multicolumn indexes

An index containing `(status, opened_at)` does not also contain every subject text. An ordinary index scan locates entries and fetches the table rows. A covering index can include extra output columns, but larger entries cost space and write work. Even an index-only scan may need table visits to establish row visibility under MVCC. The node name is not a promise of zero heap fetches. Index-only scans

## 7.10 Partial Indexes Cover a Deliberate Subset

If most tickets are closed but the operational queue reads only active rows, a partial index may be smaller:

**Listing 82. SQL**

```
CREATE INDEX perf_tickets_active_opened_idx
ON performance_lab.tickets (opened_at DESC)
WHERE status IN ('new', 'open', 'in_progress');
```

The query predicate must imply the index predicate for the planner to use it. Partial indexes add design specificity: if the application’s definition of active changes, the index and queries may need coordinated revision.

## 7.11 Indexes Have Costs

Each useful index can add:

- storage;
- write work on inserts, updates, and deletes;
- write-ahead log volume;
- backup size or duration;
- cache competition; and
- maintenance and cognitive overhead.

Duplicate, unused, or speculative indexes are not free. Start with a defined workload and baseline measurements.

Not every column update changes every index entry, and PostgreSQL can sometimes use heap-only tuple optimizations when indexed values are unchanged and page conditions permit. The beginner design question remains: does the benefit to the important reads justify maintaining another data structure? A small read-only test does not measure the cost of a busy production write workload.

## 7.12 Worked Example: Test an Index Hypothesis

**Workload:** list the 20 newest open tickets in a large operational table.

Start with the complete disposable setup in Appendix C. The preceding index definitions are examples to study, not extra indexes to leave installed before the baseline. Running the setup resets only `performance_lab` and removes its earlier indexes. The setup also refreshes table statistics. Measure the baseline twice before creating an index, then avoid another statistics refresh between measurements. This keeps the proposed access path as the deliberate change. Cache state and background activity may still vary, so it is not a controlled hardware benchmark.

### 7.12.1 Baseline

**Listing 83. SQL**

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_id, subject, opened_at
FROM performance_lab.tickets
WHERE status = 'open'
ORDER BY opened_at DESC
LIMIT 20;
```

Record:

- scan node;
- estimated and actual rows at the scan;
- whether a sort occurs;
- planning and execution time; and
- buffer activity.

### 7.12.2 Hypothesis

An index on `(status, opened_at DESC)` can locate open rows in requested order and may avoid scanning and sorting unrelated rows.

### 7.12.3 Add the Candidate

**Listing 84. SQL**

```sql
CREATE INDEX perf_tickets_status_opened_idx
ON performance_lab.tickets (status, opened_at DESC);
```

### 7.12.4 Remeasure

Run the identical analyzed query twice. Do not change the selected columns, predicate, order, or limit while comparing.

The corresponding audited plan was:

**Listing 85. Text**

```
Limit (actual rows=20 loops=1)
  -> Index Scan using perf_tickets_status_opened_idx on tickets
       (actual rows=20 loops=1)
       Index Cond: status = 'open'
```

The index supplies matching rows in opening-time order, so no separate sort is needed and the parent stops after twenty. The earlier run recorded 1,191 shared buffer hits; this indexed run recorded 11 hits and four reads. These are local observations of this fixture, not a service-level guarantee. Confirm the same twenty ticket IDs as well as the reduced work. A faster query returning different requests is not an optimization of the original question.

### 7.12.5 Decide

Keep the index only if the workload benefit is meaningful relative to its cost. A changed node name alone is not enough. On a warm cache and small practice fixture, timing can vary; plan shape, rows, sort removal, and buffers may be more stable indicators.

## 7.13 Separate Query Tuning From Lock Diagnosis

An analyzed plan reports execution after the statement can proceed. If the query waits on another transaction, `pg_stat_activity`, wait events, and `pg_blocking_pids` may be the first useful diagnostic. Do not add an index to solve a transaction left open in a client.

## 7.14 Common Misconceptions

### 7.14.1 “Sequential scan means the index is missing”

The table may be small, the condition unselective, the statistics reasonable, or the available index mismatched to the query.

### 7.14.2 “Lower estimated cost is milliseconds”

Planner cost units are relative estimates, not elapsed-time units.

### 7.14.3 "`EXPLAIN ANALYZE DELETE` is only an explanation"

`ANALYZE` executes the statement. Use a safe copy and transaction or avoid it.

### 7.14.4 "One fast run proves the change"

Caching, concurrent load, and measurement noise affect timing. Repeat carefully and compare multiple measurements.

## 7.15 Study and Practice

Begin an index decision with the workload, not with a list of available index types. Explain which rows must be found, how they must be ordered, and which measurement would support or reject the proposed index.

For a query that filters `assignee_id`, selects active statuses, orders newest first, and returns 25 rows:

1. state the result grain and workload;
2. propose a composite or partial index;
3. predict which plan work it may remove;
4. name three before/after observations; and
5. state one write or maintenance cost.

## 7.16 Retrieval and Transfer

1. How does `EXPLAIN` differ from `EXPLAIN ANALYZE`?
2. Why might PostgreSQL choose a sequential scan when an index exists?
3. What does a large estimated-versus-actual row mismatch suggest?
4. Why does composite-index column order matter?
5. When can a partial index be useful?
6. Which observations would tell you the query is waiting rather than scanning slowly?

## 7.17 Further Reading

- PostgreSQL `EXPLAIN`
- PostgreSQL planner statistics
- PostgreSQL index types
- PostgreSQL multicolumn indexes
- PostgreSQL partial indexes

# 8 A Backup Matters Only When Recovery Works

## 8.1 Operating Question

If the original database disappears or a harmful change commits, what artifact can recreate the required state, where can it be restored safely, and which checks confirm that the result is usable?

## 8.2 Learning Outcomes

After this chapter, you can:

- distinguish high availability, backup, restore, and disaster recovery;
- use recovery point and recovery time objectives to clarify a promise;
- explain logical and physical backup tradeoffs;
- create a free-tier-appropriate PostgreSQL logical backup command;
- restore into a separate target and verify structure, data, and behavior; and
- write a concise runbook with prerequisites, safety boundaries, and verification checks.

## 8.3 Four Mechanisms Solve Different Problems

At 14:17, an administrator commits a DELETE with the wrong condition. The database is still running, the network is healthy, and other queries work. A server restart will not bring back the deleted rows: from the DBMS's perspective, the deletion was an authorized, committed change. The team needs an earlier recoverable state and a decision about which later valid changes to preserve. This is a recovery problem even though it is not a hardware outage.

- **High availability** reduces service interruption when a component fails.
- **Backup** creates recoverable data outside the active state.
- **Restore** reconstructs data or objects from a backup artifact.
- **Disaster recovery** combines people, systems, procedures, alternate locations, and tested decisions for a serious failure.

A replica can quickly copy an accidental deletion. A backup can exist but be corrupt, incomplete, inaccessible, or impossible to restore within the required time. Redundancy and recovery complement each other.

## 8.4 RPO and RTO Turn "We Have Backups" Into a Promise

**Recovery Point Objective (RPO)** is the maximum acceptable data-loss window. A daily export may imply up to roughly one day of lost changes, depending on when failure occurs.

**Recovery Time Objective (RTO)** is the target time to restore an acceptable service. It includes obtaining credentials, provisioning a target, transferring data, restoring, verifying, and reconnecting consumers.

RPO and RTO are requirements, not properties automatically created by writing them down. Backup frequency, artifact retention, restore speed, and staffing must support them.

If $t_f$ is the failure time and $t_r$ is the timestamp of the newest recoverable state, the realized data-loss window is:

$$\Delta_{\text{loss}} = t_f - t_r.$$

The recovery design meets an RPO target $RPO_{max}$ only when $\Delta_{\text{loss}} \leq RPO_{max}$. If service becomes acceptable again at $t_a$, the realized recovery duration is $\Delta_{\text{recovery}} = t_a - t_f$ and must be compared with the RTO target. These inequalities turn vague confidence into something that can be measured during a restore drill.

### 8.4.1 Work Through the Clock Times

Suppose the newest usable export represents the database at 14:00. The harmful deletion occurs at 14:17, is recognized at 14:21, and the team finishes restoration and verification at 14:29. The recoverable-data gap is 17 minutes. The service-recovery interval measured from the failure is 12 minutes, not just the eight minutes spent restoring and checking after detection. An RPO of five minutes and an RTO of ten minutes would both be missed.

The 17-minute gap does not tell us how many records were lost. A quiet period and a burst of thousands of writes can have the same time gap. It also does not mean that every write after 14:00 is necessarily unrecoverable: another supported history source might preserve some. State which sources the calculation assumes. For a running logical export, the state represented is its consistent snapshot, not automatically the wall-clock time when the output file finished writing.

RPO and RTO therefore start a conversation about a service's needs. A historical practice dataset may tolerate yesterday's state; an active request desk may not. The target must be justified by the consequences of lost work and unavailable service, not copied from a cloud provider's marketing page.

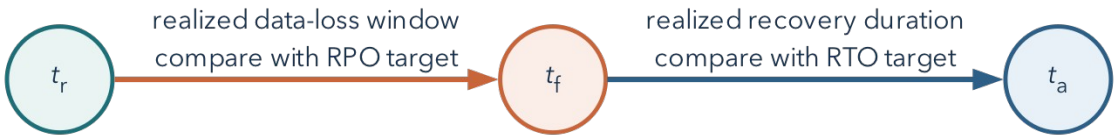



*Figure 8.1: RPO constrains the recoverable-data gap before failure; RTO constrains the service-recovery interval after failure.*

## 8.5 Choose the Failure Scope First

Ask what must be recovered:

- one table after an incorrect delete;
- one schema after a migration failure;
- an entire database after project loss;
- credentials and permissions after access corruption;
- a service in a different region; or
- an application-consistent state spanning multiple systems.

The correct artifact and procedure depend on scope. A CSV export of one table may help recover rows but does not preserve keys, constraints, indexes, views, functions, roles, or transactionally consistent relationships by itself.

## 8.6 Logical and Physical Backups

### 8.6.1 Logical Backup

Logical tools export SQL objects and data in a form the DBMS can reconstruct. PostgreSQL uses `pg_dump` for one database and `pg_dumpall` for cluster-wide logical content such as roles, within supported permissions.

Benefits include object-level selection and portability across some PostgreSQL versions. Costs can include longer export/restore time and incomplete coverage of cluster-level configuration.

The logical export reconstructs the meaning of objects rather than copying the server's live storage files byte for byte. It can recreate a table, load its rows, and recreate its indexes. A consistent database snapshot keeps related exported rows from representing unrelated moments. By contrast, independently downloading several CSVs while the application changes can produce a child event whose parent ticket is absent from the earlier parent export.

### 8.6.2 Physical Backup

Physical backups copy database storage in a format tied more closely to the DBMS and version, often combined with write-ahead logs for point-in-time recovery. They can support large-system recovery and precise points but require platform support, storage coordination, and operational expertise.

Managed services may expose snapshots or point-in-time recovery only on selected plans. A recovery design must account for the features actually available on its plan. The logical-recovery examples below use free tools.

Write-ahead logging records information needed to recover database changes. A supported physical recovery design can restore a base backup and replay a continuous sequence of retained log records to a selected point. The log is not an ordinary spreadsheet of past business events. Losing required segments or using an incompatible base backup can break the recovery chain. Setting up that production infrastructure is beyond the small logical-recovery examples here. The distinction explains why a single CSV cannot replace it.

## 8.7 Free-Tier Recovery

Supabase documentation states that projects on the Free plan do not receive the automatic database backups available on paid plans. Logical backup and restore provide a way to practice recovery without purchasing a hosted backup feature.

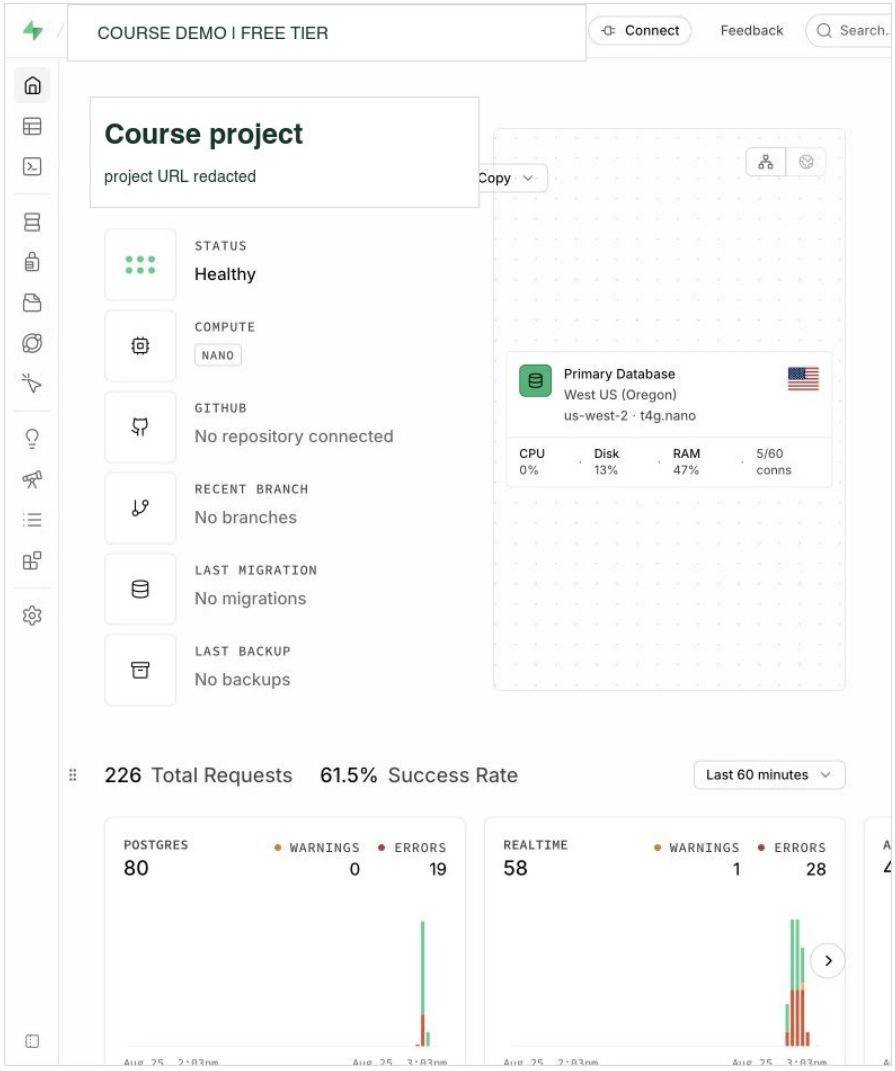



*Figure 8.2: A live Supabase Free project reports a healthy database and "No backups" at the same time. Health is present availability; backup is retained recovery history. Project identifiers are redacted. Interface captured August 25, 2026.*

The two status fields describe different aspects of the system. **Healthy** describes present availability. **No backups** reports the absence of a retained backup in this displayed project history. Neither field establishes that a separately stored export exists or can be restored. A logical recovery drill tests those additional requirements.

Platform offerings change. Before relying on a backup feature, check the current plan documentation and record that dependency in the recovery plan.

The optional recovery notebook runs a real PostgreSQL dump and separate restore in disposable local databases. It does not require a paid hosted backup feature. The command-line examples below explain the same tools and provide a reference for a permitted remote source. They are shell commands, not SQL to paste into a web SQL editor.

## 8.8 Use `pg_dump` Without Publishing a Password

Obtain the source hostname, port, database name, and user from the approved connection panel. Set the non-password shell variables `PGHOST`, `PGPORT`, `PGDATABASE`, and `PGUSER` to those values. `--password` asks

interactively for the password without putting it in the command itself. Do not save a full password-bearing URI in a script or pass it as a visible command-line argument.

For a remote Supabase connection, use a reachable direct endpoint or the IPv4-compatible **session pooler** where needed, and the documented TLS settings. Do not switch off certificate verification to address a network-routing error. Use compatible PostgreSQL client tools: the same major version as the source is a straightforward starting choice. An older `pg_dump` cannot dump a newer-major server. A restore to an older-major server is not promised to work.

Custom-format backup of the baseline schema:

**Listing 86. Shell**

```
pg_dump \
  --format=custom \
  --schema=metro_support \
  --file=metro_support.dump \
  --host="$PGHOST" --port="$PGPORT" \
  --username="$PGUSER" --dbname="$PGDATABASE" --password
```

Plain SQL backup of one schema:

**Listing 87. Shell**

```
pg_dump \
  --format=plain \
  --schema=metro_support \
  --no-owner \
  --no-privileges \
  --file=metro_support.sql \
  --host="$PGHOST" --port="$PGPORT" \
  --username="$PGUSER" --dbname="$PGDATABASE" --password
```

For an automated job, use an approved secret store or a restricted PostgreSQL password file rather than inventing a way to echo the password into a command.

In plain SQL output, `--no-owner` and `--no-privileges` omit ownership and grant commands to ease a portable restore. For a custom archive, ownership suppression belongs on `pg_restore`; `pg_dump --no-owner` is ignored for that archive format. The restore below intentionally omits original owners and grants, so a separate reviewed access configuration is necessary before application use. PostgreSQL dump options

`--schema=metro_support` is a deliberate boundary. It is not a backup of the whole Supabase project, its authentication system, file storage, or external services. Objects outside that schema may be dependencies in a real application. A source manifest should say what was included and what was not.

## 8.9 Inspect the Artifact Before Depending on It

For a custom-format dump:

**Listing 88. Shell**

```
pg_restore --list metro_support.dump
```

For a plain SQL file, inspect its beginning and end with a text viewer and search for expected table and data statements. Record file size and a cryptographic hash if the artifact will be transferred or retained:

**Listing 89. Shell**

```
shasum -a 256 metro_support.dump
```

A list or checksum proves properties of the artifact, not that PostgreSQL can successfully restore it.

Treat an untrusted dump as executable input. Current PostgreSQL documentation warns that restoring a dump can execute SQL selected by source superusers. Inspect the source and archive list, restore first into an isolated disposable target with a least-privileged restore role, and never test an untrusted artifact directly against production.

## 8.10 Restore Into a Separate Target

Never test by overwriting the only copy. Create an empty, disposable database or authorized, isolated restore target.

Use separate `RESTORE_PGHOST`, `RESTORE_PGPORT`, `RESTORE_PGUSER`, and `RESTORE_PGDATABASE` variables for the target. Inspect those values before running the restore. A new schema name in the source database is not automatically a separate restore target: the archive recreates the schema names it contains. The optional companion notebook creates a distinct database explicitly.

Custom format:

**Listing 90. Shell**

```
pg_restore \
  --host="$RESTORE_PGHOST" --port="$RESTORE_PGPORT" \
  --username="$RESTORE_PGUSER" --dbname="$RESTORE_PGDATABASE" \
  --password --single-transaction \
  --no-owner \
  --no-privileges \
  --exit-on-error \
  metro_support.dump
```

Plain SQL:

**Listing 91. Shell**

```
psql \
  --set=ON_ERROR_STOP=on \
  --host="$RESTORE_PGHOST" --port="$RESTORE_PGPORT" \
  --username="$RESTORE_PGUSER" --dbname="$RESTORE_PGDATABASE" \
  --password --single-transaction \
  --file=metro_support.sql
```

`--exit-on-error` and `ON_ERROR_STOP` make failures visible rather than allowing a long process to appear successful after skipped errors.

For these small practice archives, `--single-transaction` also avoids keeping a half-restored set of transactional objects after an error. It is not suitable for every restore mode or every possible script, and it does not clean up old objects already in the target. Begin with the specified empty target; do not add destructive `--clean` options to force a restore over something unfamiliar. PostgreSQL restore options

## 8.11 Verify Structure, Data, and Behavior

### 8.11.1 Structure

Listing 92. SQL

```sql
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'metro_support'
ORDER BY table_name;
```

Inspect expected constraints and indexes as well as tables.

### 8.11.2 Data

Listing 93. SQL

```sql
SELECT 'users' AS object, count(*) FROM metro_support.users
UNION ALL
SELECT 'tickets', count(*) FROM metro_support.tickets
UNION ALL
SELECT 'ticket_events', count(*) FROM metro_support.ticket_events;
```

Counts are necessary but not sufficient. Check a known relationship and a boundary row.

For the full, unchanged Metro Support source fixture, the expected counts are 8 users, 12 tickets, and 21 events. The optional recovery notebook deliberately uses a smaller fixture: 3 users, 3 tickets, and 5 events. Establish the baseline for the source you actually exported rather than borrowing counts from a different example. In the full fixture, a second check can answer a specific question:

Listing 94. SQL

```sql
SELECT t.ticket_id, t.status, t.assignee_id, u.display_name AS requester
FROM metro_support.tickets AS t
JOIN metro_support.users AS u ON u.user_id = t.requester_id
WHERE t.ticket_id = 1004;
```

The baseline result is ticket 1004, status `new`, a NULL assignee, and requester Requester 104. The NULL is expected, not a failed restoration. Compare with the source state you actually backed up if earlier exercises committed changes. Two different databases can have the same row counts, so this identity-and-value check asks a different question from the count query.

### 8.11.3 Behavior

Run one meaningful report and one expected-failure constraint test inside a transaction. If access configuration is part of the recovery scope, test an allowed and denied operation.

An error alone is insufficient. An invalid status should fail because of the named status constraint, not because the server disconnected or the table name was misspelled. PostgreSQL SQLSTATE `23514` identifies a check-constraint violation. Check that code and the exact constraint name, then confirm the test row was not retained. Use a transaction that cannot commit even if a missing constraint unexpectedly allows the insert. This is a small example of testing the failure mechanism rather than accepting any failure as success.

Also compare a ticket's subject and requester between source and target. An incorrect subject can leave every table count unchanged. Conversely, two equal query results can both be wrong if the source itself was wrong. Compare the result with the intended record in the original fixture as well. Recovery verification and source-data quality answer related but distinct questions.

### 8.11.4 Application Boundary

If the recovery objective includes application service, use a temporary safe configuration to test a real read and write path. Do not redirect production traffic before verification and approval.

## 8.12 Worked Example: A Recovery Runbook Entry

**Listing 95. Text**

```
Purpose: recover the Metro Support schema after an accidental destructive change.

Artifact: custom-format logical dump, created daily by an approved operator.

Prerequisites: pg_dump/pg_restore major version compatible with the server;
temporary source and restore credentials; empty restore target; enough storage.

Safety boundary: never restore over the source database. Never commit URLs.

Procedure: create dump; record exit code, size, and SHA-256; list contents;
restore with --exit-on-error into the separate target.

Verification: compare source and target table definitions; expected row counts;
ticket 1004 and its requester; one report; one rejected foreign-key change.
If security is in scope, restore reviewed roles/grants and test them separately.

Success decision: the required checks pass for this recovery scope. Application
traffic is not redirected until its owner approves the verified target.

Cleanup: revoke temporary credentials and remove the restore target according to
the retention policy.
```

## 8.13 Safe Migrations and Recovery Are Connected

A backup before a migration is useful only if:

- it includes the necessary scope;
- it can be restored within the decision window;
- the old application remains compatible with the restored state; and
- the team knows whether rollback, forward fix, or restore is safest.

For small changes, transactional DDL and a tested rollback may be faster. For a destructive data transformation, restore may be part of the response. Plan before execution.

## 8.14 Common Misconceptions

### 8.14.1 “The dump command returned, so recovery is ready”

Exit status, artifact inspection, separate restore, and verification are still required.

### 8.14.2 “A CSV is a full database backup”

CSV can preserve selected row values but usually omits schema, constraints, indexes, permissions, and transactionally consistent multi-table state.

### 8.14.3 “A replica protects against deletion”

Replication can reproduce the deletion. Recovery needs an artifact or history outside the active state and a tested procedure.

### 8.14.4 “RPO and RTO are the same”

RPO concerns acceptable data loss; RTO concerns acceptable recovery duration.

## 8.15 Study and Practice

A recovery check should establish more than successful command execution. Connect each check to a failure it could detect, and distinguish restored data from restored access, structure, and application behavior.

Write a restore verification checklist for Metro Support with:

1. two structure checks;
2. two data checks;
3. one behavior check;
4. one access check; and
5. one statement of what the checklist does not prove.

## 8.16 Retrieval and Transfer

1. How does high availability differ from backup?
2. What does an RPO of four hours mean?
3. Why restore into a separate target?
4. What do `--no-owner` and `--no-privileges` trade away?
5. Why is a row count not complete restore verification?
6. How does the absence of automatic backups change a free-tier recovery plan?

## 8.17 Further Reading

- PostgreSQL backup and restore
- PostgreSQL `pg_dump`
- PostgreSQL `pg_restore`
- Supabase database backups
- Supabase connection guidance

# Part III: Documents and MongoDB

NoSQL systems did not replace relational databases with one universal alternative. They made different abstractions and tradeoffs convenient for changing workloads. Chapters 9-12 trace that history, introduce key-value, wide-column, document, graph, and vector models, and then develop beginner MongoDB skills in context.

The progression is deliberate: valid JSON before BSON, BSON before MQL, MQL before document-model decisions, and modeling before operational claims about indexes, validation, replication, or recovery. Atlas provides an authentic managed interface, while local and static paths preserve the conceptual outcome when the cloud path is unavailable.

# 9 Documents Emerged From Changing Workloads

## 9.1 Operating Question

If you designed a data system today, which relational ideas would you keep, what might you change, and which workload would justify that change?

## 9.2 Learning Outcomes

After this chapter, you can:

- explain why NoSQL emerged without claiming that relational databases became obsolete;
- compare key-value, wide-column, document, graph, and vector abstractions;
- write and validate strict JSON objects and arrays;
- distinguish JSON text from MongoDB's BSON document representation;
- represent the same CSV relationships in more than one valid JSON shape; and
- evaluate flexibility in terms of reads, writes, duplication, growth, and integrity.

## 9.3 Relational Databases Solved a Real Historical Problem

The preceding chapters asked how to preserve meaning while many people query and change shared tables. Those questions do not disappear when records become documents or when a database moves to several machines. This chapter changes the shape of the discussion, not the standard of correctness. We will ask what an application needs to retrieve together, how relationships change, and which responsibilities a different storage model moves to the application.

In 1970, E. F. Codd proposed the relational model as a way to separate logical data relationships from physical storage details. Relations, keys, declarative queries, constraints, and transactions remain powerful because they let many workloads ask new questions while preserving shared meaning.

The later rise of NoSQL did not prove those ideas wrong. It reflected new pressures:

- globally distributed services;
- very high write or read throughput;
- data whose fields changed frequently;
- hierarchical objects moving through web applications;
- specialized traversals, sparse records, caches, and search; and
- engineering teams willing to trade generality for a workload-specific shape.

## 9.4 "NoSQL" Has More Than One Historical Meaning

Carlo Strozzi used the name NoSQL in 1998 for a relational DBMS that used Unix tools rather than SQL. That system was still relational. Around 2009, the label was reused for a meetup and a growing set of non-relational, often distributed datastores. The later slogan "not only SQL" emphasizes coexistence, but it is not a precise technical definition.

Important influences included:

- Google's 2006 Bigtable paper, which described a distributed structured store with a sparse, multidimensional data model; and
- Amazon's 2007 Dynamo paper, which described a highly available key-value system designed for an "always-on" service experience.

Products inspired by these systems made different choices. No single consistency, schema, transaction, or query rule applies to every NoSQL database.

Bigtable and Dynamo illustrate different pressures rather than a single replacement for relational systems. Bigtable organized a very large sparse map using row keys, column keys, and timestamps. Dynamo emphasized availability for key-addressed data and described replication and version-reconciliation choices. Reading their design problems helps explain why workload matters more than the label "NoSQL." Google's Bigtable paper, Amazon's Dynamo paper

Do not transfer a 2007 research-system limitation to every present-day product with a similar name. Amazon's Dynamo research system and the later DynamoDB service are not identical specifications. Modern systems also cross categories: a relational system can store JSON or support vector search, and a document system can support transactions. The categories describe useful abstractions, not mutually exclusive boxes containing every possible feature.

## 9.5 Choose an Abstraction for a Workload

### 9.5.1 Key-Value Store

Model: a mapping from a unique key to a value.

**Listing 96. Text**

```
"session:8fd2" -> {"user_id": 101, "expires_at": "..."}
```

Strengths include direct lookup, caching, sessions, counters, and simple partitioning. If the system cannot inspect fields inside the value efficiently, questions that do not know the key become difficult. Redis is a widely used example with richer value structures than a bare string map.

For a session lookup, the application already has the session key and wants its value. For "find every unexpired session belonging to this user," it may not know the relevant keys. That second question needs another access structure, a scan, or a redesigned key arrangement. This is the same distinction as finding a ticket by its primary key versus reporting tickets by neighborhood: a fast path for one question is not a fast path for every question.

### 9.5.2 Wide-Column Store

Model: rows located by a row key, with sparse or dynamic columns commonly grouped into column families.

This abstraction suits large, distributed workloads organized around known access keys and ranges, such as time-ordered events by device. Google Bigtable and systems influenced by it demonstrate this family. It is not the same as an analytical columnar file or warehouse merely because both use the word "column."

For example, a row key beginning with a device identifier can place that device's measurements together. That helps "read this device's recent measurements" but may not help "find all devices that exceeded a threshold today." A key that groups data conveniently can also concentrate writes. Chapter 13 returns to that distribution problem without requiring you to administer a sharded cluster.

### 9.5.3 Document Store

Model: field-named, nested records addressed and queried by their contents.

**Listing 97. JSON**

```
{
  "ticket_id": 1001,
  "status": "open",
  "requester": {"user_id": 101, "name": "Requester 101"},
  "tags": ["streetlight", "safety"]
}
```

Documents align naturally with objects and API payloads. Related facts read and changed together can live together. The design must still address duplication, unbounded arrays, shared ownership, validation, indexes, and access patterns. MongoDB is the document database used in this book's examples.

A field name makes structure visible, but not every meaning self-evident. `"priority": 3` does not explain whether 3 is urgent, low, or an external code. Likewise, a nested requester can be the requester's current profile or a snapshot from the day a ticket opened. A document needs a semantic contract just as a relational table does.

### 9.5.4 Graph Database

A graph is commonly written $G = (V, E)$, where $V$ is a set of vertices and $E$ is a set of edges. Vertices represent entities; edges represent relationships. A property graph can store attributes on both.

**Listing 98. Text**

```
(resident 101)-[REQUESTED]->(ticket 1001)-[ASSIGNED_TO]->(agent 201)
```

Basic graph ideas:

- **directed edge:** `A -> B` has a direction;
- **undirected edge:** connection has no direction for the model;
- **degree:** $deg(v)$, the number of edges incident to vertex $v$;
- **walk:** a sequence $v_0, e_1, v_1, \ldots, e_k, v_k$ of connected vertices and edges, possibly revisiting a vertex;
- **simple path:** a walk with no repeated vertex;
- **shortest path:** a path minimizing edge count or total edge weight; and
- **connected component:** a maximal set of mutually reachable vertices under the chosen direction rules.

For a directed graph, distinguish **in-degree** from **out-degree**: incoming connections and outgoing connections answer different questions. In a directed dependency graph, "depends on" and "is depended on by" are not interchangeable. Connectivity also has two common meanings: weak connectivity ignores arrow direction; strong connectivity requires directed paths both ways. We use simple examples without parallel edges or self-loops unless stated otherwise.

For example, `A -> B -> C` is a simple path; `A -> B -> C -> A` closes a cycle rather than being a simple path. Query languages may use "path" more broadly and impose their own repetition rules. Neo4j's default matching permits revisited vertices but not a repeated relationship within one match. Read the chosen query mode before assuming a traversal forbids every kind of revisit. MIT's graph definitions and Neo4j's path-matching rules distinguish mathematical and database terminology.

Graph databases are useful when the relationship path is the question: fraud rings, dependency impact, network topology, recommendations, authorization paths, or knowledge graphs. A graph is not automatically better for ordinary records that are mostly retrieved by identifier.

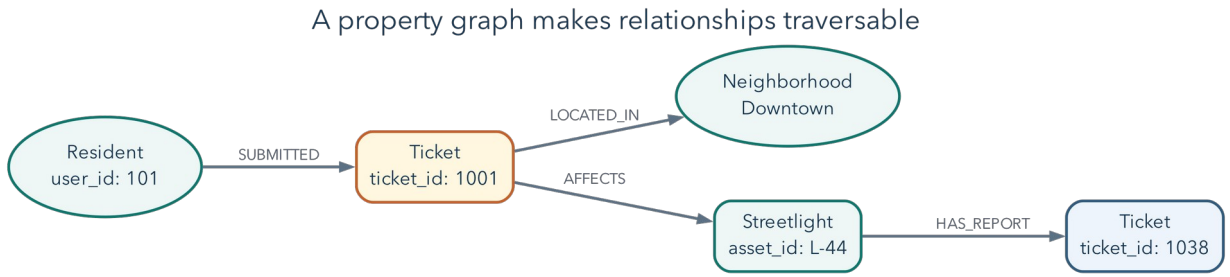



*Figure 9.1: A property graph represents records as vertices and named relationships as directed edges. A traversal can move from a resident to a ticket, neighborhood, affected asset, and related reports.*

In the pictured path, the question "Which other reports involve an asset touched by this resident's ticket?" is naturally a traversal. A relational database can answer it with joins and a recursive common table expression; a graph database makes the path the primary abstraction. The choice follows the dominant workload, not the presence of relationships by itself.

Consider a second example with assets and their dependencies:

**Listing 99. Text**

```
repair portal -> identity service -> database
repair portal -> ticket API      -> database
```

An edge means "the left service depends on the right service." From the portal, following arrows finds its dependencies. Starting at the database and following arrows backward finds potentially affected services. Counting one-hop neighbors alone misses the portal, which is two hops away. A breadth-first traversal explores one-hop neighbors, then two-hop neighbors, and so on; tracking visited vertices prevents repeatedly circling a dependency cycle. With equal edge costs, the first discovered path has the fewest hops. If edges represent unequal travel times or costs, fewest hops is no longer necessarily cheapest.

This is why graphs are useful for dependency impact and connected patterns. Whether a particular failure actually affects a service still depends on redundancy, fallback paths, and the meaning assigned to each edge. A diagram of connections is not automatically an outage prediction.

### 9.5.5 Vector Store or Vector Search

A vector represents an item as a point in a numeric space, often produced by a machine-learning embedding model. Similar items should be near one another under a chosen measure.

You can begin with a vector as an ordered list of numbers. In two dimensions, `[3, 4]` can mean a point three units along one axis and four along another. An embedding model may instead produce hundreds of

coordinates learned from text, images, or other data. Individual coordinates need not have simple human labels. The model turns an item into numbers; the database stores and searches those numbers. Adding a vector index does not itself train the model or establish that the retrieved items are relevant.

For nonzero vectors $a, b \in \mathbb{R}^n$, cosine similarity is:

$$s_{\cos}(a,b) = \frac{a \cdot b}{\|a\|_2 \|b\|_2}$$

The dot product and Euclidean norm are:

$$a \cdot b = \sum_{i=1}^{n} a_i b_i, \|a\|_2 = \sqrt{\sum_{i=1}^{n} a_i^2}.$$

It gives the cosine of the angle between nonzero vectors. Two vectors pointing in the same direction have similarity 1 even if one has larger magnitude. Euclidean distance measures straight-line distance and is sensitive to magnitude. Cosine is often useful when direction represents semantic pattern and vector length is not the desired signal.

Euclidean distance is written:

$$d_2(a,b) = \|a-b\|_2 = \sqrt{\sum_{i=1}^{n} (a_i - b_i)^2}.$$

Cosine similarity compares direction; Euclidean distance compares geometric separation. The metric must match how the embedding model and application assign meaning to the vector space.

### 9.5.6 Calculate a Small Example

Let the query be $q = (1,0)$, with candidates $a = (10,0)$ and $b = (1,1)$. The dot product multiplies matching coordinates and adds the products. Thus $q \cdot a = 10$ and $q \cdot b = 1$. The Euclidean norm is a vector's length from the origin: the lengths are $1$, $10$, and $\sqrt{2}$ respectively.

$$s_{\cos}(q,a) = \frac{10}{1 \cdot 10} = 1, s_{\cos}(q,b) = \frac{1}{\sqrt{2}} \approx 0.707.$$

But the straight-line distances are:

$$d_2(q,a) = 9, d_2(q,b) = 1.$$

Cosine ranks A as more similar because it points in exactly the same direction. Euclidean distance ranks B as closer because its coordinates are nearer. Neither calculation is wrong. They answer different questions. If magnitude represents an important amount, discarding it can be a mistake. If vector length varies for reasons irrelevant to the desired similarity, direction can be the better signal. These toy coordinates explain geometry; they are not measurements of language meaning.

Cosine is undefined for the zero vector because the denominator would be zero. For other real vectors its range is from -1 to 1; a negative score means an opposing direction, not a negative probability. Also, many

retrieval systems transform raw similarities into their own score ranges. Read the definition of the returned score before comparing it to a threshold.

Normalizing a nonzero vector means dividing every coordinate by its length. For unit-length vectors, the measures are closely related:

$$\left\|\hat{a}-\hat{b}\right\|_2^2 = 2 - 2\,s_{\cos}(a,b).$$

The hat marks a normalized vector. Expanding the squared distance gives the first vector's squared length, plus the second's, minus twice their dot product. The first two terms are each 1 after normalization. Therefore, on the same normalized vectors, maximizing cosine gives the same exact ranking as minimizing Euclidean distance, apart from ties and numerical precision. This is more precise than saying cosine is always better than distance.

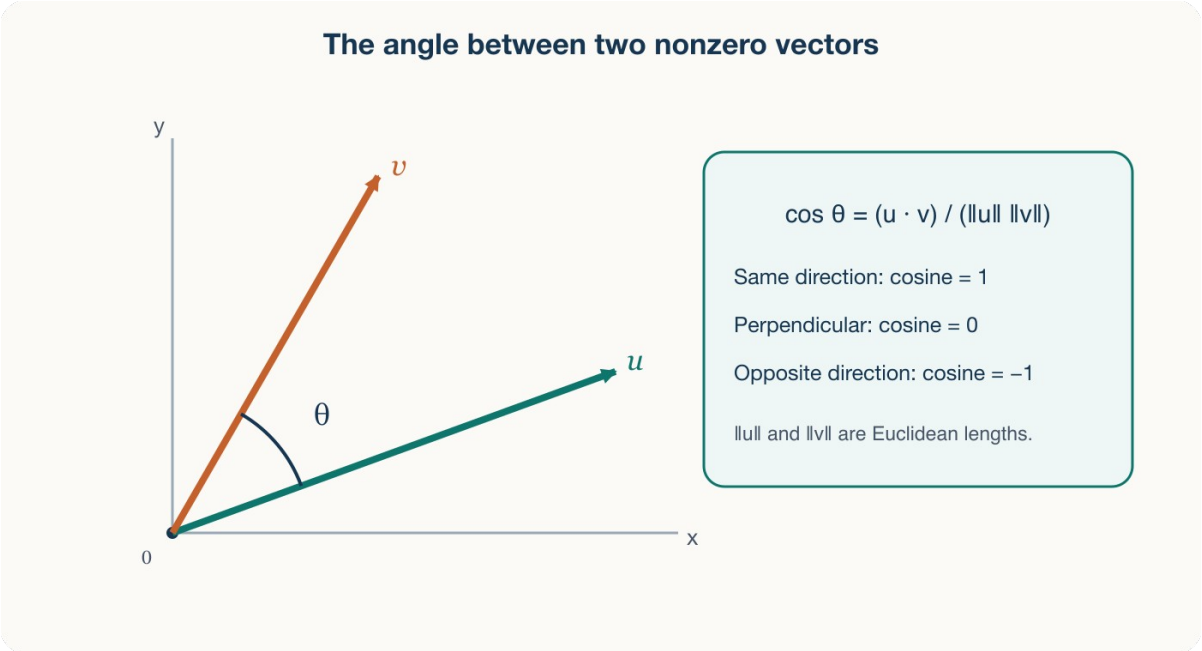



*Figure 9.2: Cosine similarity depends on the angle between nonzero vectors, while Euclidean distance also changes with magnitude.*

Exact nearest-neighbor search returns the closest records under the chosen measure. A brute-force baseline compares every vector; exact index methods can also prune candidates without changing the answer. For example, a k-d tree can skip regions that cannot contain a closer point. Approximate nearest-neighbor indexes may omit true neighbors in exchange for less search work. Whether the tradeoff helps depends on the data, workload, and index configuration. SciPy's k-d tree query documentation distinguishes exact and approximate queries and describes pruning. Vector search may be a feature inside relational or document systems rather than a completely separate database family. The operational questions remain: which model produced the vector, how it is versioned, which distance measure matches the meaning, how recall is evaluated, and how source records stay synchronized.

For example, if an exact search's top five contain five known record IDs and an approximate search returns three of those IDs, recall at five is $3/5 = 0.6$ for that query. It measures agreement with the exact retrieval target, not whether a human considers the results useful or fair. Evaluate relevance with appropriate examples too. Privacy rules still apply: a similar ticket is not necessarily a ticket the current user may read.

Vector search is a conceptual comparison in this chapter, not a requirement to buy a service, obtain an embedding API key, or configure a new index. Current MongoDB Vector Search documentation describes the product-specific implementation. The JSON representations below can be explored with only a text editor.

## 9.6 Specialized Models Trade Generality for Directness

| Workload question | Natural first model | Important tradeoff |
|---|---|---|
| fetch session by exact token | key-value | limited ad hoc field queries |
| write time-ordered device readings by device | wide-column | design tied to row key and access pattern |
| load a ticket with bounded nested details | document | duplication and document growth |
| find paths among accounts and devices | graph | relationship-oriented operations and new tooling |
| find semantically similar incident descriptions | vector search | approximate results and embedding lifecycle |
| enforce shared entities and flexible reporting | relational | joins and schema-change coordination |

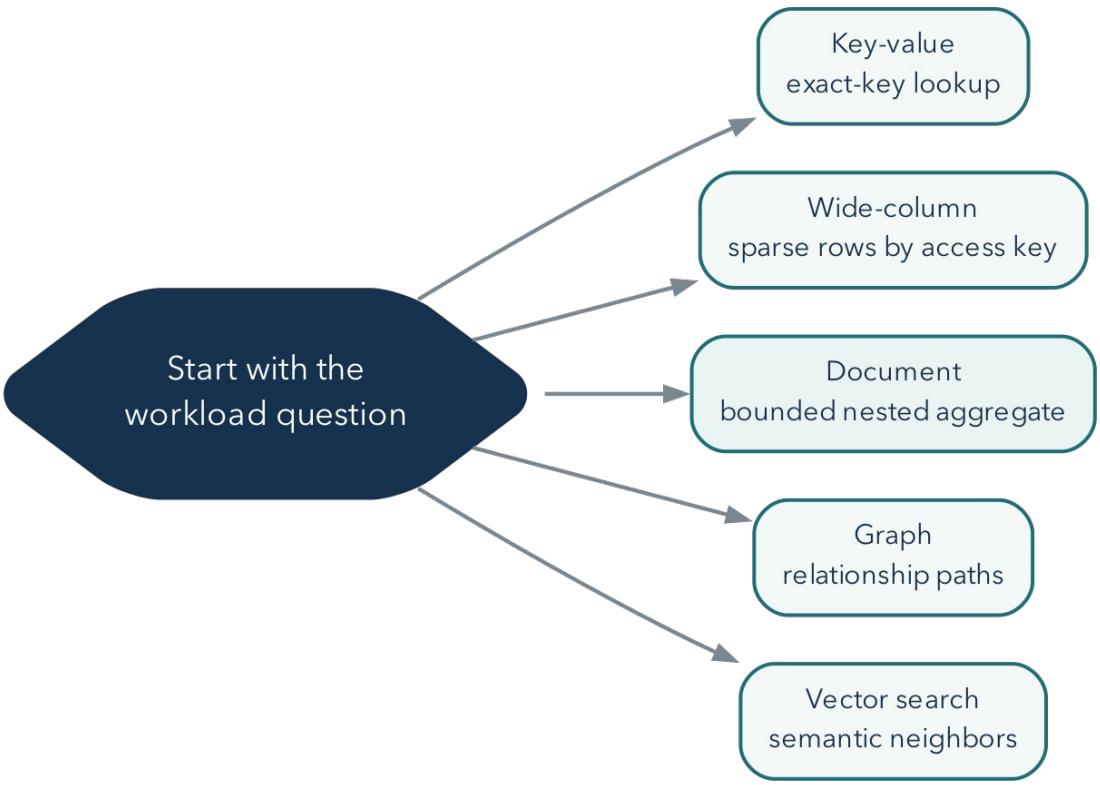



*Figure 9.3: NoSQL model choice begins with the dominant workload question, not with a product name.*

Real systems can combine models. Each added store creates more access control, monitoring, backup, synchronization, and expertise obligations. Polyglot design is a cost-benefit decision, not a trophy.

## 9.7 JSON Became a Common Interchange Format

JavaScript Object Notation grew from JavaScript object-literal syntax and was standardized as a minimal, textual, language-independent interchange format. The current Internet standard is RFC 8259; ECMA-404 defines the syntax in parallel. The registered media type is `application/json`.

Douglas Crockford's RFC 4627 documented JSON in 2006. RFC 7159 revised the specification in 2014, and RFC 8259 became the Internet-standard specification in 2017. The sequence reflects the need for different implementations to exchange the same data predictably. Historical documents explain the development; use the current specification for interoperability rules. RFC 4627, RFC 7159, RFC 8259

JSON became popular because it is:

- compact enough for network exchange;
- readable and writable across many programming languages;
- natural for nested API data;
- easy to parse with standard libraries; and
- less verbose than XML for many application payloads.

Popularity does not make every JSON design good. A valid document can still have ambiguous names, inconsistent types, duplicated facts, unsafe content, or no versioning strategy.

Consider an API that returns a ticket and a list of events. A CSV naturally represents a rectangular table; it needs conventions to represent a nested list within one field. JSON has objects and arrays built into its syntax, so the API can represent that structure directly. It is still text that must be parsed and validated. A JavaScript expression, a Python dictionary, and a JSON document can look similar while being different languages with different valid spellings.

## 9.8 JSON Has Six Value Kinds

A JSON value can be:

- object;
- array;
- string;
- number;
- `true` or `false`; or
- `null`.

Object names are strings. Objects are unordered collections of name/value pairs; arrays are ordered sequences.

Read `{}` as "one object with named properties" and `[]` as "a list of values." A colon connects a property name to its value. Commas separate neighbors at the same level. Indentation does not change the represented data, but it makes the containment visible to a reader.

**Listing 100. JSON**

```
{
  "ticket_id": 1001,
  "open": true,
  "closed_at": null,
  "priority": "high",
  "coordinates": [40.69, -73.99],
  "requester": {
    "user_id": 101,
    "display_name": "Requester 101"
  }
}
```

Strict JSON requires double-quoted names and strings. It does not permit comments, trailing commas, `undefined`, single-quoted strings, or native date values.

These are invalid:

**Listing 101. Text**

```
{'status': 'open'}                  # single quotes
{"status": "open",}               # trailing comma
{"opened_at": 2026-02-01T10:00Z} # unquoted timestamp
```

An application commonly encodes a date as an agreed string. MongoDB's BSON format adds types such as date and ObjectId that are not native JSON types. Atlas and drivers may display Extended JSON when representing those types as text.

### 9.8.1 Missing, Null, Empty, and Zero Are Different

These four objects can describe different situations:

**Listing 102. JSON**

```
[
  {"ticket_id": 1004},
  {"ticket_id": 1004, "assignee_id": null},
  {"ticket_id": 1004, "events": []},
  {"ticket_id": 1004, "estimated_cost": 0}
]
```

The first omits an assignee property. The second explicitly supplies null. The third supplies a list with no events, while the fourth supplies a numeric zero. An application may choose to treat omitted and null assignees alike, but that is its contract, not something the JSON syntax guarantees. An empty event list also does not tell us whether history truly does not exist or simply was not included in this response. A field such as `history_included` could make that distinction explicit if the application needs it.

### 9.8.2 Valid Syntax Does Not Guarantee Interoperability

Use distinct property names within an object. Some parsers accept duplicate names but retain different values, which makes the document unreliable for exchange. Use strings for identifiers whose spelling matters, such as postal codes with leading zeros. JSON has a number syntax, but receiving languages can impose different range and precision limits. Very large numeric IDs can lose precision in a JavaScript number; an agreed string representation avoids treating such IDs as arithmetic quantities. For exact

amounts, define a documented representation such as integer cents within a safe range or a decimal string interpreted by a decimal-capable application.

Use a JSON parser, not a language's general-purpose `eval`, to read JSON text. Parsing recognizes data syntax; evaluating arbitrary expressions executes code. For optional local checking, Python's standard library can parse a short example:

**Listing 103. Python**

```
import json

text = '{"ticket_id": 1004, "assignee_id": null, "events": []}'
ticket = json.loads(text)
print(ticket["ticket_id"])       # 1004
print(ticket["assignee_id"])     # None: Python's null-like value
print(json.dumps(ticket, indent=2))
```

`loads` turns JSON text into Python values; `dumps` turns supported Python values back into JSON text. Python `None` becomes JSON `null`. For this standard-compliant input, the round trip checks representation, not the truth of the ticket's facts. Python's default parser also accepts `NaN` and `Infinity`, which are outside standard JSON, and retains the last value when a member name repeats. Therefore, successful `json.loads` alone is not a strict interoperability check. `allow_nan=False` rejects nonfinite numbers when writing JSON; rejecting them when reading requires a `parse_constant` policy. These are separate settings. The Python JSON documentation explains these behaviors. The document-design examples can also be studied as plain text without running Python.

## 9.9 Flexibility Means Multiple Shapes Are Possible

Three CSV tables can become many valid JSON designs. The following excerpts contain all source facts used in the complete comparison below; no download or database connection is needed. The first line names columns and each remaining line describes one record.

### Users

**Listing 104. CSV**

```
user_id,display_name
101,Requester 101
201,Agent 201
```

### Tickets

**Listing 105. CSV**

```
ticket_id,subject,status,requester_id,assignee_id
1001,Streetlight dark near bus stop,open,101,201
```

### Events

**Listing 106. CSV**

```
event_id,ticket_id,event_type,event_at
5001,1001,created,2026-02-01T23:10:00Z
5002,1001,assigned,2026-02-02T14:05:00Z
```

`requester_id` and `assignee_id` identify users; `ticket_id` connects each event to its ticket. CSV text alone does not enforce those relationships or assign database types. In the JSON below, these identifiers become numbers and the timestamps remain strings. A field containing a comma would need CSV quoting; none of these particular values contains one.

The short sketches below isolate a design idea, so they omit selected fields or events. They are not complete conversions of ticket 1001. The complete paired example afterward retains the same selected facts in both shapes.

### 9.9.1 Referenced Collections

**Listing 107. JSON**

```
{
  "ticket_id": 1001,
  "requester_id": 101,
  "assignee_id": 201,
  "status": "open"
}
```

Events can remain separate documents containing `ticket_id`. Shared users have one source of truth, but reading a full ticket history requires additional queries or a join-like operation.

### 9.9.2 Embedded Ticket

**Listing 108. JSON**

```
{
  "ticket_id": 1001,
  "status": "open",
  "requester": {
    "user_id": 101,
    "display_name": "Requester 101"
  },
  "events": [
    {
      "event_id": 5001,
      "type": "created",
      "at": "2026-02-01T23:10:00Z"
    }
  ]
}
```

One read can return the ticket and the included portion of its history. This sketch includes event 5001 but omits event 5002 from the source CSV. It therefore does not contain the complete supplied history. The document duplicates requester display data and can grow without bound if events continue forever.

### 9.9.3 Snapshot Plus Reference

**Listing 109. JSON**

```
{
  "ticket_id": 1001,
  "requester_id": 101,
  "requester_name_at_open": "Requester 101",
  "recent_events": [
    {"event_id": 5002, "type": "assigned"}
  ]
}
```

This design intentionally keeps a historical snapshot and a reference. It needs a rule explaining which field is authoritative and how recent events are bounded.

The right question is not “Which JSON is correct?” It is “Which shape makes the important reads and atomic changes direct while keeping duplication, growth, and integrity manageable?”

## 9.10 Flexible Schema Is Still a Schema

Even if the database accepts different fields, applications assume names, types, and structures. That assumed contract is a schema. If one document stores `priority: "high"` and another stores `priority: 3`, every query and client must handle both or fail.

Flexibility helps when variation is intentional and understood. It is harmful when variation is accidental. MongoDB supports schema validation for established invariants, which Chapter 11 applies.

## 9.11 Worked Example: Compare Two Ticket Shapes

Before comparing access patterns, make the information in the two alternatives equivalent. Use ticket 1001, users 101 and 201, and events 5001 and 5002 from Metro Support. Retain the ticket ID, subject, status, requester, and assignment, the users' IDs and display names, and each event's ID, type, and timestamp. Other CSV fields are outside this example's scope.

### 9.11.1 Complete Referenced Representation

**Listing 110. JSON**

```
{
  "users": [
    {"user_id": 101, "display_name": "Requester 101"},
    {"user_id": 201, "display_name": "Agent 201"}
  ],
  "tickets": [{
    "ticket_id": 1001,
    "subject": "Streetlight dark near bus stop",
    "status": "open",
    "requester_id": 101,
    "assignee_id": 201
  }],
  "events": [
    {"event_id": 5001, "ticket_id": 1001,
     "event_type": "created", "event_at": "2026-02-01T23:10:00Z"},
    {"event_id": 5002, "ticket_id": 1001,
     "event_type": "assigned", "event_at": "2026-02-02T14:05:00Z"}
  ]
}
```

This is one interchange object containing three arrays, not three MongoDB collections created by writing a file. For a ticket page, match `requester_id` and `assignee_id` with `user_id`. Match each event's `ticket_id` with the ticket. The IDs carry the relationships while the records remain separate.

### 9.11.2 Complete Embedded Representation

**Listing 111. JSON**

```json
{
  "ticket_id": 1001,
  "subject": "Streetlight dark near bus stop",
  "status": "open",
  "requester": {"user_id": 101, "display_name": "Requester 101"},
  "assignee": {"user_id": 201, "display_name": "Agent 201"},
  "events": [
    {"event_id": 5001, "event_type": "created",
     "event_at": "2026-02-01T23:10:00Z"},
    {"event_id": 5002, "event_type": "assigned",
     "event_at": "2026-02-02T14:05:00Z"}
  ]
}
```

Containment now connects events to ticket 1001. Both event IDs and timestamps survive. A later export of an event on its own would need its ticket context again. The two embedded names are copies if authoritative user records exist elsewhere. A field meaning the current name needs an update policy. A deliberately historical name, such as `requester_name_at_open`, should retain its historical meaning rather than silently follow every future change.

### 9.11.3 The Same Facts, Different Operating Consequences

**Access patterns:**

1. show a ticket with its five most recent events;
2. append one event when status changes;
3. report event counts across all tickets by event type; and
4. preserve history for years.

Embedding every event makes the first two operations direct and single-document atomic. Unbounded growth and the cross-ticket report become concerns. Keeping all events separate supports long history and cross-ticket aggregation but requires a second query for the ticket page.

A plausible beginner design references a separate authoritative events collection and optionally embeds a small, explicitly bounded recent-event summary. The important part is documenting the boundary, not maximizing nesting.

The recent-event list is a copy, so it also needs a refresh rule. Retaining only five preview events must not accidentally delete the complete authoritative history. Moreover, updating separate ticket and event documents is not automatically one atomic operation. A later implementation must choose a supported coordination strategy or explicitly define acceptable temporary inconsistency. Identifying that responsibility is a design decision; implementing a synchronization service is a separate engineering task.

## 9.12 Common Misconceptions

### 9.12.1 “NoSQL means no schema”

The schema may be implicit, flexible, or validated selectively. Applications still depend on structure.

### 9.12.2 "JSON supports dates"

Strict JSON does not have a date type. A string convention or an extended storage format supplies date semantics.

### 9.12.3 "Embedding removes relationships"

It represents a relationship through containment. Ownership, duplication, and growth still need decisions.

### 9.12.4 "Vector similarity understands meaning"

Similarity reflects a model, data, metric, and index. It is an engineered estimate that must be evaluated for the task.

## 9.13 Study and Practice

Begin with a small subset whose facts you can inspect, then compare alternative representations of the same information. The goal is to explain the consequences of a shape, not to discover a single universally correct JSON layout.

Choose another ticket and its related rows from the complete Metro Support data printed in Appendix C. You can read its INSERT rows as data without executing SQL or obtaining CSV files. Following the small example above, sketch two JSON designs:

1. a referenced design; and
2. an embedded or hybrid design.

For each, state one read it improves, one update it complicates, one duplicated or growing field, and one rule that should be validated.

## 9.14 Retrieval and Transfer

1. Why is the 1998 use of "NoSQL" different from the later movement?
2. Which data-model family makes paths a first-class question?
3. How do cosine similarity and Euclidean distance differ?
4. Which JSON value kinds exist?
5. Why is a valid JSON document not necessarily a good data model?
6. Which workload measurements would support embedding ticket events?

## 9.15 Further Reading

- RFC 8259, JSON
- Debian's preserved package record for Carlo Strozzi's NoSQL RDBMS
- Google, *Bigtable* paper
- Amazon, *Dynamo* paper
- Martin Fowler and Pramod Sadalage, NoSQL key points
- MongoDB data modeling

- MongoDB `$similarityCosine` versioned reference
- MongoDB Vector Search

# 10 MongoDB Models the Way an Application Reads

## 10.1 Operating Question

How can we query documents safely and choose which related facts belong in one document?

## 10.2 Learning Outcomes

After this chapter, you can:

- distinguish an Atlas project, cluster, database, collection, and document;
- read common BSON values and Extended JSON representations;
- perform beginner `insert`, `find`, `update`, and `delete` operations;
- query nested fields and arrays with MQL operators;
- choose embedding or referencing from access patterns, growth, and ownership; and
- verify a write by inspecting matched and modified counts plus the final document.

## 10.3 Atlas and MongoDB Are Different Layers

The JSON designs from Chapter 9 were text representations. This chapter puts documents in a database and asks observable questions: which document matched, which field changed, and whether the stored shape supports the application. MQL, MongoDB Query Language, is the family of document-shaped query and update expressions used by MongoDB. The shell and the Python driver express those operations with slightly different host-language syntax.

- **MongoDB** is the document database system and query model.
- **Atlas** is MongoDB's managed cloud platform.
- An **Atlas project** organizes clusters, users, access, and services.
- A **cluster** is a deployed MongoDB service. Atlas Free uses a fixed three-node replica set with plan limitations.
- A **database** contains collections.
- A **collection** contains BSON documents.
- A **document** is a field/value structure with an `_id` identifier.

To connect, a learner normally creates a database user, configures temporary network access, and selects a connection method. Atlas account identity, database user identity, and application identity are separate.

The hierarchy is useful when troubleshooting. An Atlas project can exist with no deployed database service. A working deployment can contain a database with no application collections yet. Selecting a database name in a shell does not necessarily create persistent data; writing data or explicitly creating a collection does. Finally, an Atlas website login does not supply the database password used by a Python connection.

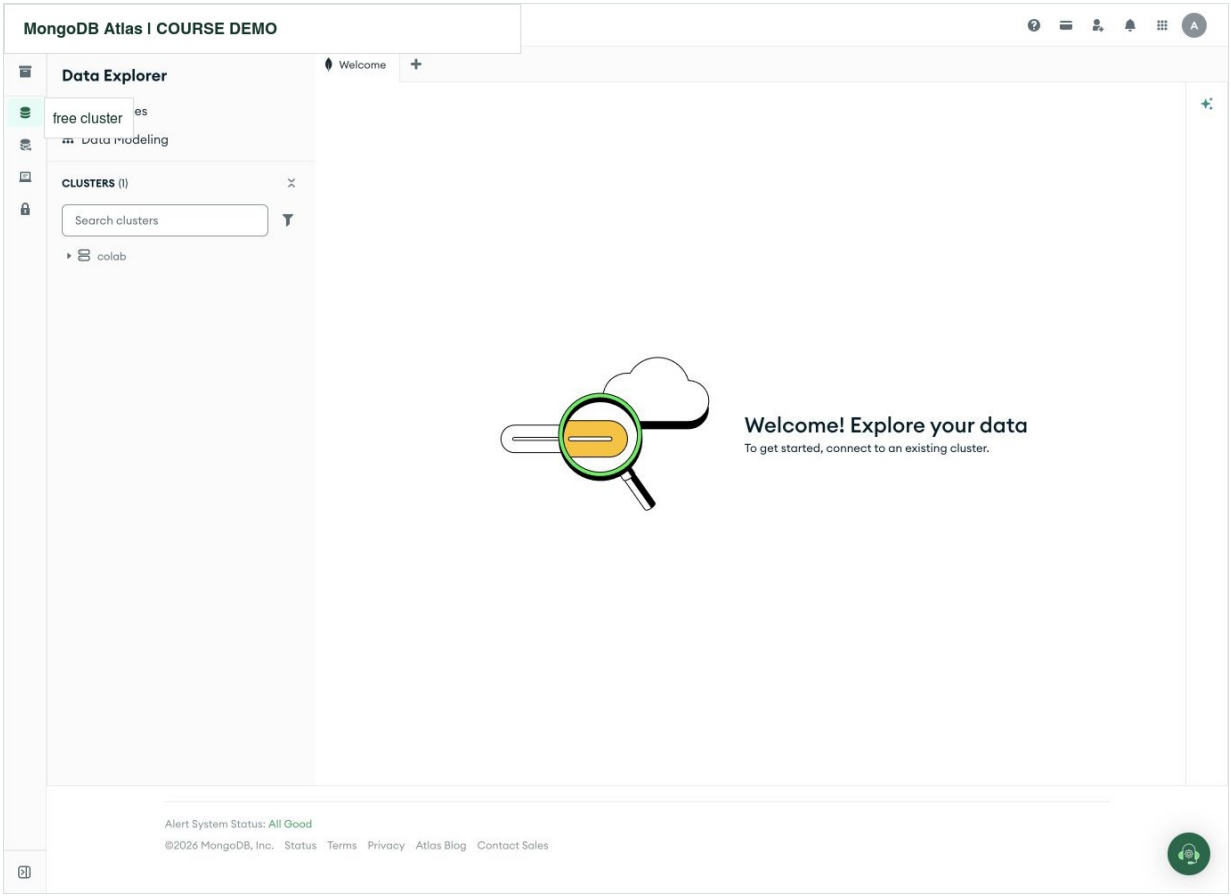



*Figure 10.1: Atlas Data Explorer can see a free deployment yet still ask the user to connect. Deployment state, database credentials, and network access are separate prerequisites. Organization, project, and deployment names are redacted. Interface captured August 25, 2026.*

In this captured interface, the deployment appears in the left tree while the workspace requests a connection. That screen alone does not identify the cause. Check deployment readiness and the selected connection path before interpreting an empty workspace as missing application data. For Python, also check the database user, network access list, DNS/TLS path, and requested database name.

### 10.3.1 Connect From the Machine That Runs the Code

A Colab notebook runs Python on a remote runtime, not on your laptop. Atlas sees that runtime's outgoing public IP address. Adding your laptop's IP does not necessarily allow Colab to connect. Chapter 13 prints the Python connection and runtime-IP examples; Appendix C explains local shell setup. The optional companion MQL notebook shows the runtime IP, uses a temporary narrow access-list entry, prompts for the connection URI with `getpass`, and sends a `ping` before attempting data operations.

The `mongodb+srv://` connection scheme uses DNS service records to discover servers; it is not an HTTP URL to open as a web page. Use the connection string generated for the actual deployment and URL-encode reserved characters in a password embedded in that string. Do not print the URI to diagnose it. A TLS handshake error has more than one possible cause; disabling certificate checks does not establish that the intended server is reachable and correctly trusted.

When cloud practice ends, remove its temporary access-list entry and close the connection. The optional notebook has a local document-query alternative for readers without a working cloud route. That alternative teaches the stated CRUD and array behavior, not Atlas networking or production durability.

## 10.4 BSON Extends JSON's Types

MongoDB stores BSON, a binary document format. BSON supports nested documents and arrays plus types such as ObjectId and Date.

**Listing 112. MongoDB shell / JavaScript**

```
{
  _id: ObjectId("65c2f56f1d4a9d7d3c101001"),
  ticket_id: 1001,
  opened_at: ISODate("2026-02-01T23:10:00Z")
}
```

This is `mongosh` representation, not strict JSON. Extended JSON supplies text forms for BSON types when data crosses JSON-only tools. Preserve types during import, export, and application conversion.

The string `"2026-02-01T23:10:00Z"` and a BSON Date are different stored types. They may look alike in a display while behaving differently in date operations or validation. BSON Date represents an instant with millisecond resolution; keep an original time-zone name separately if it has business meaning. `_id` is the document identifier. Our `ticket_id` is a domain identifier retained from the running example, so we explicitly protect it from duplication too.

Shell notation such as `ISODate(...)` creates a typed value. It is not legal inside a strict JSON file. Python instead uses a `datetime` value through its driver. Read the label on each listing before choosing where to run it.

## 10.5 Create Documents Intentionally

The chapter's runnable listings use **mongosh**, MongoDB's JavaScript shell, on a personal practice deployment. The companion notebook teaches the same concepts in Python, with its own fresh fixture. Do not mix shell commands into Python cells. This opening block selects a newly named disposable database, creates a unique domain-key index, and inserts one complete teaching document.

Listing 113. MongoDB shell / JavaScript

```
const practiceName = "cst4714_book_" + ObjectId().toHexString().slice(-8);
db = db.getSiblingDB(practiceName);
db.tickets.createIndex({ ticket_id: 1 }, { unique: true });

db.tickets.insertOne({
  ticket_id: 1001,
  category: "streetlight",
  priority: "high",
  status: "open",
  subject: "Streetlight dark near bus stop",
  requester: {
    user_id: 101,
    display_name: "Requester 101"
  },
  assignee_id: 201,
  opened_at: ISODate("2026-02-01T23:10:00Z"),
  tags: ["lighting", "safety"],
  events: [
    {
      event_id: 5001, type: "created", actor_role: "resident",
      at: ISODate("2026-02-01T23:10:00Z")
    },
    {
      event_id: 5002, type: "assigned", actor_role: "agent",
      at: ISODate("2026-02-02T14:05:00Z")
    }
  ]
})
```

`insertOne` returns an acknowledgment and inserted identifier. That is one write result, but verify by reading the identifier and important fields.

The unique `ticket_id` index means rerunning only the insertion rejects a second ticket 1001. Without it, two documents could have different generated `_id` values but the same ticket number. Rerunning the entire opening block creates another practice database; clean up the previous one rather than leaving unused databases behind. Continue the remaining listings in this same shell and practice database.

This teaching document adapts the CSV case rather than importing its columns automatically. The embedded event field `type` corresponds to CSV `event_type`, and `at` corresponds to `event_at`. `actor_role` is an added teaching field that makes the array counterexample visible. A field does not acquire a new name or type merely because it moves from a table to a document: an application or import step must make that transformation deliberately.

Use a unique practice database name. Do not practice destructive commands in the sample databases supplied by Atlas or in somebody else's collection.

## 10.6 A Worked Bridge From SQL to MongoDB

Recall Chapter 2's source, row selection, and output columns. Our question is: **which active tickets have high or urgent priority?** Active means `new`, `open`, or `in_progress`.

The following comparison copies these six tickets' ID, status, and priority into a temporary SQLite table and a MongoDB collection. Both queries start with the same facts. This is a separate instance from the twelve-ticket dataset and the single-document shell example above.

| Ticket | Status | Priority | Belongs in the answer? |
|---|---|---|---|
| 1001 | open | high | yes: both conditions hold |
| 1002 | in_progress | medium | no: priority is medium |
| 1003 | resolved | urgent | no: the request is resolved |
| 1004 | new | low | no: priority is low |
| 1005 | resolved | high | no: the request is resolved |
| 1006 | in_progress | medium | no: priority is medium |

Run the following Python setup once, then the Python comparison blocks in the same session. Install PyMongo and start the loopback-only MongoDB instance as described in Appendix C. SQLite comes with Python. This local route needs no Atlas account. The generated database name isolates these six records from the chapter's mongosh example.

**Listing 114. Python**

```python
import sqlite3
from uuid import uuid4
from pymongo import MongoClient

rows = [
    (1001, "open", "high"),
    (1002, "in_progress", "medium"),
    (1003, "resolved", "urgent"),
    (1004, "new", "low"),
    (1005, "resolved", "high"),
    (1006, "in_progress", "medium"),
]
sql_connection = sqlite3.connect(":memory:")
sql_connection.execute(
    "CREATE TABLE tickets (ticket_id INTEGER PRIMARY KEY, "
    "status TEXT NOT NULL, priority TEXT NOT NULL)"
)
sql_connection.executemany("INSERT INTO tickets VALUES (?, ?, ?)", rows)
sql_connection.commit()

client = MongoClient("mongodb://127.0.0.1:27018/",
                     serverSelectionTimeoutMS=5000)
client.admin.command("ping")
bridge_name = "book_bridge_" + uuid4().hex[:12]
tickets = client[bridge_name]["tickets"]
tickets.create_index("ticket_id", unique=True)
tickets.insert_many([
    {"ticket_id": number, "status": status, "priority": priority}
    for number, status, priority in rows
])
```

Here is the SQL query. Read it first; the next Python block executes it through the `sql_connection` created above:

**Listing 115. SQL**

```sql
SELECT ticket_id, status
FROM tickets
WHERE status IN ('new', 'open', 'in_progress')
  AND priority IN ('high', 'urgent')
ORDER BY ticket_id;
```

`IN` supplies alternatives within a condition. `AND` requires both conditions to hold. Ticket 1003 is urgent but resolved, so only one condition is true and the query excludes it.

**Listing 116. Python**

```
sql_result = sql_connection.execute("""
    SELECT ticket_id, status FROM tickets
    WHERE status IN ('new', 'open', 'in_progress')
      AND priority IN ('high', 'urgent')
    ORDER BY ticket_id
""").fetchall()
print(sql_result)  # [(1001, 'open')]
```

The corresponding **Python/PyMongo** query uses the same six facts:

**Listing 117. Python**

```
filter_doc = {
    "status": {"$in": ["new", "open", "in_progress"]},
    "priority": {"$in": ["high", "urgent"]},
}
projection = {"_id": 0, "ticket_id": 1, "status": 1}

for ticket in tickets.find(filter_doc, projection).sort("ticket_id", 1):
    print(ticket)
```

The expected document is `{'ticket_id': 1001, 'status': 'open'}`. Only this comparison's six records exist in the newly named collection.

Read the Python expression from its parts. `tickets` is the collection handle. `find` takes a filter and a projection. Multiple field conditions at the filter's top level must all hold. Each `$in` accepts one of its listed values. In the projection, `1` includes a field. The special `_id: 0` excludes the document ID that MongoDB would otherwise include. Sorting with `1` requests ascending order. PyMongo query filters

The driver returns a **cursor**, which retrieves results as Python iterates it. SQLite returns a tuple; MongoDB returns a dictionary. Those different Python representations describe the same selected ticket. Check the equivalence, then remove only this example's generated database and close both clients:

**Listing 118. Python**

```
mongo_result = list(tickets.find(filter_doc, projection).sort("ticket_id", 1))
assert [(doc["ticket_id"], doc["status"]) for doc in mongo_result] == sql_result
client.drop_database(bridge_name)
client.close()
sql_connection.close()
```

Continue the chapter's JavaScript examples in the earlier mongosh session, whose separate `practiceName` database still contains its single teaching ticket. Closing a Python client does not stop the MongoDB server; Appendix C explains how to stop the local server you started when all practice is finished.

### 10.6.1 A New Work Queue Changes the Predicate

Now the lighting and sanitation crews need their active requests. They also need medium-priority work, so the earlier priority restriction no longer belongs in the query. The required conditions become an active status **and** a category of `streetlight` or `sanitation`. Priority becomes information displayed in the result rather than a reason to exclude a request.

Adding the active-status condition to a category query, while displaying priority and retaining newest-first ordering, is a small code change with a meaningful consequence: a resolved sanitation request must

disappear from the work queue without disappearing from the database. Keeping a record, selecting a record, and displaying one of its fields are three different actions.

This comparison covers the fields and operators shown. Missing fields and nested arrays need additional rules, developed in the following examples. Matching one query does not make the languages interchangeable.

## 10.7 Read With Filters and Projections

**Listing 119. MongoDB shell / JavaScript**

```
db.tickets.find(
  {
    status: { $in: ["new", "open", "in_progress"] },
    category: { $in: ["streetlight", "sanitation"] }
  },
  { _id: 0, ticket_id: 1, category: 1, priority: 1, status: 1, subject: 1 }
).sort({ opened_at: -1 })
```

The first document is the filter. The second is the projection. A projection that includes selected fields normally uses `1`; `_id` is included by default unless explicitly excluded.

Read the query in words: keep tickets whose status belongs to the active set **and** whose category is streetlight or sanitation; return only the listed fields; order the results by opening time, newest first. The chapter's current one-ticket fixture returns ticket 1001. Priority is displayed, not filtered: an active medium-priority sanitation ticket would also qualify, while a resolved sanitation ticket or an active plumbing ticket would not. `-1` requests descending order and `1` ascending order. Projection and sorting do not change stored documents. You can sort using `opened_at` even though this find projection does not display that field. With an inclusion projection, do not mix arbitrary field exclusions; `_id: 0` is the usual permitted exception.

Common comparison and logical operators include:

- `$eq`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`;
- `$in`, `$nin`;
- `$and`, `$or`, `$not`; and
- `$exists`.

**Listing 120. MongoDB shell / JavaScript**

```
db.tickets.find({
  priority: { $in: ["high", "urgent"] },
  closed_at: { $exists: false }
})
```

Missing and `null` are distinct modeling states, though some query forms can match both. State which one the workload means.

In the inserted example `closed_at` is absent. The filter `{ closed_at: { $exists: false } }` finds absent fields. `{ closed_at: null }` matches both absent fields and explicit null values, while `{ closed_at: { $type: 10 } }` selects an explicit BSON null. That difference matters when "not included in this document" is distinct from "known not to have closed." MQL is not simply SQL with different punctuation; compare the actual operator semantics.

For basic comparisons, `$gt` means greater than, `$gte` greater than or equal, `$lt` less than, and `$lte` less than or equal. For example, `{ ticket_id: { $gte: 1000 } }` asks for IDs at least 1000. `$in` asks whether a value is among the supplied alternatives. Several top-level field conditions must all hold unless an explicit logical operator combines them differently.

## 10.8 Query Nested Fields and Arrays

Dot notation reaches nested fields:

**Listing 121. MongoDB shell / JavaScript**

```
db.tickets.find({ "requester.user_id": 101 })
```

A scalar equality condition on an array field matches when the array contains that value:

**Listing 122. MongoDB shell / JavaScript**

```
db.tickets.find({ tags: "safety" })
```

Suppose the question is “did an agent create this ticket?” The example contains a resident-created event and an agent-assigned event. This tempting query matches the ticket even though no single event says an agent created it:

**Listing 123. MongoDB shell / JavaScript**

```
db.tickets.find({
  "events.type": "created",
  "events.actor_role": "agent"
})
```

For multiple conditions that must match the same embedded array element, use `$elemMatch`:

**Listing 124. MongoDB shell / JavaScript**

```
db.tickets.find({
  events: {
    $elemMatch: {
      type: "created",
      actor_role: "agent"
    }
  }
})
```

Without `$elemMatch`, separate predicates can be satisfied by different elements, which may answer a different question.

Here `$elemMatch` is part of the **filter**. It decides which ticket documents qualify; it does not trim the returned `events` array to the matching element. For example, filtering for an agent-assigned event returns ticket 1001 with both of its stored events unless a separate projection changes the output. MongoDB also has an `$elemMatch` projection operator, whose different role should not be confused with the query predicate used here.

Before the later update, the first query returns ticket 1001 and the second returns no document. The empty result is correct. This counterexample is more informative than two queries that happen to agree on a fixture with only one event. The companion notebook supplies a similar contrast on its own small data.

## 10.9 Update With Operators, Not Accidental Replacement

**Listing 125. MongoDB shell / JavaScript**

```
db.tickets.updateOne(
  { ticket_id: 1001, status: "open" },
  {
    $set: { status: "in_progress", assignee_id: 202 },
    $push: {
      events: {
        event_id: 5099,
        type: "status_changed",
        at: new Date(),
        actor_id: 202,
        actor_role: "agent"
      }
    }
  }
)
```

The filter is part of write safety. Including the expected current status can prevent changing a document that has already moved to another state.

Inspect `matchedCount` and `modifiedCount` in driver results, then read the final document. A matched document can remain unmodified when the new value equals the old value.

For the first run here, expect one match and one modification. Rerunning this same update finds no match because ticket 1001 is no longer `open`. That is the expected-state condition doing useful work, not a connection failure. It also prevents this particular repeated operation from appending event 5099 again. This is a bounded example, not a complete retry design for arbitrary failures.

In mongosh the result fields use names such as `matchedCount`; PyMongo uses `matched_count` and `modified_count`. The meanings are related, but the property spelling belongs to the language's driver. Always read the final document when checking the intended state.

`replaceOne` replaces the document's other contents while preserving the immutable identifier. Omitted fields can disappear. `updateOne` with update operators changes selected fields; a plain replacement object is not silently accepted as an operator update. Choose the operation intentionally rather than assuming replacement means "merge these fields."

## 10.10 Delete With the Same Predicate Discipline

Create a clearly marked disposable record first. Without this insertion, the later deletion correctly affects zero documents.

**Listing 126. MongoDB shell / JavaScript**

```
db.tickets.insertOne({
  ticket_id: 1099, status: "new", subject: "Deletion practice",
  test_record: true
})
```

Preview the exact predicate:

**Listing 127. MongoDB shell / JavaScript**

```
db.tickets.find({ ticket_id: 1099, test_record: true })
```

Then delete the exact disposable record:

**Listing 128. MongoDB shell / JavaScript**

```
db.tickets.deleteOne({ ticket_id: 1099, test_record: true })
```

Verify `deletedCount` is one and the final `find` is empty. Repeating the delete returns zero because the record is already absent. An empty filter `{}` matches every document: `deleteOne({})` can remove an arbitrary matching document, while `deleteMany({})` removes all matches. Neither is a substitute for identifying the record you intended to delete.

## 10.11 Single-Document Atomicity Shapes Modeling

MongoDB write operations are atomic at the single-document level. Embedding facts that must change together can make an invariant easier to maintain. Multi-document transactions exist, but they add coordination and should not substitute for a workload-aware model.

Atomicity is not the only criterion. A document also needs bounded growth, reasonable duplication, useful access paths, and a clear owner.

The update above changed current state and appended its embedded event in one document operation. Another reader does not see only half that write. If events live in a separate collection, the same application operation crosses a document boundary and needs a considered coordination strategy. Conversely, putting every event from every year in one ticket is not justified merely by the convenience of atomicity. The model must account for growth and competing updates too.

## 10.12 Embed When the Data Belongs and Travels Together

Embedding is often appropriate when related data:

- is read with the parent most of the time;
- is updated atomically with the parent;
- has a clear ownership relationship;
- is not independently shared across many parents; and
- remains bounded within the document.

An address snapshot on an order or a small set of ticket classification fields can fit this pattern.

Distinguish **ownership** from mere association. A ticket's small set of classification answers may be owned by that ticket. A resident's current account profile belongs to the resident and is shared by many tickets. Copying that profile everywhere makes each future correction a multi-document update problem. An explicitly named historical snapshot is different: it intentionally records what was known at one time and need not change with the current profile.

## 10.13 Reference When Identity or Growth Is Independent

Referencing is often appropriate when related data:

- has its own lifecycle and queries;
- is shared by many parents;

- grows without a practical bound;
- changes frequently and should have one source of truth; or
- participates in many-to-many relationships.

Long-lived ticket events may be a separate collection because history grows and cross-ticket event reports are important. A ticket can store `requester_id` and a historical requester-name snapshot only if the authoritative versus snapshot semantics are explicit.

A reference is not an automatically enforced foreign key. Storing `requester_id: 101` does not by itself require that user 101 exists in another collection. The application must coordinate reference creation, deletion, and validation where the relationship matters. This is a direct comparison with Chapter 3, not an argument that either model is always superior.

## 10.14 Worked Example: Decide the Event Boundary

Access patterns:

1. retrieve a ticket summary by ID;
2. display the latest two events, newest first;
3. append events frequently;
4. retain years of history; and
5. aggregate event types across all tickets.

Options:

- **Embed all events:** direct ticket read and single-document append, but unbounded growth and cross-ticket analysis become concerns.
- **Reference all events:** bounded ticket and independent event-wide queries, but the ticket page needs another query or aggregation.
- **Hybrid:** authoritative events remain separate; ticket embeds a bounded recent summary. Reads improve, but synchronization logic and duplicate semantics must be documented.

A beginner project should choose the simplest option that supports the stated workload. Hybrid duplication is not automatically advanced or better.

### 10.14.1 A Complete Referenced Page

The following `mongosh` examples continue in the disposable database selected earlier. They create `page_tickets`, `people`, and `history` as a second model of ticket 1001's initial state. They do not migrate or synchronize the earlier `tickets` collection. Keeping the two examples separate lets us compare their behavior without silently changing the first experiment.

Create one authoritative person and one ticket that refers to that person:

**Listing 129. MongoDB shell / JavaScript**

```
db.people.createIndex({ user_id: 1 }, { unique: true });
db.page_tickets.createIndex({ ticket_id: 1 }, { unique: true });
db.people.insertOne({
  user_id: 101, display_name: "Requester 101",
  email: "requester101@example.org"
});
db.page_tickets.insertOne({
  ticket_id: 1001, requester_id: 101, status: "open",
  subject: "Streetlight dark near bus stop"
});
```

The unique indexes protect each collection's domain identifier. They do not enforce the reference between collections. The email is stored on the person, so a ticket does not contain another supposed current copy to maintain.

Each history document carries its own ticket reference. These two timestamps and event IDs match the initial CSV case:

**Listing 130. MongoDB shell / JavaScript**

```
db.history.createIndex({ event_id: 1 }, { unique: true });
db.history.insertMany([
  {
    event_id: 5001, ticket_id: 1001, event_type: "created",
    event_at: ISODate("2026-02-01T23:10:00Z")
  },
  {
    event_id: 5002, ticket_id: 1001, event_type: "assigned",
    event_at: ISODate("2026-02-02T14:05:00Z")
  }
]);
```

This representation retains the CSV field names `event_type` and `event_at`, unlike the shorter names in the embedded example. The parent ticket needs no array containing every event ID. Moving complete events out while retaining an ever-growing parent ID array would still leave a growing parent document. Run each insertion block once in this fresh database. Repeating an insertion rejects a duplicate domain key; it does not silently replace the earlier data.

Read the page in three operations. First, retrieve the ticket:

**Listing 131. MongoDB shell / JavaScript**

```
db.page_tickets.findOne(
  { ticket_id: 1001 },
  { _id: 0, ticket_id: 1, requester_id: 1, status: 1, subject: 1 }
)
```

It returns ticket 1001 with `requester_id: 101` and `status: "open"`. The application takes that requester ID and looks up the person's current details:

**Listing 132. MongoDB shell / JavaScript**

```
db.people.findOne(
  { user_id: 101 },
  { _id: 0, display_name: 1, email: 1 }
)
```

Finally, retrieve only this ticket's latest two history records:

**Listing 133. MongoDB shell / JavaScript**

```
db.history.find(
  { ticket_id: 1001 },
  { _id: 0, event_id: 1, event_type: 1, event_at: 1 }
).sort({ event_at: -1, event_id: -1 }).limit(2)
```

The returned IDs are 5002, then 5001. `event_at` sets the chronological order; the unique `event_id` breaks equal-time ties deterministically. Here the IDs happen to increase with time, but an identifier is not a substitute for the event timestamp. `limit(2)` restricts this result, not the stored history.

The constants make the read path visible. A real page handler would use the ID from its first result rather than hard-code 101. It would also handle a missing ticket or person explicitly. Successful lookup in this fixture does not prove that every possible reference in an application exists.

An index can support the history filter and ordering:

**Listing 134. MongoDB shell / JavaScript**

```
db.history.createIndex({ ticket_id: 1, event_at: -1, event_id: -1 })
```

The first field groups index entries for a ticket. Within that group, the next fields provide the requested newest-first order. This is a candidate access path, not a speed measurement. Chapter 11 explains how to inspect the plan and work counts. A two-event collection is too small to demonstrate production performance, and the index has its own storage and update cost.

### 10.14.2 A Contact Correction and a New Event

Change the person's email without rewriting ticket documents:

**Listing 135. MongoDB shell / JavaScript**

```
db.people.updateOne(
  { user_id: 101 },
  { $set: { email: "maya.updated@example.org" } }
)
```

Expect one matched and one modified person. Rerun the person lookup to see the new email, then rerun the ticket lookup: its reference remains 101 and it still has no copied email field. Repeating the same correction reports one match and zero modifications. A separate `requester_name_at_open` snapshot would have a different purpose and would not automatically change with the current profile.

Now append a new synthetic follow-up event, created for this experiment rather than copied from the original CSV:

**Listing 136. MongoDB shell / JavaScript**

```
db.history.insertOne({
  event_id: 5998, ticket_id: 1001, event_type: "note_added",
  event_at: ISODate("2026-02-03T12:00:00Z")
})
```

Repeating the latest-two query now returns 5998 and 5002. Event 5001 remains in the collection. Check the distinction between retained history and displayed history directly:

**Listing 137. MongoDB shell / JavaScript**

```
db.history.countDocuments({ ticket_id: 1001 })
```

The result is 3. No growing array or event-ID list was added to `page_tickets`. If the application must coordinate an event with a ticket status change, however, those separate documents introduce a different problem from this independent note append. The three page reads also do not promise one atomic snapshot across all collections. State the required consistency before choosing a transaction, a retry/reconciliation design, or an acceptable delay between related changes.

### 10.14.3 Growth and the Cost of a Convenient Preview

MongoDB limits a BSON document to 16 MiB, or 16,777,216 bytes. There is no universal safe event count: event sizes, field names, and array overhead vary. An embedded history can become inconvenient well before it reaches the hard size limit, especially when a page needs only a small part of it. Fifty thousand events is a useful design stress question, not a claim that every such document exceeds the limit.

A hybrid model might keep all events in `history` and copy only two recent summaries into the ticket. The copied preview needs a maintenance rule. An out-of-order arrival should be compared by event time; an edited or deleted event may require a refreshed preview. The application must decide whether a temporarily stale preview is acceptable and how it recovers from an interrupted update. For this beginner workload, the referenced read is easier to explain and verify. A hybrid is worth considering only when its extra maintenance solves a measured need.

These choices also occur in relational systems. PostgreSQL supports arrays and JSON values, and SQL does not force every application into third normal form. Normalization remains useful for reasoning about dependencies and update anomalies; workload analysis remains useful for choosing reads and storage. The important comparison is the consequence of a specific design, not a claim that one product has flexible data while the other cannot.

## 10.15 More Than One Model Can Be Valid

Data modeling is a testable hypothesis. Document:

- most important reads;
- atomic writes;
- expected array or document growth;
- duplicated facts and their owner;
- required indexes;
- integrity rules; and
- one operation the design makes harder.

Then create representative documents and query them. A diagram without a workload is not enough.

## 10.16 Common Misconceptions

### 10.16.1 “MongoDB stores JSON exactly”

MongoDB stores BSON. Tools render BSON values using `mongosh` or Extended JSON forms.

### 10.16.2 "Embedding is denormalization, so it is always faster"

It can reduce reads but increase size, duplication, update cost, and growth risk. Measure the actual workload.

### 10.16.3 "One collection should contain every kind of document"

Flexibility permits variation; it does not remove the need for cohesive ownership, indexes, validation, and predictable queries.

### 10.16.4 "`modifiedCount: 0` means the filter failed"

The document may have matched but already contained the requested value. Inspect both matched and modified counts.

## 10.17 Study, Cleanup, and Practice

The scenario below connects query behavior to modeling choices. Use the access pattern, growth of related data, and atomicity boundary to justify a design rather than copying the shape of a relational table automatically.

If you ran the shell examples, remove only the database created by their opening block and then leave the shell. This does not delete your Atlas deployment.

**Listing 138. MongoDB shell / JavaScript**

```
db.getSiblingDB(practiceName).dropDatabase()
```

For a tutoring scheduler, compare embedded and referenced appointment notes. State the read pattern, write pattern, ownership, growth, duplication, and one query for each design.

## 10.18 Retrieval and Transfer

1. How do an Atlas project and a MongoDB database differ?
2. Why is an ObjectId not a native JSON type?
3. When does `$elemMatch` matter?
4. Why should an update filter include the expected current state?
5. Which five conditions support embedding?
6. Which workload or growth pattern would make a referenced events collection preferable?

## 10.19 Further Reading

**Query and write behavior:** MongoDB CRUD operations, query documents, array-element matching, and single-document atomicity.

**Document design and growth:** MongoDB data modeling, embedding and references, sort support from indexes, and BSON document limits.

**Comparison and practice:** PostgreSQL arrays, PostgreSQL JSON types and document design, and MongoDB University's free data-modeling course.

# 11 MongoDB Operations Connect Pipelines, Rules, and Indexes

## 11.1 Operating Question

How can a MongoDB workload transform documents, reject invalid states, and find the intended records without examining everything?

## 11.2 Learning Outcomes

After this chapter, you can:

- trace document grain through an aggregation pipeline;
- use `$match`, `$project`, `$unwind`, `$group`, `$sort`, and `$limit`;
- add focused schema validation with `$jsonSchema`;
- read basic `explain("executionStats")` measurements;
- connect a compound index to filter and sort order; and
- make a keep/remove/test-further recommendation with operational tradeoffs.

## 11.3 A Pipeline Is an Ordered Document Transformation

An active-workload report asks a different question from a ticket lookup. It needs to select active tickets, combine them into categories, count each group, and order the summaries. A pipeline makes those transformations visible. The important idea is not the number of operators: at every step, know what one output document represents.

The runnable examples below include their own four-ticket setup and use `mongosh` JavaScript notation. Appendix C explains how to start the shell. A stage shown by itself is a fragment to place inside an aggregation array, not a complete command. The optional aggregation and validation notebook offers a Python version with its own data. In Python, operator names and field names are quoted dictionary keys, and a collection variable replaces `db.tickets`; do not paste shell commands into a Python cell.

### 11.3.1 A Fresh Four-Ticket Case

For the shell examples, start a new disposable database with the complete fixture below. This is a simplified teaching case, not the full twelve-ticket relational dataset. Each ticket includes the categories and priorities used by the pipelines below; event objects include identifiers and dates for the later validation example. It is independent of Chapter 10's practice database.

Listing 139. MongoDB shell / JavaScript

```
const pipelinePracticeName = "cst4714_pipeline_" + ObjectId().toHexString().slice(-8);
db = db.getSiblingDB(pipelinePracticeName);
db.tickets.createIndex({ ticket_id: 1 }, { unique: true });
db.tickets.insertMany([
  {
    ticket_id: 1001, category: "streetlight", priority: "urgent", status: "open",
    subject: "Dark streetlight", assignee_id: 201,
    opened_at: ISODate("2026-02-01T00:00:00Z"),
    events: [
      { event_id: 5001, type: "created", at: ISODate("2026-02-01T00:00:00Z") },
      { event_id: 5002, type: "assigned", at: ISODate("2026-02-01T01:00:00Z") }
    ]
  },
  {
    ticket_id: 1002, category: "sanitation", priority: "high", status: "in_progress",
    subject: "Missed pickup", assignee_id: 202,
    opened_at: ISODate("2026-02-02T00:00:00Z"),
    events: [{ event_id: 5003, type: "created", at: ISODate("2026-02-02T00:00:00Z") }]
  },
  {
    ticket_id: 1003, category: "streetlight", priority: "urgent", status: "resolved",
    subject: "Lamp repaired", assignee_id: 201,
    opened_at: ISODate("2026-02-03T00:00:00Z"),
    events: [
      { event_id: 5005, type: "created", at: ISODate("2026-02-03T00:00:00Z") },
      { event_id: 5007, type: "resolved", at: ISODate("2026-02-03T02:00:00Z") }
    ]
  },
  {
    ticket_id: 1004, category: "streetlight", priority: "low", status: "new",
    subject: "Unassigned report", assignee_id: null,
    opened_at: ISODate("2026-02-04T00:00:00Z"), events: []
  }
]);
```

There are four tickets, but only three are active. The resolved urgent ticket is deliberately present so a query that counts every urgent ticket gives the wrong active-workload answer. The empty event array makes another boundary case visible without requiring a large dataset.

An aggregation pipeline passes documents through stages. Each stage receives the previous stage's output.

Listing 140. MongoDB shell / JavaScript

```
db.tickets.aggregate([
  { $match: { status: { $in: ["new", "open", "in_progress"] } } },
  {
    $group: {
      _id: "$category",
      active_count: { $sum: 1 },
      newest_opened_at: { $max: "$opened_at" }
    }
  },
  { $sort: { active_count: -1, _id: 1 } }
])
```

Trace the grain:

1. input: one document per ticket;
2. after `$match`: zero or one output per input ticket, still one per active ticket;
3. after `$group`: one document per category; and
4. after `$sort`: same documents, new order.

This is the document equivalent of reasoning about relational operations and SQL logical order.

The expected groups are streetlight with two active tickets and newest opening February 4, and sanitation with one and newest opening February 2. Ticket 1003 does not contribute because it was filtered out before grouping. The resolved ticket remains stored in the collection: this pipeline reads data and does not delete the filtered document. Pipelines can write through explicit stages such as `$merge` or `$out`, but neither appears here.

The two uses of a dollar sign are worth separating. `$group` names an operator. `"$category"` is an expression that reads the current document's category field. Writing `"category"` instead would group every input under the same literal word. In `$sum: 1`, the number is a contribution from each input document, so the accumulator counts group members. In `$max: "$opened_at"`, each input contributes its opening date and the accumulator keeps the largest.

## 11.4 Put Selective Work Early When It Preserves Meaning

`$match` filters documents. When it appears early and matches an indexable query, MongoDB may reduce work before later stages.

`$project` selects or computes fields:

**Listing 141. MongoDB shell / JavaScript**

```
{
  $project: {
    _id: 0,
    ticket_id: 1,
    category: 1,
    event_count: { $size: { $ifNull: ["$events", []] } }
  }
}
```

Do not project fields away before a later stage needs them. Pipeline order is meaning, not only optimization.

`$ifNull` supplies an empty array when `events` is missing or null. It does not turn an arbitrary string into an array. The `$size` expression still needs an array, which is why the fixture's shape and later validation matter. A defensive expression must match a stated input contract rather than silently redefine bad data.

## 11.5 `$unwind` Changes One Document Into Many

**Listing 142. MongoDB shell / JavaScript**

```
db.tickets.aggregate([
  { $match: { ticket_id: 1003 } },
  { $unwind: "$events" },
  {
    $project: {
      _id: 0,
      ticket_id: 1,
      event_type: "$events.type",
      event_at: "$events.at"
    }
  },
  { $sort: { event_at: 1 } }
])
```

After unwind, the grain is one document per array element. Ticket 1003 in this chapter has two events, so this pipeline returns two rows ordered by their dates. A ticket with three events would produce three. This is like joining a one-to-many event table: multiplication may be correct.

Use `preserveNullAndEmptyArrays` when the question should keep documents with no array elements.

### 11.5.1 A Correct Total Can Hide an Incorrect Count

Consider a separate four-ticket instance with the following stated facts. Its active tickets are 1001, 1002, and 1004. Their event-array lengths are 2, 1, and 0.

| Stream | Ticket IDs represented | Meaning of one document |
|---|---|---|
| after the active filter | 1001, 1002, 1004 | one active ticket |
| after ordinary unwind | 1001, 1001, 1002 | one event inside an active ticket |

Both streams contain three documents:

$$N_{\text{tickets}} = 3, N_{\text{events}} = 2 + 1 + 0 = 3.$$

Yet ticket 1004 vanished and ticket 1001 contributes twice. Checking only the grand total misses both mistakes. For a ticket report, aggregate before unwinding or explicitly recover distinct ticket identity. For an event report, keep the unwind and label the result as events. Preserving an empty array keeps ticket 1004 in the stream but still does not remove the duplicate contribution of 1001.

## 11.6 `$group` Creates One Document Per Group Key

**Listing 143. MongoDB shell / JavaScript**

```
{
  $group: {
    _id: "$events.type",
    event_count: { $sum: 1 },
    latest_event_at: { $max: "$events.at" },
    ticket_ids: { $addToSet: "$ticket_id" }
  }
}
```

`_id` is the group key in this stage. `$sum`, `$avg`, `$min`, `$max`, `$push`, and `$addToSet` are common accumulators. `$group` does not guarantee output order; add `$sort` when order matters.

Grouping is a blocking operation: it may need to receive all relevant inputs before returning a group result. Filter unnecessary documents early.

Here `ticket_ids` is a set of distinct identifiers, while `event_count` counts the incoming event records. One ticket can contribute several events but only one member of that set. `$push` would instead retain repeated identifiers. Sets do not establish a presentation order; explicitly sort any output where order has meaning.

## 11.7 Worked Example: Active Workload by Agent

**Listing 144. MongoDB shell / JavaScript**

```
db.tickets.aggregate([
  {
    $match: {
      status: { $in: ["new", "open", "in_progress"] },
      assignee_id: { $exists: true, $ne: null }
    }
  },
  {
    $group: {
      _id: "$assignee_id",
      active_count: { $sum: 1 },
      urgent_count: {
        $sum: { $cond: [{ $eq: ["$priority", "urgent"] }, 1, 0] }
      }
    }
  },
  { $sort: { active_count: -1, _id: 1 } }
])
```

Verification: choose one agent ID and run a simple `find` with the same active predicate. Count and inspect those tickets. The simpler query verifies one group through a different path.

For the chapter fixture, agent 201 has one active ticket and one urgent active ticket; agent 202 has one active ticket and no urgent active ticket. The `$cond` expression contributes 1 when the priority is urgent and 0 otherwise. Resolved ticket 1003 and unassigned ticket 1004 are outside this report. An absent agent is not represented as a zero row, just as Chapter 2's assigned-only SQL grouping did not produce all staff. Producing a complete staff roster would require that roster as part of the query's starting population.

## 11.8 Flexible Documents Benefit From Focused Validation

MongoDB schema validation can require types, allowed values, ranges, and selected fields. It need not make every document identical.

The following rule checks both ticket fields and the structure of each event. It assumes events carry `event_id`, `type`, and `at`, as in the fixture above. Inspect or migrate existing event objects before adopting it. Do not paste a stricter rule onto unrelated data and assume the old documents now satisfy it. `required` checks field presence; add property type rules when the event values themselves must also have a particular type.

**Listing 145. MongoDB shell / JavaScript**

```
db.runCommand({
  collMod: "tickets",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["ticket_id", "status", "priority", "subject", "opened_at"],
      properties: {
        ticket_id: { bsonType: ["int", "long"] },
        status: {
          enum: ["new", "open", "in_progress", "resolved", "closed"]
        },
        priority: {
          enum: ["low", "medium", "high", "urgent"]
        },
        subject: { bsonType: "string" },
        opened_at: { bsonType: "date" },
        events: {
          bsonType: "array",
          items: {
            bsonType: "object",
            required: ["event_id", "type", "at"]
          }
        }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
})
```

Existing documents must be audited before strict validation. A rule can be added with different levels and actions for migration, but a warning-only policy is not the same as rejection. Record the intended rollout.

Adding the rule does not scan and repair every existing document. `strict` and `error` describe enforcement on relevant subsequent writes. A previously stored invalid document may remain until it is identified and migrated. This differs from assuming that successful configuration proves all historical data valid.

The supplied rule requires an event to *contain* `event_id`, `type`, and `at`, but does not yet constrain their value types. A stronger production rule can type-check event values too. Choose the invariant you need and test both a valid and invalid write rather than declaring "schema validation is on" as the result.

For example, after applying this chapter's rule, the following change should be rejected with validation error code 121:

**Listing 146. MongoDB shell / JavaScript**

```
db.tickets.updateOne(
  { ticket_id: 1004 },
  { $set: { opened_at: "2026-02-04T00:00:00Z" } }
)
```

The spelling looks like a date, but the proposed value is a string. Read ticket 1004 afterward and check that its BSON Date remains. In contrast, adding an optional field such as `campus_zone: "west"` is permitted by this rule: focused validation can protect core meaning while allowing intentional variation.

MongoDB uses BSON type names such as `date`, not JSON string conventions, in validation.

## 11.9 An Index Is an Ordered Access Path for a Workload

**Listing 147. MongoDB shell / JavaScript**

```
db.tickets.createIndex({ status: 1, opened_at: -1 })
```

This compound index is ordered by status, then opening time within status. It may support a query that filters one status and sorts newest first.

**Listing 148. MongoDB shell / JavaScript**

```
db.tickets.find(
  { status: "open" },
  { ticket_id: 1, subject: 1, opened_at: 1 }
).sort({ opened_at: -1 }).limit(20)
```

Index order and query shape matter. An index adds storage and write work and can complicate maintenance and backups. Avoid creating one index per field without a workload.

MongoDB's `$sort` is not stable: equal sort-key values need not keep a repeatable order. For deterministic pagination or a leaderboard, add a unique tie-breaker such as `ticket_id` to both the sort and a compatible compound index.

## 11.10 Explain Shows Plan and Execution Work

**Listing 149. MongoDB shell / JavaScript**

```
db.tickets.find({ status: "open" })
  .sort({ opened_at: -1 })
  .limit(20)
  .explain("executionStats")
```

Important beginner measurements include:

- `nReturned`: documents returned;
- `totalDocsExamined`: documents inspected;
- `totalKeysExamined`: index keys inspected;
- `executionTimeMillis`: observed server execution time for this run; and
- winning-plan stages such as `COLLSCAN`, `IXSCAN`, `FETCH`, and `SORT`.

Explain output can differ by MongoDB version and query engine. Focus on documented meaning rather than memorizing one nested path.

A collection scan is reasonable for a tiny collection or a query needing most documents. An index scan is useful only when it reduces or orders relevant work.

Read the counts together. A hypothetical plan returning 20 documents after examining 10,000 is doing a different amount of work from one returning the same 20 after examining 20 documents and 20 index keys. The ratio of documents examined to documents returned is 500 in the first case and 1 in the second. That ratio is undefined for zero returned documents and is not a complete cost model: an index-only result, a complex sort, and repeated cached runs need context too. Check record identity before comparing speed.

On this chapter's four-document fixture, the open-only query returns one ticket, not twenty. The limit is a maximum, and a scan can be a reasonable choice. A larger representative workload is needed to investigate

index selectivity and sort costs; this small fixture demonstrates query meaning, not production index performance.

## 11.11 Aggregations Can Use Indexes at the Beginning

An early `$match` and compatible `$sort` may use an index before stages reshape documents. `$unwind` and `$group` can prevent later stages from using a source collection index in the same way.

For example, an index on `category` does not precompute `active_count` for every possible filter. Sorting grouped results by that newly computed count still concerns the grouped stream. Likewise, moving `$limit: 20` before a workload group would count only twenty input tickets, not compute the complete workload and then choose the twenty largest groups. A cheaper computation is useful only when it answers the same question.

**Listing 150. MongoDB shell / JavaScript**

```
db.tickets.explain("executionStats").aggregate([
  { $match: { status: "open" } },
  { $sort: { opened_at: -1 } },
  { $limit: 20 },
  { $project: { ticket_id: 1, subject: 1, opened_at: 1 } }
])
```

Compare plans before and after the candidate index while keeping the pipeline identical.

Atlas Free does not allow aggregation disk spill and currently limits in-memory sort work to 32 MB. Keep practice fixtures bounded, filter early, and use a compatible index when ordering matters. A fast result on a tiny teaching collection describes that run, not production capacity.

## 11.12 Validation and Indexing Solve Different Problems

- Validation answers: may this document state exist?
- Indexing answers: how can the DBMS find or order matching documents?
- Aggregation answers: how should input documents become output documents?

One feature does not replace the others. A fast query can return inconsistent data; valid data can still be expensive to scan.

## 11.13 Common Misconceptions

### 11.13.1 “A pipeline stage is a table”

Each stage produces a document stream. The shape and grain can change repeatedly.

### 11.13.2 “Schema validation removes flexibility”

Focused rules preserve intentional variation while rejecting known invalid types or values.

### 11.13.3 “Docs examined should always be zero”

Some queries need document fields not covered by an index. The useful comparison depends on returned results and workload.

### 11.13.4 "Moving `$match` first is always correct"

Only move a predicate before a transformation when it has the same meaning on the earlier document shape.

## 11.14 Study, Cleanup, and Practice

Connect three distinct decisions: which records belong in the result, which stored states are valid, and how the database can find the records efficiently. An aggregation, a validator, and an index answer different parts of that problem.

If you ran the shell fixture, remove only its uniquely named practice database:

**Listing 151. MongoDB shell / JavaScript**

```
db.getSiblingDB(pipelinePracticeName).dropDatabase()
```

Design a pipeline for one row-like output per category containing active ticket count, urgent count, and newest opening time. Annotate the grain after every stage, then propose one index for its first stages and one validation rule for its input.

## 11.15 Retrieval and Transfer

1. Which pipeline stage commonly changes one document into many?
2. What does `_id` mean inside `$group`?
3. Why audit existing documents before strict validation?
4. How do `COLLSCAN` and `IXSCAN` differ?
5. Which three execution counts help evaluate selectivity?
6. Why should a performance comparison keep the pipeline identical?

## 11.16 Further Reading

- MongoDB aggregation stages
- MongoDB `$group`
- MongoDB schema validation
- MongoDB explain results
- MongoDB aggregation optimization

# 12 Reliability Is a Set of Explicit Promises

## 12.1 Operating Question

When a managed document database has multiple nodes, which failures can it hide, which data can a client read or lose, and which separate recovery artifact still needs to exist?

## 12.2 Learning Outcomes

After this chapter, you can:

- explain primary, secondary, oplog, election, and failover roles in a replica set;
- distinguish read preference, read concern, and write concern;
- describe the CAP tradeoff during a network partition without using "pick any two" as a product label;
- distinguish replication from backup;
- create a free-tier-appropriate MongoDB logical recovery plan; and
- connect a reliability promise to mechanism, failure, verification, and limitation.

## 12.3 Reliability Begins With a Specific Promise

The resident presses Submit, sees a confirmation, and closes the browser. One database member fails immediately afterward. The resident's question is not "how many servers were provisioned?" It is "does my confirmed request still exist, and can I read it?" Reliability connects a user-visible promise to the failure conditions under which the system can keep it.

"Highly available" is incomplete. A useful promise identifies:

- operation: read, write, query, restore, or reconnect;
- failure: process crash, node loss, network partition, bad deployment, or deletion;
- acceptable behavior: continue, reject, retry, use older data, or recover later;
- time or data-loss objective; and
- verification and limitation.

Example:

> A ticket write acknowledged with majority write concern should survive loss of a minority of voting data-bearing replica-set members, subject to the deployment's durability settings. This does not protect against a later authorized deletion, so a separate logical recovery artifact is required.

## 12.4 Replica Sets Copy an Ordered Change History

A MongoDB replica set contains members that maintain the same logical dataset.

- The **primary** accepts writes under normal operation.

- **Secondaries** replicate operations from the primary's oplog and apply them.
- The **oplog** is a bounded, ordered log of data changes used for replication.
- An **election** can choose a new primary when the old one is unavailable and a voting majority can agree.

Replication is asynchronous. A secondary can lag behind the primary. Election and client reconnection take time, so failover is not instantaneous or invisible to every operation. Drivers discover topology and retry only according to supported settings and operation semantics.

Suppose A is primary and B and C are secondaries. A write reaches A first and is then replicated. During a brief interval, B may have applied it while C has not. That does not mean the replicas are independent backups; they are catching up with the active change history. If a secondary falls farther behind than the retained oplog history can cover, normal incremental catch-up may no longer be enough and resynchronization is needed. A bounded replication log is not an indefinite archive of every historical application state.

Atlas Free uses a fixed three-node replica-set configuration. This tier does not let you change its replication factor or run Atlas failover testing. The chapter explains topology and client behavior without requiring either operation.

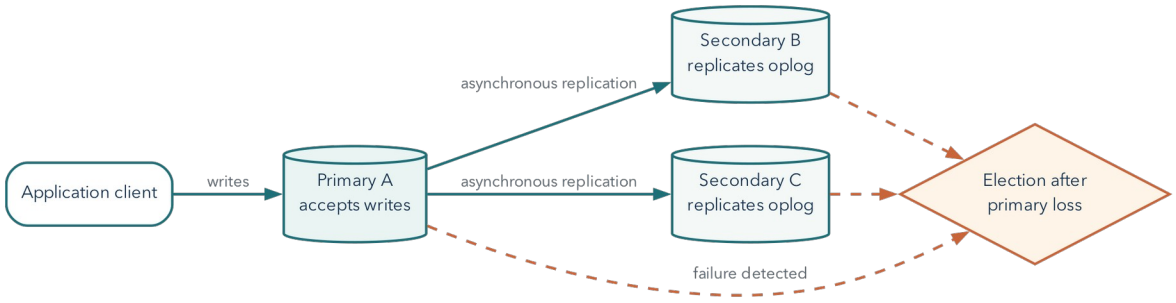



*Figure 12.1: A replica set copies an ordered change history; failover requires detection, an election, and client reconnection.*

## 12.5 Write Concern Defines Required Acknowledgment

Write concern describes what acknowledgment a client waits for.

The following shell example belongs in a fresh disposable database, not in the previous chapter's validated fixture. Its stable `_id` identifies this particular submission attempt. The example observes a normal acknowledgment; it does not perform or certify a failover test.

**Listing 152. MongoDB shell / JavaScript**

```
const reliabilityPracticeName =
  "cst4714_reliability_" + ObjectId().toHexString().slice(-8);
db = db.getSiblingDB(reliabilityPracticeName);
db.tickets.insertOne(
  {
    _id: "submission-1099", ticket_id: 1099,
    status: "new", subject: "Write concern test"
  },
  { writeConcern: { w: "majority", wtimeout: 5000 } }
)
```

For a typical three-member replica set, `w: "majority"` requires acknowledgment from a calculated majority of voting data-bearing members. Starting in MongoDB 8.0, with the default majority journaling setting, that acknowledgment waits for durable oplog entries, not for every acknowledging secondary to apply the change to its collection. A secondary can retain an acknowledged insert and still need to apply it before a collection query can see the document. Waiting for more acknowledgments can improve the write's resilience to rollback but can add latency or reduce successful writes when enough members are unavailable.

A write-concern timeout means the requested acknowledgment was not received in time. It does not necessarily prove that no member applied the write. Applications need idempotent identifiers, read-back strategy, and retry rules that avoid creating duplicates.

Here the identifier gives a read-back question: does `submission-1099` exist, and does its content match the requested operation? Repeating the insert with the same identifier cannot silently create a second document with that `_id`. A duplicate-key error is not a general success signal, however; a caller must confirm that the existing record represents the same intended submission. An identifier reused for a different request would be another error.

For the ordinary three-data-bearing-member example, a majority is two. Durability also depends on journaling and the documented write-concern behavior; do not generalize this arithmetic to every voting or arbiter configuration. A `wtimeout` bounds the acknowledgment wait, not necessarily completion of the underlying write. Losing the response can leave the caller uncertain even when the write is eventually retained. MongoDB write concern

Write concern is not backup retention. A majority-acknowledged deletion is still a durable deletion.

## 12.6 Read Preference Chooses Eligible Members

Common read preference modes include:

- `primary`: read from the primary; default for normal driver reads;
- `primaryPreferred`: prefer primary, permit secondary under documented conditions;
- `secondary`: read from a secondary;
- `secondaryPreferred`: prefer a secondary, permit primary; and
- `nearest`: select among eligible members by latency window and tags.

Reading from a secondary can distribute some read work or serve locality goals, but asynchronous replication means the result may be behind the primary. A read-your-own-write workflow should not switch casually to a lagging secondary.

Read preference is routing. It is not the same as read concern.

## 12.7 Read Concern Defines Visibility Guarantees

Read concern controls the consistency and isolation properties of returned data. For example, `local` can return data from the member without guaranteeing that a majority has durably committed it; `majority` limits reads to majority-committed data under its documented semantics.

The combined behavior depends on read preference, read concern, write concern, sessions, and topology. Avoid describing a whole database as simply "consistent" or "eventually consistent" without naming the operation and settings.

Think of the three controls as separate questions: **where may I read**, **which state may that read return**, and **what must be acknowledged before my write returns**? A majority read from a lagging eligible member is not a promise that the latest possible write is immediately visible everywhere. For workflows that must read their own preceding write, MongoDB documents causal sessions with the appropriate majority read and write concerns. A primary-only read shortcut does not replace understanding failure, session, and ordering behavior. Read isolation, consistency, and recency

## 12.8 CAP Is a Partition-Time Tradeoff

The CAP theorem concerns a distributed read/write data object when messages between parts of the system can be lost or delayed indefinitely.

- **Consistency** in the theorem is a single-copy/linearizable style guarantee, not the generic ACID consistency property.
- **Availability** means each operation requested at a non-failing node completes with the required operation response. Returning "unavailable" to every request is not a solution, and this definition is not a monthly uptime percentage.
- **Partition tolerance** addresses communication partitions between nodes.

When a partition separates nodes, a system cannot guarantee both that every partition continues answering every request and that all answers behave like one current copy. It must reject or delay some operations, permit potentially divergent or stale behavior, or use a more specific compromise.

"Choose two of three" is misleading because a real distributed system cannot wish network partitions away. The practical question is which operations remain available during a partition and which consistency behavior they sacrifice or preserve.

### 12.8.1 What the Partition Forces Us to Decide

Suppose A and B can communicate, but C cannot exchange messages with them. A resident's write reaches the A/B side and completes there. A later read reaches C. C cannot know that new value by asking the other side while the partition persists. If C answers from old state, it can violate the single-current-copy promise. If it refuses to complete the read until coordination is possible, that operation is unavailable under the theorem's definition. This is the concrete conflict, not an instruction to disable an arbitrary feature.

**Linearizability** means operations can be understood as taking effect one at a time, in an order respecting completed operations before later requests begin. It is not the same as constraints keeping each document valid. Two perfectly valid ticket documents can still show incompatible current answers to clients. The Gilbert-Lynch formalization explains the impossibility under its stated network model. Real designs specify narrower operations, bounded-staleness promises, and failure assumptions rather than replacing that result with a product slogan. Gilbert and Lynch, 2002

## 12.9 Replication Is Not Backup

Replication helps with member failure and service continuity. It can also replicate:

- an accidental deletion;
- a bad update;
- corrupted application data;
- an authorized destructive command; or
- an unwanted schema change.

A backup or logical export creates recoverable history outside the active replica state. Recovery still needs a target, procedure, credentials, compatibility, and verification.

## 12.10 Recovery Objectives Describe Different Losses

Chapter 8 introduced two objectives that remain useful for document databases. The **recovery point objective (RPO)** is the tolerated gap in recoverable work, expressed as time. The **recovery time objective (RTO)** is the tolerated interval until service resumes. Both are targets. A rehearsal supplies observations to compare with those targets; setting a target does not cause a database to meet it.

Suppose a usable artifact represents the complete dataset at 16:10. An accidental deletion interrupts service at 16:22. No later export or replayable event log exists. With times expressed on the same clock, the uncovered interval is

$$\Delta_{\text{data}} = t_{\text{incident}} - t_{\text{artifact}} = 12\,\text{minutes}.$$

The artifact misses a five-minute RPO. Restoring it faster cannot add the missing changes. The calculation does not tell us how many tickets were lost: we would need the actual intervening writes to establish that. It tells us the period for which this artifact offers no recovery coverage.

The represented time matters. An export that *finishes* at 16:10 might have read its data over an earlier interval. Completion time alone does not prove that related records form one consistent 16:10 state. This example explicitly grants a complete point-in-time artifact so that the calculation has a defensible basis.

For the same incident, suppose detection takes two minutes, locating and authorizing the artifact takes one minute, and restoration plus application checks takes four minutes. If these phases follow one another, then

$$\Delta_{\text{service}} = t_{\text{detection}} + t_{\text{preparation}} + t_{\text{restore and checks}} = 2 + 1 + 4 = 7\,\text{minutes}.$$

That rehearsal fits a ten-minute RTO while still missing the five-minute RPO. When only restore execution is measured, the whole incident-to-service interval remains unknown. A realistic recovery plan must also provide access to the artifact, an appropriate target, and someone who can carry out the procedure. More frequent retained exports may reduce the recovery-point gap, but only a working schedule and a consistent capture method can support the proposed bound.

## 12.11 Atlas Free Requires Manual Logical Recovery

Current Atlas documentation states that native Atlas backups are unavailable for Free clusters. Free clusters also do not support sharded-cluster creation or primary-failover testing. The documented backup alternative is `mongodump` and `mongorestore`.

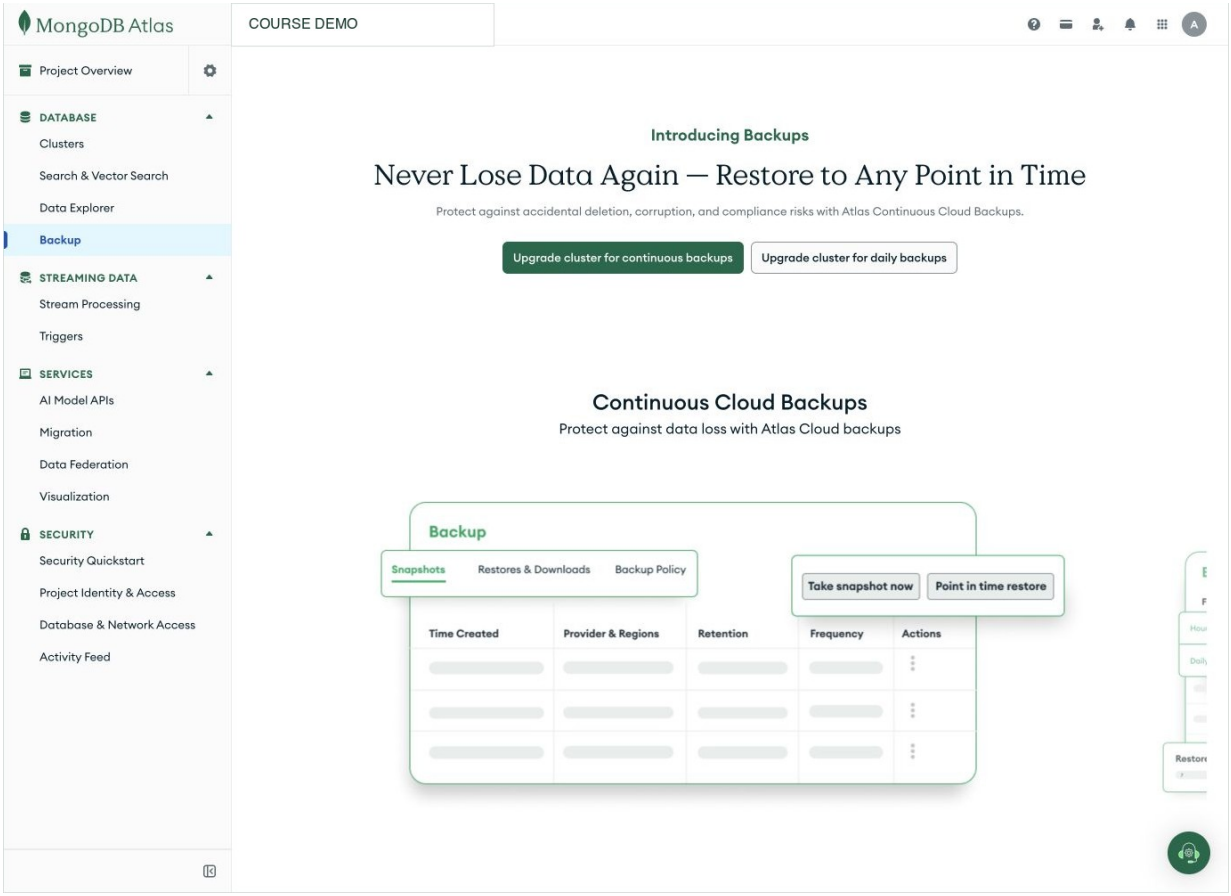



*Figure 12.2: The Atlas Free backup screen presents paid continuous or daily backup upgrades rather than an active native backup. A free deployment therefore needs a separate logical recovery plan. Account identifiers are redacted. Interface captured August 25, 2026.*

The interface shows that native backup is outside the current plan boundary; it does not represent a completed backup. Record the limitation, create a logical backup with current Database Tools, and restore it into a separate target to test the recovery procedure.

**Listing 153. Shell**

```
mongodump \
  --uri="$MONGODB_URI_NO_PASSWORD" --username="$MONGODB_USER" \
  --db="$SOURCE_DATABASE" \
  --archive=metro_support.archive.gz \
  --gzip
```

These are terminal commands, not MQL or Python cells. Set the source database to the actual practice database being exported. The URI variable must contain only connection routing/options, not a password; with a username and no supplied password, the tool prompts for it. For unattended use, Database Tools support a restricted configuration file for sensitive values. A password-bearing URI expanded into a command argument can be exposed in a process listing even when the script itself contains only a variable name. MongoDB Database Tools

Inspect the exit status, file size, hash, and tool output. A logical dump taken while related collections are changing is not automatically a consistent application-wide snapshot. This example assumes a small quiescent dataset. Production snapshot and oplog options have deployment-specific limits and must be evaluated separately; do not assume an Atlas Free cluster provides every option.

Restore into a different database name:

**Listing 154. Shell**

```
mongorestore \
  --uri="$MONGODB_RESTORE_URI_NO_PASSWORD" --username="$MONGODB_RESTORE_USER" \
  --archive=metro_support.archive.gz \
  --gzip \
  --stopOnError \
  --nsFrom="${SOURCE_DATABASE}.*" \
  --nsTo="${RESTORE_DATABASE}.*"
```

Review current Database Tools compatibility. Logical dumps include collection documents, metadata/options, and index definitions within documented behavior, but cluster users, project network rules, and every Atlas configuration are not simply recreated by restoring one database dump.

Choose a new restore database name and confirm it is absent before restoring. Unlike a PostgreSQL transactional restore, this command is not an all-or-nothing transaction across the archive. After an error, a partially restored target may exist. Isolate it, inspect the error, and recreate only that disposable target before repeating a corrected restore. Do not use a broad drop option on the source to make a failed test appear clean.

## 12.12 `mongoexport` Is Useful but Narrower

`mongoexport` produces JSON or CSV data suitable for interchange. It is helpful for a collection-level exercise or migration, but it is not a complete backup system. Type preservation depends on the chosen representation and parser: CSV does not encode BSON types, while Canonical Extended JSON can preserve type distinctions for a BSON-aware importer. Neither format, by itself, reinstalls collection options, validators, indexes, privileges, or a consistent multi-collection point.

The worked example below uses **BSON-aware Extended JSON** serialization and parsing to preserve its date values. Its data file does not by itself carry collection validators or indexes; those are examined and rebuilt separately. This transparent collection exercise is compared with `mongodump`, not mislabeled as an equivalent backup of an Atlas project. A plain JSON round trip that turns dates into strings would teach a different and incomplete recovery.

## 12.13 Verify a MongoDB Restore

Connect mongosh to the restore deployment and select the actual restore database before running these checks. Do not leave the earlier write-concern practice database selected. The expected collection names and rules come from your source manifest, not from whatever happens to appear in the current shell.

### 12.13.1 Structure and Rules

**Listing 155. MongoDB shell / JavaScript**

```
db.getCollectionNames()
db.getCollectionInfos({ name: "tickets" })
db.tickets.getIndexes()
```

Confirm collections, validators, and required indexes.

### 12.13.2 Data and Relationships

**Listing 156. MongoDB shell / JavaScript**

```
db.tickets.countDocuments({})
db.ticket_events.countDocuments({})
db.tickets.countDocuments({ status: "open" })
```

Trace known identifiers and check for references with no target when the model uses references.

Only expect a `ticket_events` collection when the model actually stores events separately. Counting a nonexistent collection can return zero, which is not proof that its intended backup succeeded. The worked example below has just one collection, `tickets`, and records its rules separately. It has no `ticket_events` collection to restore.

### 12.13.3 Behavior

Run one meaningful aggregation and one expected validation failure in the restore target. If recovery scope includes application access, test a temporary read and write using a least-privilege credential.

### 12.13.4 Limitation

Counts and one query do not prove every document value, historical moment, permission, or application path is correct.

### 12.13.5 Worked Recovery: Correct Count, Incorrect Subject

The following Python experiment starts with five fresh synthetic tickets. It retains a separate document artifact, then restores into a differently named database. Its active ticket IDs are 1001, 1002, and 1004; the two resolved tickets also belong in the recovery. An active-ticket query alone would not verify the complete collection.

Install PyMongo and start the local MongoDB server from Appendix C. Run this section's Python blocks in order in one session. Each database gets a generated name; nothing here drops an existing application database. A temporary directory holds the rehearsal's artifact until the explicit cleanup block. This is an observable recovery experiment, not a retained production backup.

Listing 157. Python

```
from datetime import datetime, timezone
from hashlib import sha256
from pathlib import Path
from tempfile import TemporaryDirectory
from uuid import uuid4
from bson import json_util
from pymongo import MongoClient

client = MongoClient("mongodb://127.0.0.1:27018/",
                     serverSelectionTimeoutMS=5000)
client.admin.command("ping")
run_id = uuid4().hex[:12]
source_name = "book_source_" + run_id
restore_name = "book_restore_" + run_id
validator = {"$jsonSchema": {
    "bsonType": "object",
    "required": ["ticket_id", "status", "subject", "opened_at"],
    "properties": {
        "ticket_id": {"bsonType": "int"},
        "status": {"enum": ["new", "open", "in_progress", "resolved"]},
        "subject": {"bsonType": "string"},
        "opened_at": {"bsonType": "date"},
    },
}}
source_tickets = client[source_name].create_collection(
    "tickets", validator=validator
)
source_tickets.create_index("ticket_id", unique=True)
source_tickets.create_index([("status", 1), ("opened_at", -1)])
source_tickets.insert_many([
    {"ticket_id": number, "status": status, "subject": subject,
     "opened_at": datetime(2026, 2, 1, 23, 10, tzinfo=timezone.utc)}
    for number, status, subject in [
        (1001, "open", "Streetlight dark near bus stop"),
        (1002, "in_progress", "Missed collection"),
        (1003, "resolved", "Water meter repaired"),
        (1004, "new", "Damaged park bench"),
        (1005, "resolved", "Pothole repaired"),
    ]
])
```

Read from the server rather than assuming the inserted Python values are the stored representation. Save the documents and a manifest as separate files. This small source is quiescent: nothing else writes it during the export.

**Listing 158. Python**

```python
workspace = TemporaryDirectory(prefix="book-recovery-")
artifact = Path(workspace.name) / "tickets.ejson"
manifest_file = Path(workspace.name) / "manifest.ejson"
options = json_util.CANONICAL_JSON_OPTIONS
documents = list(source_tickets.find().sort("ticket_id", 1))
artifact.write_text(
    json_util.dumps(documents, json_options=options, sort_keys=True),
    encoding="utf-8",
)
manifest = {
    "sha256": sha256(artifact.read_bytes()).hexdigest(),
    "ids": [doc["ticket_id"] for doc in documents],
    "count": len(documents),
    "validator": source_tickets.options()["validator"],
    "indexes": [dict(index) for index in source_tickets.list_indexes()],
}
manifest_file.write_text(
    json_util.dumps(manifest, json_options=options), encoding="utf-8"
)
print("Saved", manifest["count"], "documents")  # Saved 5 documents
```

The file uses Canonical Extended JSON. These are selected fields from ticket 1001, not its complete stored document:

**Listing 159. JSON**

```json
{
  "ticket_id": { "$numberInt": "1001" },
  "opened_at": {
    "$date": { "$numberLong": "1769987400000" }
  }
}
```

The date number represents milliseconds since the Unix epoch. The following `json_util.loads` call understands that marker and reconstructs a Python `datetime`. Python's ordinary `json.loads` instead returns the nested dictionary. Both parsers read valid JSON; only one interprets the BSON type information. The `$numberInt` marker also distinguishes an integer from a floating-point value with the same numeric magnitude.

The next block compares the saved file's SHA-256 with its earlier manifest before parsing or resetting the target. Matching hashes support the claim that the file bytes have not changed relative to that manifest. They do not establish that the export included every intended record or came from a trusted author. The manifest therefore also records the expected IDs, count, and separate collection rules. The default `_id` index is automatic; recreate the two additional indexes from their saved key order and uniqueness setting. This handles the ordinary indexes used here, not every possible index option.

Listing 160. Python

```
saved_manifest = json_util.loads(manifest_file.read_text(encoding="utf-8"))
if sha256(artifact.read_bytes()).hexdigest() != saved_manifest["sha256"]:
    raise RuntimeError("Artifact changed; do not restore it")
restored_documents = json_util.loads(artifact.read_text(encoding="utf-8"))
restore_tickets = client[restore_name].create_collection(
    "tickets", validator=saved_manifest["validator"]
)
for index in saved_manifest["indexes"]:
    if index["name"] != "_id_":
        restore_tickets.create_index(
            list(index["key"].items()), name=index["name"],
            unique=index.get("unique", False),
        )
restore_tickets.insert_many(restored_documents)
restored = list(restore_tickets.find().sort("ticket_id", 1))
assert len(restored) == saved_manifest["count"] == 5
assert [doc["ticket_id"] for doc in restored] == saved_manifest["ids"]
assert all(isinstance(doc["opened_at"], datetime) for doc in restored)
expected = json_util.dumps(
    restored_documents, json_options=options, sort_keys=True
)
assert json_util.dumps(restored, json_options=options, sort_keys=True) == expected
print("Restored identifiers, types, and document values match")
```

After a correct restore, an erroneous migration replaces ticket 1001's subject with `Text replaced by an incorrect migration`. The count remains five, the IDs still match, and every opening date remains a `datetime`. Even a validator that requires the subject to be a string accepts this wrong text. A schema can check the form of a value without knowing whether it describes the intended request.

The following type-aware comparison of all saved document values exposes the mistake. For this small, consistently ordered fixture, it compares canonical serializations with sorted object keys. That also catches a numeric-type change that ordinary Python equality might treat as equal. Larger recoveries need verification methods appropriate to their data volume and application invariants.

Listing 161. Python

```
restore_tickets.update_one(
    {"ticket_id": 1001},
    {"$set": {"subject": "Text replaced by an incorrect migration"}},
)
damaged = list(restore_tickets.find().sort("ticket_id", 1))
print("Count:", len(damaged))  # Count: 5
print("Values match:", json_util.dumps(
    damaged, json_options=options, sort_keys=True
) == expected)  # Values match: False
```

The saved version supplies a repair without guessing the original text. Run this example after the preceding blocks have verified and parsed the artifact into `restored_documents` and initialized its separate `restore_tickets` target:

Listing 162. Python

```
saved_by_id = {
    doc["ticket_id"]: doc for doc in restored_documents
}
recovery_document = saved_by_id[1001]
repair = restore_tickets.replace_one(
    {"ticket_id": 1001}, recovery_document
)
print(repair.matched_count, repair.modified_count)
```

The lookup selects the document retained in the artifact. The filter selects the damaged ticket in the target. `replace_one` supplies its full saved version, including the original `_id`. For this damage case the first repair prints `1 1`: one document matched and changed. Repeating it prints `1 0`: the same document matches but already has the saved values. There are still five tickets. This repeatable replacement is suitable for the isolated recovery target; blindly replacing a live document could overwrite legitimate changes made after the export.

Finally, test the rules, not just the documents. The manifest, not the data file, supplied the unique index and validator. This block checks a permitted write, a duplicate ticket number, and three invalid document shapes against the real MongoDB server. A permission error is not accepted as proof of validation.

**Listing 163. Python**

```python
from pymongo.errors import DuplicateKeyError, WriteError

candidate = dict(recovery_document)
candidate.pop("_id")
try:
    restore_tickets.insert_one(dict(candidate))
    raise AssertionError("Duplicate ticket number was accepted")
except DuplicateKeyError as error:
    assert error.code == 11000

candidate["ticket_id"] = 1099
restore_tickets.insert_one(dict(candidate))
assert restore_tickets.delete_one({"ticket_id": 1099}).deleted_count == 1
for field, value in [("status", "unknown"), ("opened_at", "2026-02-01"),
                     ("opened_at", None)]:
    invalid = dict(candidate)
    if value is None:
        invalid.pop(field)
    else:
        invalid[field] = value
    try:
        restore_tickets.insert_one(invalid)
        raise AssertionError("Invalid document was accepted")
    except WriteError as error:
        assert error.code == 121
final = list(restore_tickets.find().sort("ticket_id", 1))
assert json_util.dumps(final, json_options=options, sort_keys=True) == expected
print("Repair and rule checks passed")

client.drop_database(restore_name)
client.drop_database(source_name)
client.close()
workspace.cleanup()
```

Cleanup removes these two generated databases and this temporary rehearsal's files. A real recovery policy would retain protected artifacts independently of the source and of this process. The optional recovery notebook offers another interface; its no-account library path is not a substitute for the server validation exercised above.

## 12.14 Worked Example: Reliability Decision Matrix

**Workload:** a resident submits a ticket and immediately sees confirmation. A short delayed analytics report is acceptable. A confirmed ticket should not be silently lost after one member failure.

| Decision | Choice | Reason | Cost/limit |
|---|---|---|---|
| ticket write | majority acknowledgment with bounded timeout | stronger resilience for confirmed write | latency and possible timeout ambiguity |
| confirmation read | primary with appropriate concern/session behavior | avoid stale immediate confirmation | primary read load |
| analytics read | secondary-preferred where acceptable | stale report is tolerable | data can lag |
| accidental deletion | logical backup/restore | replication copies deletion | recovery point and restore time depend on export schedule |

The choices follow user expectations, not a universal strongest-setting rule.

## 12.15 Common Misconceptions

### 12.15.1 "Three nodes means three backups"

Replicas share the active history and can reproduce destructive changes. Backup is a separate artifact and recovery process.

### 12.15.2 "Majority write concern means the write definitely failed on timeout"

The acknowledgment condition timed out. The write may exist on some members. The application needs safe retry and verification logic.

### 12.15.3 "Secondary reads are always faster"

Topology, latency, lag, indexes, workload, and routing determine behavior. They may be stale and are not automatically faster.

### 12.15.4 "CAP says every database picks two letters forever"

The tradeoff is about behavior during a network partition under specific formal definitions.

## 12.16 Study, Cleanup, and Practice

Separate the promises of replication, acknowledgment, and recovery. For each decision below, state the failure it addresses and one failure that would still require a different mechanism.

If you ran the acknowledgment example, its cleanup is:

Listing 164. MongoDB shell / JavaScript

```
db.getSiblingDB(reliabilityPracticeName).dropDatabase()
```

That removes only the newly named practice database, not the Atlas deployment or an unrelated source or restore database.

Write a reliability matrix for a small inventory system. Include one critical write, one stale-tolerant read, one partition scenario, one accidental deletion, and one restore verification. For every choice, name a mechanism and limitation.

## 12.17 Retrieval and Transfer

1. What is the oplog's role in a replica set?
2. How do write concern and read preference differ?
3. Why can a write-concern timeout produce an ambiguous application outcome?
4. What CAP decision appears only during a partition?
5. Why does a replica not replace a backup?
6. How do Atlas Free limits affect a feasible reliability and recovery rehearsal?

## 12.18 Further Reading

- MongoDB replication
- Replica-set read and write semantics
- MongoDB write concern
- MongoDB read concern
- Atlas Free limits
- MongoDB `mongodump`
- PyMongo Extended JSON
- MongoDB `mongoexport` limitations and formats
- Gilbert and Lynch, CAP formalization

# Part IV: Scale and Integration

Scale is not merely a larger number of rows. It changes coordination, routing, capacity, cost, recovery, observability, and the number of failure combinations an operator must understand. Chapters 13-14 examine workload-based capacity, sharding decisions, safe imports, client connection contracts, polyglot persistence, event-driven propagation, and reconciliation.

The central question remains ownership of truth. A multi-database design is defensible only when each fact has an authoritative source, partial failures are expected, and a repair path is designed before an incident.

# 13 Scale Changes the Questions a System Must Answer

## 13.1 Operating Question

When data or traffic grows, how do we decide whether to tune, scale up, partition, or shard, and how can a small Python client expose the consequences?

## 13.2 Learning Outcomes

After this chapter, you can:

- describe capacity using workload, storage, latency, throughput, and growth;
- distinguish vertical scaling, read replicas, partitioning, and sharding;
- explain the roles of shards, replica sets, config servers, and `mongos`;
- evaluate shard-key cardinality, frequency, monotonicity, and query targeting;
- compare ranged and hashed distribution;
- connect to a cloud database from Python without saving credentials in code or output;
- explain stable-key imports and their transaction boundaries; and
- compare stored values and a grouped result with the selected source.

## 13.3 Capacity Planning Starts With a Workload

The word scale can describe several different changes. A table can grow while traffic stays low. Traffic can grow while the dataset stays small. A once-per-day report can become an interactive request whose user will not wait. These cases stress storage, throughput, and response time differently. Buying more capacity before identifying the pressure may increase cost without improving the user experience.

“Will this database scale?” has no answer without quantities and objectives.

Record:

- current and expected records or documents;
- average and high-percentile object size;
- read and write operations by query shape;
- peak versus average throughput;
- acceptable latency and error rate;
- working set versus available memory;
- index and backup growth;
- retention and deletion behavior; and
- expected growth horizon.

A beginner project should not fabricate enterprise traffic. It should state a small current workload, one plausible growth scenario, and the first measurement that would trigger a change.

**Throughput** is completed work per unit time, such as requests per second. **Latency** is the time one request takes. A system can finish many requests per second while some users wait a long time. A percentile describes that spread: the 95th-percentile latency is a value at or below which roughly 95 percent of the measured requests fall, under the stated calculation method. It is not the average, and it says nothing about requests omitted from the measurement.

The **working set** is the data and index pages an active workload repeatedly uses. A database larger than memory can still serve a small hot working set efficiently. Conversely, adding many speculative indexes can crowd useful pages out of cache. Before changing architecture, connect the observed pressure to the actual query mix and data distribution.

### 13.3.1 Worked Example: An Average and a Slow Tail

Suppose 18 requests take 20 milliseconds each, one takes 500, and one takes 1,000. These are invented observations used to explain the calculation, not a benchmark of a cloud provider. Their arithmetic mean is:

$$\bar{t} = \frac{18(20) + 500 + 1000}{20} = 93\,\text{ms}.$$

The nearest-rank 95th percentile uses rank $\lceil 0.95\,n \rceil$ in the sorted observations. With $n = 20$, that is rank 19, whose value is 500 ms. Python's list positions start at zero, so the implementation subtracts one from the rank:

Listing 165. Python

```python
from math import ceil

latencies_ms = [20] * 18 + [500, 1000]
ordered = sorted(latencies_ms)
mean_ms = sum(ordered) / len(ordered)
p95_ms = ordered[ceil(0.95 * len(ordered)) - 1]
print(mean_ms, p95_ms)  # 93.0 500
```

The mean and percentile describe different aspects of the same observations. Neither describes failed requests that the measurement omitted. Other percentile conventions can interpolate between observations, so record the calculation method, sample size, query mix, and observation window when comparing reports.

## 13.4 Fix Waste Before Distributing It

Scaling options include:

- **query/model improvement:** reduce unnecessary work;
- **vertical scaling:** add CPU, memory, or faster storage to one node or tier;
- **caching:** avoid repeated work with an explicit freshness policy;
- **read replication:** serve selected reads from copies with consistency tradeoffs;
- **partitioning:** divide a logical dataset within one database system; and
- **sharding:** distribute parts of a collection or dataset across database nodes.

Horizontal distribution adds network failures, routing, rebalancing, cross-shard operations, monitoring, and recovery complexity. Do not shard a bad query merely to run the bad query on more machines.

Replication and sharding are especially easy to confuse. Replicas maintain copies of a logical dataset so a service can tolerate member failure or serve eligible reads elsewhere. Shards hold different portions of a dataset. In a sharded MongoDB deployment, each shard can itself be replicated. Adding three replicas of one dataset is not the same as dividing that dataset into three independently placed portions.

## 13.5 MongoDB Sharding Distributes Collection Ranges

A sharded MongoDB cluster includes:

- **shards:** each shard stores part of the collection and is normally a replica set for availability;
- **config servers:** store cluster metadata and configuration;
- **`mongos`:** query routers that use metadata to target shards and merge results; and
- **chunks/ranges:** non-overlapping shard-key ranges assigned across shards.

The shard key is an indexed field or compound of fields that determines document distribution. It becomes a long-lived architectural decision even though modern MongoDB supports refinement and resharding.

Atlas Free cannot create a sharded cluster. You can still compare candidate shard keys against the query and write patterns below without deploying that architecture. This design analysis does not measure a running sharded system.

## 13.6 Four Shard-Key Questions

### 13.6.1 Cardinality

How many distinct key values exist? A Boolean or five-value status field cannot create many independently distributable ranges by itself.

The supplied 75-record fixture has 75 distinct CVE identifiers and 38 distinct date-added values. Those candidate fields have different cardinalities. This describes the sample's distribution, not how often users will query any particular record. Always separate a measured record distribution from an assumed traffic pattern.

### 13.6.2 Frequency

How often does each value occur? A high-cardinality key can still be skewed if one value dominates traffic or document count.

Hashing a repeated value does not split its occurrences among arbitrary shards: the same input value produces the same hash value. A key with a million identical values therefore retains that concentration under hashing. A more varied full key may be necessary; "hashed" is not a repair for every skewed model.

### 13.6.3 Monotonicity

Does the key continually increase or decrease, such as a timestamp or sequential identifier? Range sharding on a monotonic key can direct new writes toward the chunk holding the current extreme, creating a hotspot.

### 13.6.4 Query Targeting

Do common queries include the shard key? If a query does not provide enough shard- key information, the router may broadcast it to multiple shards in a scatter- gather operation.

An ideal candidate balances distribution and targeting. No single field property guarantees both.

## 13.7 Ranged and Hashed Distribution Make Different Tradeoffs

### 13.7.1 Ranged Sharding

Nearby shard-key values stay in nearby ranges. A query for a bounded range can be targeted, but monotonic inserts or skew can concentrate load.

### 13.7.2 Hashed Sharding

MongoDB hashes the shard-key value to distribute documents more evenly. This can spread monotonic identifiers, but range queries on the original values lose locality and may contact many shards.

Example:

- `{ neighborhood: 1, opened_at: 1 }` can support locality and targeted neighborhood-time queries but may skew if one neighborhood dominates.
- `{ ticket_id: "hashed" }` can distribute identifier writes but does not target a query that only filters neighborhood.

Choose from the actual query and write distribution.

Imagine sorting cards by neighborhood and date. A neighborhood/date-range question can identify a contiguous region of that arrangement. Shuffling the cards by a stable hash of ticket ID distributes nearby IDs but loses that date locality. The analogy explains the tradeoff; actual MongoDB placement also uses chunks, routing metadata, and balancing. A simple bucket simulation is not an implementation of the server's hash function or placement algorithm.

## 13.8 Cross-Shard Operations Cost More Coordination

A request can be:

- **targeted:** routed to one shard or limited shard set using the shard key; or
- **scatter-gather:** sent broadly and merged by `mongos`.

Cross-shard aggregations, transactions, uniqueness, and resharding can add network and coordination cost. A design can still use them; the cost should be visible.

## 13.9 Worked Example: Evaluate Metro Support Candidates

Hypothetical workload, not a measurement of the small supplied fixture:

- writes arrive across neighborhoods;
- dashboards filter neighborhood and recent opening time;

- support staff retrieve exact ticket IDs; and
- one central neighborhood produces 45 percent of tickets.

| Candidate | Distribution | Targeting | Risk |
|---|---|---|---|
| `status` | very low cardinality | helps status query only | jumbo/hot values |
| `opened_at` ranged | high and monotonic | good time ranges | latest-write hotspot |
| `ticket_id` hashed | likely even writes | exact ID target | neighborhood/time reports scatter |
| `(neighborhood, ticket_id hashed component)` | spreads within neighborhood | neighborhood queries can target a subset depending on full key | more complex routing and index |

The table does not produce a universal winner. It identifies what data and query measurements are needed before deployment.

## 13.10 Python Makes the Client Contract Visible

A driver connection includes network routing, TLS, authentication, server selection, timeouts, and query behavior. Keep the first program small.

A **driver** is the library that speaks the database protocol from a programming language. Installing it does not create a database or authorize access. The following examples use the Python environment in Appendix C. For the Atlas read, first create Chapter 10's single-ticket shell example on your personal deployment and retain its generated database name. For the PostgreSQL read, run Appendix C's complete relational baseline first. The integration notebook is an optional alternative interface, not a prerequisite for these programs.

### 13.10.1 MongoDB Atlas

For Colab, add the notebook runtime's public IPv4 `/32` in Atlas's IP access list, not just the laptop address suggested by a browser's **Add Current IP** button. Run this Python block on the machine that will connect. It asks an external IP-echo service which public IPv4 address made the request. The result can change after a runtime restart. It is not a database credential; do not substitute this value into the database hostname.

**Listing 166. Python**

```python
from ipaddress import IPv4Address
from urllib.request import urlopen

with urlopen("https://api4.ipify.org", timeout=10) as response:
    runtime_ip = IPv4Address(response.read().decode("ascii").strip())
print("Temporary Atlas access-list entry:", str(runtime_ip) + "/32")
```

In Atlas, open the project's network access list, add that single address, and choose an expiring entry when available. Use an Atlas database user and the SRV URI from **Connect > Drivers**, with URI-reserved password characters percent encoded. The following read-only example assumes the earlier Metro Support `tickets` collection already exists. It does not load the CISA fixture.

Listing 167. Python

```python
from getpass import getpass
from pymongo import MongoClient
from pymongo.server_api import ServerApi

database_name = input("Existing practice database name: ").strip()
mongodb_uri = getpass("Atlas SRV connection URI (hidden): ")
try:
    if not mongodb_uri.startswith("mongodb+srv://"):
        raise ValueError("Use the Atlas Drivers SRV string.")
    with MongoClient(
        mongodb_uri, tls=True, tlsInsecure=False,
        server_api=ServerApi("1", strict=True),
        serverSelectionTimeoutMS=10000, timeoutMS=10000,
    ) as client:
        client.admin.command("ping")
        tickets = client[database_name]["tickets"]
        for ticket in tickets.find(
            {"status": "open"}, {"_id": 0, "ticket_id": 1}
        ).limit(20):
            print(ticket)
except Exception:
    raise RuntimeError(
        "Atlas read failed. Check the user, URI, runtime IP rule, "
        "cluster, TLS, and intended database. Keep TLS checks on."
    ) from None
finally:
    mongodb_uri = None
```

Do not add `tlsInsecure=True` as a routine fix. It disables certificate validation and can hide the real issue. Check the URI, current driver, system time, network access list, DNS, and supported TLS path.

The `with` block closes a created client when the example finishes or raises an error. The `finally` block clears the temporary URI variable, and the error message avoids repeating a credential-bearing driver exception. Neither action revokes a credential or removes copies accidentally saved elsewhere. The query reads at most twenty matching documents and makes no changes. A successful ping with an empty result can mean that the database name is wrong, the collection is absent, or no ticket is open. Check the intended source and a known identifier instead of interpreting an empty loop as proof of a successful import. Remove the temporary runtime IP rule when cloud practice is finished.

### 13.10.2 PostgreSQL/Supabase

Copy the **Session pooler** URI from Supabase's **Connect** panel and replace the password placeholder with your database password. Percent-encode special characters in the password. If the URI has no `sslmode`, append `?sslmode=require`, or `&sslmode=require` if it already has a query string. Keep an existing `verify-ca` or `verify-full` setting. Enter the URI only at the hidden prompt; do not save it in code or submitted output.

Listing 168. Python

```python
from getpass import getpass
import psycopg
from psycopg.conninfo import conninfo_to_dict

database_url = getpass("PostgreSQL connection URL (hidden): ")
try:
    try:
        parameters = conninfo_to_dict(database_url)
    except psycopg.ProgrammingError:
        raise ValueError("Malformed connection URI. Check password encoding.") from None
    sslmode = parameters.get("sslmode", "prefer")
    if sslmode not in {"require", "verify-ca", "verify-full"}:
        raise ValueError("Add sslmode=require to the URI query parameters.")
    with psycopg.connect(database_url, connect_timeout=10) as connection:
        with connection.cursor() as cursor:
            cursor.execute("SET LOCAL statement_timeout = '10s'")
            cursor.execute("SELECT count(*) FROM metro_support.tickets")
            print(cursor.fetchone())
except psycopg.Error as error:
    raise RuntimeError(
        f"PostgreSQL read failed ({type(error).__name__}, "
        f"SQLSTATE {error.sqlstate or 'not supplied'}). Check the session-pooler URL, "
        "database password, TLS mode, and existing practice table."
    ) from None
finally:
    database_url = None
```

On a network without direct IPv6 connectivity, use the provider's current IPv4- compatible session-pooler URI rather than changing database code randomly. `sslmode=require` prevents an unencrypted fallback but does not verify server identity. For production, use `verify-full` with the provider's CA certificate. Use the exact pooler username and database password from Supabase's **Connect** panel. A website login and an API key do not replace those database credentials. The example reads the earlier `metro_support.tickets` table and assumes it already exists. Its full schema and records are printed in Appendix C.

Inside the cursor block, pass changing **values** as parameters rather than building SQL by joining strings. This fragment uses the already opened cursor:

Listing 169. Python

```python
requested_status = "open"
cursor.execute(
    "SELECT ticket_id FROM metro_support.tickets "
    "WHERE status = %s ORDER BY ticket_id",
    (requested_status,),
)
print(cursor.fetchall())
```

The `%s` is a driver placeholder; it is not an invitation to apply Python string formatting. The separate one-item tuple carries the value. The driver keeps data from becoming SQL syntax. Do not put quotes around the placeholder. Table and column **names** are identifiers, not values; dynamic identifiers require a different composition mechanism such as `psycopg.sql.Identifier`. Psycopg parameter guide

## 13.11 Import in Small, Verifiable Batches

Before loading a public dataset:

1. inspect license, source, retrieval date, columns, types, nulls, duplicates, and size;
2. select a small subset tied to a question;

3. convert dates and missing values intentionally;
4. write a batch;
5. check acknowledged-write results or row counts; and
6. query a known sample and total.

An import script should be rerunnable or explicitly idempotent. Stable natural keys, upserts, a reset option in a disposable collection, or a recorded batch ID can prevent accidental duplicates.

**Idempotent** means repeating an operation has the same intended final effect as doing it once. An import keyed by a stable CVE identifier can update or keep that record on a second run rather than insert another copy. It must still define what happens if the source changes or a run stops partway through. Resetting an empty table before every import is a useful test setup, but it does not demonstrate idempotency: rerun the import against its existing output to test that property.

### 13.11.1 Atomicity and Repeat Behavior

Putting a complete SQL batch in a transaction means that if a row violates a required constraint, that transaction rolls back its earlier writes. A previous successful transaction remains committed. By contrast, the following complete Python program uses separate document-atomic replacements. It loads three **invented device readings**, not security-catalog records, so that the entire input is visible. Use the local MongoDB setup in Appendix C; no cloud account, download, or previous notebook is required.

**Listing 170. Python**

```
from uuid import uuid4
from pymongo import MongoClient

records = [
    {"reading_id": "r1", "device": "a", "temperature_c": 20},
    {"reading_id": "r2", "device": "a", "temperature_c": 21},
    {"reading_id": "r3", "device": "b", "temperature_c": 19},
]
with MongoClient("mongodb://127.0.0.1:27018/",
                 serverSelectionTimeoutMS=5000) as client:
    client.admin.command("ping")
    import_name = "book_import_" + uuid4().hex[:12]
    collection = client[import_name]["readings"]
    try:
        collection.create_index("reading_id", unique=True)
        for attempt in range(2):
            for record in records:
                collection.replace_one(
                    {"reading_id": record["reading_id"]},
                    dict(record), upsert=True,
                )
            actual = list(collection.find(
                {}, {"_id": 0}
            ).sort("reading_id", 1))
            assert actual == records
            print("Attempt", attempt + 1, "count:", len(actual))
    finally:
        client.drop_database(import_name)
```

Both attempts print a count of three. The unique `reading_id` index enforces one logical source record per identifier. If the third replacement fails, the first two completed replacements remain. Retry the unchanged, checked source against the same collection, then verify the result. This is not an all-document transaction. MongoDB supports transactions in appropriate deployments, but this loop does not request one.

The two properties can therefore vary independently. Atomicity defines which changes succeed together; idempotency describes the intended effect of repeating an operation. Neither supplies a complete production recovery policy. This import does not preserve previous versions, remove records absent from a later feed, resolve conflicting writers, or protect application-owned annotations that a whole-document replacement would erase. Each requires an explicit ownership or synchronization decision.

For a vulnerability catalog, the same pattern would use a unique `cveID` key. The next section studies a fixed public-data snapshot rather than inventing security facts. The optional integration notebook applies the pattern to that larger input and offers PostgreSQL and SQLite alternatives.

### 13.11.2 A Correct Count Can Conceal an Incorrect Value

Imagine changing only the vulnerability title for one imported record. The total can remain 75, the identifier can still exist, and vendor counts can still match. A comparison of all eight selected fields for that known identifier detects the error. Rerunning the authoritative, unchanged source can then repair it.

To compare across systems, convert SQL dates and serialized JSON back to the selected source representation before comparing values. PostgreSQL's `date` object and a JSON date string can represent the same day without being equal Python objects. SQLite's CWE value needs JSON decoding, while MongoDB and PostgreSQL return the list in a directly comparable form. A representation mismatch should not be mistaken for a changed real-world fact.

| Check | Detects in this example | Does not establish |
|---|---|---|
| Count equals 75 | Missing or extra records that change the total | Correct values |
| Known ID exists | Absence of the chosen record | Correct title or dates |
| Eight selected fields match | A changed selected value for that ID | Every other record is correct |
| Grouping matches the source | A discrepancy in the five displayed groups | Complete live-catalog coverage |

Verification is stronger when checks answer different questions. Report their scope rather than describing any one passing check as proof of production fitness.

### 13.11.3 Inspect the Public Source Before Loading It

The linked snapshot contains a documented 75-record selection from CISA's Known Exploited Vulnerabilities catalog. Its metadata records a July 2026 source version and retrieval, so it is a reproducible teaching snapshot, not a current operational security feed. A CVE identifies a vulnerability record; `dateAdded` describes catalog inclusion, not necessarily the date a vulnerability was discovered or first exploited. The sample selects the first source records, not a random sample of all software vulnerabilities.

Download kev_sample.json and place it beside this Python exercise, or upload it through Colab's Files panel. Unlike CSV, the JSON file preserves the CWE list as an array. The outer object contains source metadata; the actual records are inside its `vulnerabilities` array.

Listing 171. Python

```
import json
from collections import Counter

with open("kev_sample.json", encoding="utf-8") as source_file:
    source = json.load(source_file)

records = source["vulnerabilities"]
ids = set()
dates = Counter()
for record in records:
    ids.add(record["cveID"])
    dates[record["dateAdded"]] += 1

print("Source version:", source["catalogVersion"])
print("Records:", len(records))
print("Distinct CVE identifiers:", len(ids))
print("Distinct date-added values:", len(dates))
print("Most frequent dates:", dates.most_common(3))
```

`set` keeps distinct identifiers; `Counter` counts how often each date occurs. The two cardinalities and the largest date counts give different information about candidate keys. None of these counts ranks vendors by security quality or measures traffic to a future application. A key can spread records well while one popular record still receives most requests.

For the supplied fixture, the first three printed counts are 75 records, 75 distinct identifiers, and 38 distinct dates. The largest date frequency is eight. Counting the other candidate keys yields:

| Candidate | Distinct values | Largest repeated value count | Share of 75 records |
|---|---|---|---|
| `cveID` | 75 | 1 | 1.33% |
| `dateAdded` | 38 | 8 | 10.67% |
| `vendorProject` | 42 | 16 | 21.33% |
| vendor/product pair | 56 | 4 | 5.33% |

The following complete continuation uses the `records` loaded above. It computes the table's counts before simulating four simple placement buckets:

Listing 172. Python

```python
from bisect import bisect_right
from hashlib import sha256

for label, values in [
    ("cveID", [r["cveID"] for r in records]),
    ("dateAdded", [r["dateAdded"] for r in records]),
    ("vendorProject", [r["vendorProject"] for r in records]),
    ("vendor/product", [(r["vendorProject"], r["product"]) for r in records]),
]:
    counts = Counter(values)
    largest = max(counts.values())
    print(label, len(counts), largest, round(100 * largest / len(records), 2))

ordered = sorted(records, key=lambda r: (r["dateAdded"], r["cveID"]))
earlier, arrivals = ordered[:60], ordered[60:]
boundaries = [earlier[position]["dateAdded"] for position in (15, 30, 45)]
range_counts = [0, 0, 0, 0]
hash_counts = [0, 0, 0, 0]
for record in arrivals:
    range_counts[bisect_right(boundaries, record["dateAdded"])] += 1
    digest = sha256(record["cveID"].encode("utf-8")).hexdigest()
    hash_counts[int(digest, 16) % 4] += 1
print("Boundaries:", boundaries)
print("Later arrivals, ranged:", range_counts)
print("Later arrivals, hashed:", hash_counts)
```

This range experiment sorts records chronologically, uses the oldest 60 to set boundaries, and treats the newest 15 as later arrivals. The fixed boundaries are April 22, May 20, and June 1, 2026. All 15 later records enter the final date bucket. SHA-256 modulo 4 on their distinct CVE IDs gives bucket counts 3, 3, 6, and 3. Both totals equal 15. The experiment isolates placement and omits actual MongoDB balancing, migration, network work, and request latency.

Source inspection also identifies a trailing space in the vendor label `"SimpleHelp "`. Preserve it when reproducing these counts. A production import should reject missing required text, duplicate IDs, invalid dates, and non-list CWE values before loading. If a transformation trims labels, record that decision and check the changed grouping.

Grouping by `product` instead of `vendorProject` asks a different question. The product result includes `Windows: 4`, meaning four selected catalog entries whose product label is exactly Windows. It does not mean four infected computers. An independent `Counter` over the source checks the database grouping. An explicit label order breaks ties so that equal-count results remain comparable.

The optional integration notebook takes this inspected source through one selected database path, a repeat import, and a question-driven query. Select one `TARGET`: SQLite, Atlas, or PostgreSQL. Trying a second path is optional and follows cleanup of the first. The learning target is an understandable, repeatable client-to-database workflow, not accumulating accounts.

The cleanup cell removes only this run's practice table or collection and closes the connection. Other objects survive. PostgreSQL removes its generated schema only when empty. Atlas's temporary runtime-IP rule requires separate removal in the dashboard. Closing a client does not pause a hosted database or revoke network access. A useful handoff tells a future maintainer what was imported, what was verified, and which temporary resources or access rules remain.

## 13.12 Scaling Also Scales Operations

More nodes or stores mean more:

- credentials and network rules;
- metrics and alerts;
- backup components and restore order;
- version compatibility;
- failure combinations;
- cost controls; and
- people who must understand the system.

A capacity recommendation should include the operating cost, not only throughput.

A simple first-pass storage projection makes assumptions explicit. If the system starts with $D_0$ bytes, grows by $g$ bytes per day, retains $r$ days of new data, and carries an overhead multiplier $m$ for indexes, replication, and working space, then:

$$D_{\text{projected}} = (D_0 + g\,r)\,m.$$

This is not a vendor-sizing formula. It is a transparent baseline that reviewers can challenge: growth may be nonlinear, compression varies, replicas multiply physical storage, and peak working space can dominate a migration or index build.

For example, retaining an initial 100 MiB plus 20 MiB of new data per day for 30 days gives 700 MiB before overhead. An assumed factor of two gives 1,400 MiB. One MiB is 1,048,576 bytes; one MB is 1,000,000 bytes. Keep units explicit when comparing a measurement with a service quota. The multiplier is an assumption to refine, not a promise that every platform charges or accounts for replicas in that way. This formula also assumes the initial data is retained; an actual retention policy may expire some of it.

An appropriate beginner recommendation might be: “Our current data is small, so I would first measure the largest documents, index size, and the busiest query. If retained growth approaches the free quota, I would shorten justified retention, remove unnecessary copies, or evaluate another capacity tier before considering sharding.” It names a next measurement without inventing enterprise load or treating a paid upgrade as necessary to understand the decision.

## 13.13 Common Misconceptions

### 13.13.1 “High cardinality makes a perfect shard key”

Frequency, monotonicity, query targeting, and update behavior also matter.

### 13.13.2 “Hashed sharding makes every query faster”

It can improve distribution while turning range or non-key queries into broad operations.

### 13.13.3 “A free replica set lets us practice sharding”

Replication and sharding solve different problems. Atlas Free does not support a sharded cluster.

### 13.13.4 "If a script inserted rows, the import worked"

Verify counts, known identifiers, types, and a question-driven query. Handle partial failure and reruns.

## 13.14 Study and Practice

Use explicit assumptions to connect workload, data distribution, and capacity. A recommendation should identify the next useful measurement, not prescribe sharding merely because a platform offers it.

Evaluate three shard-key candidates for an event dataset: timestamp, device ID, and hashed event ID. For each, record cardinality, frequency, monotonicity, two query patterns, and likely distribution. Recommend what to measure next rather than inventing a final production answer.

## 13.15 Retrieval and Transfer

1. How do vertical scaling and sharding differ?
2. What does `mongos` do?
3. Why can a monotonic ranged shard key create a hotspot?
4. What is a scatter-gather query?
5. What tradeoff does hashed distribution make?
6. Which two connection practices protect credentials and TLS verification?

## 13.16 Further Reading

- MongoDB sharding
- MongoDB shard keys
- MongoDB choose a shard key
- PyMongo getting started
- Psycopg documentation
- Supabase database connections
- CISA's official KEV data mirror

# 14 Multiple Databases Multiply Options and Obligations

## 14.1 Operating Question

If one application uses PostgreSQL and MongoDB, which system owns each fact, how do changes cross the boundary, and how can an incident be diagnosed without making the systems disagree further?

## 14.2 Learning Outcomes

After this chapter, you can:

- distinguish polyglot persistence from casual technology accumulation;
- assign one authoritative owner to each fact;
- compare synchronous dual writes, event-driven propagation, and reconciliation;
- identify consistency, security, backup, and observability obligations across systems;
- triage a cross-database incident from symptoms, logs, and stored records; and
- defend a one-platform or two-platform architecture choice.

## 14.3 Polyglot Persistence Is Workload-Based Specialization

Two screens can disagree even when both databases are healthy. The staff screen may read the authoritative ticket row while the resident screen reads a copy maintained elsewhere. This chapter follows one change between those screens. The new skill is recognizing a boundary between systems, not building a large distributed application from scratch.

A polyglot system uses different data technologies for different responsibilities. For example:

- PostgreSQL owns users, permissions, and authoritative ticket state;
- MongoDB stores flexible, high-volume event detail or read-optimized ticket snapshots; and
- an object store holds attachments.

This can fit workloads well. It also creates more deployment, access, monitoring, recovery, and consistency work. A small application should prefer one database unless the second solves a specific problem worth that cost.

## 14.4 Every Fact Needs One Authoritative Owner

If both PostgreSQL and MongoDB contain `ticket.status`, answer:

- Which copy accepts the business update?
- Which copy is derived?
- How is the derived copy refreshed?
- How stale may it be?

- How is disagreement detected and repaired?
- Which copy is used during recovery?

Without an owner, "synchronize both" becomes an undefined circular obligation. Duplication can be deliberate when one source is authoritative and the other is a cache, projection, snapshot, or historical record.

An **authoritative** record is the record whose accepted update establishes the current business fact. A **projection** is a derived representation arranged for a particular read. A **snapshot** intentionally preserves a state at a named earlier time. These are different promises. A snapshot from February is not incorrect merely because today's authoritative value has changed; a projection advertised as current may be incorrect if it silently remains at February's state.

Ownership can differ by fact. PostgreSQL might own ticket status while an object store owns the attachment bytes. "PostgreSQL is the source of truth" is too broad if it implies the attachment can be reconstructed from a filename alone. State which fact and which accepted update you mean.

## 14.5 Synchronous Dual Writes Create Partial-Failure Risk

Naive sequence:

1. update PostgreSQL;
2. update MongoDB; and
3. return success.

If step 1 commits and step 2 fails, the stores disagree. Reversing the order only changes which partial outcome occurs. A distributed transaction protocol can coordinate some systems but adds availability, latency, and operational cost and is not generally supplied by two unrelated cloud APIs.

Do not hide the boundary behind a broad `try/except`. Record an operation ID, preserve an authoritative result, and design retry or reconciliation.

An exception handler sees what the client observed, not necessarily what a remote service committed. If MongoDB stored the projection but its response was lost, retrying can apply the change twice unless the operation is designed for that possibility. Returning an error to the browser also does not roll back a PostgreSQL transaction that has already committed. A transaction boundary inside one database is not a transaction boundary around arbitrary network calls.

## 14.6 Event-Driven Propagation Separates Commit From Projection

One pattern:

1. begin one PostgreSQL transaction;
2. update the authoritative record and insert its outgoing event in that same transaction;
3. commit both, or roll both back;
4. after commit, a relay delivers the recorded event to a consumer;
5. the consumer applies it to the MongoDB projection; and
6. monitoring identifies undelivered work, failed application, or excessive lag.

The **transactional outbox** stores the business change and outgoing event in the same local transaction. A relay publishes unsent events. This avoids the gap where the business row commits but the event is never recorded.

An **event** is a record describing a particular change. The **relay** moves that record from the outbox to its delivery channel. The **consumer** reads delivered events and applies the intended effect. Those names identify responsibilities; they do not require three separate products in a small application.

Suppose the relay sends an event successfully but stops before recording that it was sent. On restart it can send the event again. Alternatively, a consumer can apply the change and lose its acknowledgment. A design that allows redelivery can recover from those gaps, provided repeated application is safe. Recording an event in the same transaction closes the original missing-event gap; it does not magically establish exactly-once effects across all later systems.

The consumer should be idempotent: applying the same event twice should not create duplicate effect. Use stable event IDs and record processed state or use an upsert whose final state is deterministic.

For a transformation $f_e$ applying event $e$ to state $x$, idempotency means:

$$f_e(f_e(x)) = f_e(x).$$

Setting a status to a particular value can have that property. Adding one to a counter does not: applying the same increment twice changes the result twice. Even an idempotent status update can be wrong when it arrives out of order. Setting a ticket to an older `open` state twice is repeat-safe, but still wrong after a newer `resolved` state. Repetition and ordering require distinct reasoning.

Event-driven design trades immediate cross-store consistency for decoupling, retries, and observable lag. State whether that is acceptable to the user.

### 14.6.1 A Local Transaction You Can Inspect

This complete Python example makes the commit boundary executable without any cloud deployment or external file. SQLite holds three synthetic authoritative tickets and an outbox table. The first attempt updates ticket 1008 and deliberately raises an exception before recording its event. Because the exception leaves the transaction block, SQLite rolls back the ticket update. The next attempt writes the ticket and event together. Run the three Python blocks below in order in one session; Python's standard library contains everything they need.

Listing 173. Python

```
import sqlite3

connection = sqlite3.connect(":memory:")
connection.execute("PRAGMA foreign_keys = ON")
connection.executescript("""
CREATE TABLE tickets (
    ticket_id INTEGER PRIMARY KEY,
    status TEXT NOT NULL,
    version INTEGER NOT NULL
);
CREATE TABLE outbox (
    event_id TEXT PRIMARY KEY,
    ticket_id INTEGER NOT NULL REFERENCES tickets(ticket_id),
    status TEXT NOT NULL,
    source_version INTEGER NOT NULL,
    UNIQUE(ticket_id, source_version)
);
INSERT INTO tickets VALUES
    (1008, 'open', 16), (1009, 'resolved', 5), (1010, 'resolved', 7);
""")
connection.commit()
state_sql = """
    SELECT status, version, (SELECT count(*) FROM outbox)
    FROM tickets WHERE ticket_id = 1008
"""
print(connection.execute(state_sql).fetchone())
```

The initial commit establishes a starting point outside the transaction we deliberately break. Catch the exception **outside** the `with` block so that the connection context manager sees the failure and rolls the update back.

Listing 174. Python

```
try:
    with connection:
        connection.execute("""
            UPDATE tickets SET status = 'resolved', version = 17
            WHERE ticket_id = 1008 AND version = 16
        """)
        raise RuntimeError("Simulated failure before the event insert")
except RuntimeError:
    print("Attempt rolled back")
print(connection.execute(state_sql).fetchone())
```

Now carry out two accepted transitions. Each transaction checks the version it expects to replace and records that transition's full projected state. A failed event insert would roll back its ticket update, too.

Listing 175. Python

```
for old_version, new_status in [(16, "resolved"), (17, "closed")]:
    new_version = old_version + 1
    with connection:
        change = connection.execute(
            "UPDATE tickets SET status = ?, version = ? "
            "WHERE ticket_id = ? AND version = ?",
            (new_status, new_version, 1008, old_version),
        )
        if change.rowcount != 1:
            raise RuntimeError("Expected version did not match")
        connection.execute(
            "INSERT INTO outbox VALUES (?, ?, ?, ?)",
            (f"evt-1008-{new_version}", 1008, new_status, new_version),
        )
    print(connection.execute(state_sql).fetchone())
connection.close()
```

The printed tuples contain status, version, and outbox count, in that order:

| Observation in execution order | Ticket 1008 | Recorded outbox events |
|---|---|---|
| After initialization | `open`, version 16 | 0 |
| After the simulated failure and rollback | `open`, version 16 | 0 |
| After the successful commit | `resolved`, version 17 | 1 |
| After a later accepted close operation | `closed`, version 18 | 2 |

These are real SQLite transactions, not observations from a PostgreSQL server. The version predicate on the successful update prevents the same transition from matching twice. A unique ticket/version pair in the outbox also prevents two event identities from representing that same version. If the event insert fails, its transaction must roll back the ticket change too.

The optional outbox notebook provides another interface for exploring this boundary. No relay runs in the example. The events are recorded, but no delivery or downstream application has occurred. That distinction matters when interpreting an outbox count during an incident. In a PostgreSQL implementation, the same business update and event insert must share one local transaction; putting two statements next to each other in an application is not sufficient.

### 14.6.2 Try the Ordering Rule Without Cloud Infrastructure

The example below is an executable model of one projection. A **version** is a counter assigned in the authoritative order for this ticket, not the consumer's clock time. Each event carries the complete projected status at that version. Python dictionaries stand in for stored records; no database connection is made.

**Listing 176. Python**

```python
projection = {"ticket_id": 1008, "status": "open", "source_version": 16}
deliveries = [
    {"status": "resolved", "source_version": 17},
    {"status": "resolved", "source_version": 17},
    {"status": "closed", "source_version": 18},
    {"status": "open", "source_version": 16},
]

for event in deliveries:
    if event["source_version"] > projection["source_version"]:
        projection["status"] = event["status"]
        projection["source_version"] = event["source_version"]
    print(projection["status"], projection["source_version"])
```

The observed states should be `resolved 17`, `resolved 17`, `closed 18`, and `closed 18`. The duplicate changes nothing, and the late version 16 cannot move the projection backward. Removing that condition in this isolated simulation provides a counterexample: a late event can overwrite a newer state.

This works for the stated full-state events. If version 17 means "add one" and version 18 means "add two," applying only version 18 loses the earlier increment. Delta events need ordering, deduplication, or a state-reconstruction strategy appropriate to their meaning. A version number does not make arbitrary skipped operations safe.

The loop also assumes one consumer acts at a time. Two independent production consumers must not separately read an old version and then blindly write their own answers. For an existing MongoDB projection, an expected-version filter and update can compare and change the same document atomically. A missing projection needs a separate initialization policy. A zero-match result can mean "already newer" or "not present," so those cases need to be distinguished.

For an existing document whose `_id` is the authoritative ticket ID, this MongoDB command applies the complete version-18 state only when the stored version is older. To try it, connect mongosh as described in Appendix C and initialize this separate practice database. These are JavaScript shell commands, not Python:

**Listing 177. MongoDB shell / JavaScript**

```
const projectionPracticeName = "book_projection_" + ObjectId().toHexString();
db = db.getSiblingDB(projectionPracticeName);
db.ticket_projection.insertOne({
  _id: 1008, status: "open", source_version: 16,
  last_event_id: "evt-1008-16"
});
```

Now apply the guarded update:

**Listing 178. MongoDB shell / JavaScript**

```
db.ticket_projection.updateOne(
  { _id: 1008, source_version: { $lt: 18 } },
  { $set: { status: "closed", source_version: 18,
            last_event_id: "evt-1008-18" } }
)
```

MongoDB checks the filter and changes this document as one atomic operation. A duplicate version 18 or a delayed version 17 cannot lower a newer stored version when each command follows the same rule. This assumes valid numeric versions from one authoritative sequence and complete-state events.

Do not add `upsert: true` as a blanket response to a zero match. If `_id: 1008` already exists at a newer version, it fails the filter; attempting to insert another document with that same `_id` produces a duplicate-key conflict. Distinguish an absent document from an already-current one and define initialization separately. Also inspect equal-version field corruption: if version 18 is stored with the wrong status, this ordinary retry will not fix it.

For the first run, expect one matched and one modified document. Repeating the update matches zero because version 18 is no longer less than 18. Read the final value, then remove just the generated practice database:

**Listing 179. MongoDB shell / JavaScript**

```
db.ticket_projection.findOne({ _id: 1008 })
db.getSiblingDB(projectionPracticeName).dropDatabase()
```

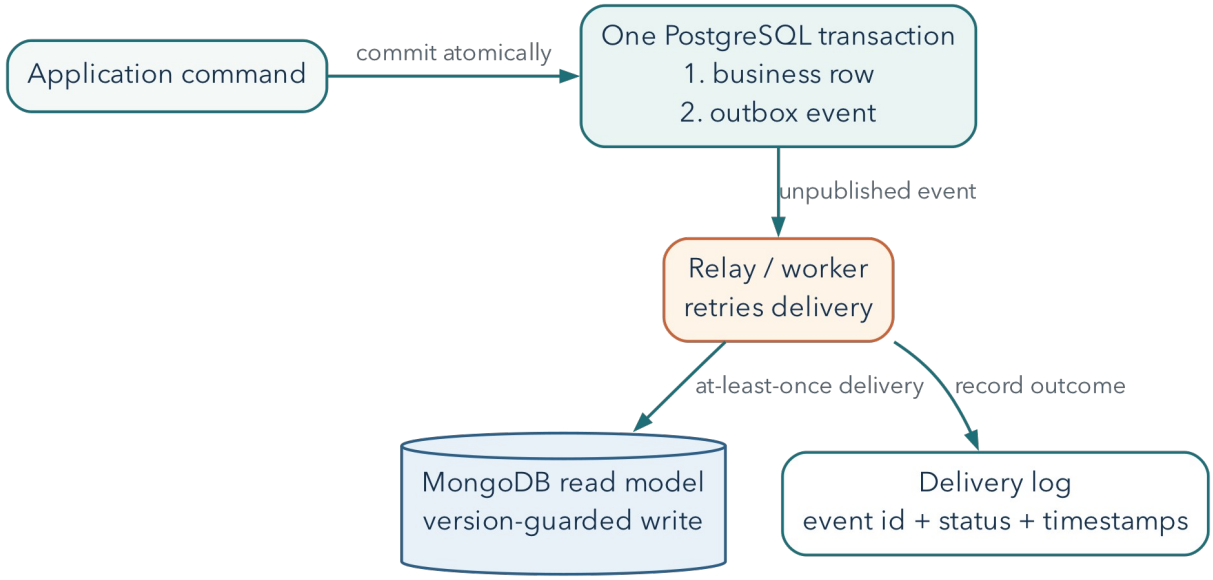



*Figure 14.1: A transactional outbox records the business change and event atomically, then delivers an idempotent projection asynchronously.*

## 14.7 Reconciliation Is a Designed Operation

Even a reliable pipeline needs a way to compare systems.

Useful checks include:

- counts by stable partition or date;
- missing authoritative identifiers;
- version or updated-at mismatches;
- hashes of canonical field sets;
- event offsets or last processed IDs; and
- samples of high-risk records.

Reconciliation output should identify an owner and repair direction. Blindly copying the newest timestamp can be wrong when clocks, delayed events, or manual repairs differ.

**Lag** is the gap between authoritative progress and applied projection progress. An empty queue does not necessarily prove a correct projection if failed messages were discarded. Counts can agree while field values differ, just as equal totals hid the array-counting mistake in Chapter 11. Compare stable identifiers and versions for the affected records, then broaden the check to the scope suggested by the failure.

### 14.7.1 Equal Counts Can Conceal Several Kinds of Disagreement

Consider the following complete comparison snapshot. There are three authoritative tickets and three resident-projection records, but their identifiers and owned fields differ:

| Ticket ID | Authoritative status/version | Resident status/version | Comparison |
|---|---|---|---|
| 1008 | `closed`, 18 | `closed`, 18 | matches after the guarded experiment |
| 1009 | `resolved`, 5 | absent | missing projection |
| 1010 | `resolved`, 7 | `open`, 7 | equal version, wrong status |
| 9999 | absent | `open`, 1 | unexpected projection ID |

Comparing the union of identifiers exposes three problems despite equal counts. Comparing versions alone misses ticket 1010. The comparison uses `status` and `source_version`, the fields owned by this authoritative source. It does not mistake projection-only processing metadata for another source-owned field.

The model constructs a separate candidate from the authoritative snapshot and checks both identifiers and values. A live rebuild requires more care: preserve independently owned data, account for writes arriving during the rebuild, verify the candidate, and coordinate the switch used by readers. A dictionary assignment does not implement that cutover. Likewise, an unexpected ID should prompt investigation of ownership and deletion history before an operator deletes it.

## 14.8 Identity and Access Cross the Boundary Too

A shared application user may map to:

- a Supabase Auth identity;
- PostgreSQL roles or RLS claims;
- an application service credential;
- an Atlas database user; and
- document-level owner identifiers.

Do not give browser clients powerful Atlas or Supabase service credentials. A backend service can mediate access, but it must enforce authorization and retain least privilege in both databases.

Audit records should correlate actions through a request or operation ID without copying secrets or unnecessary personal data.

## 14.9 Backup and Restore Need an Order

Independent logical dumps taken at different moments may not represent one application-consistent point. A recovery plan must state:

- which system is authoritative;
- backup cadence and recovery point for each;
- restore order;
- how derived data is rebuilt or reconciled;
- how events are replayed without duplication; and
- how traffic remains isolated until verification.

If MongoDB contains a rebuildable projection of PostgreSQL facts, restoring PostgreSQL and regenerating the projection may be safer than treating both dumps as equally authoritative.

## 14.10 Worked Incident: Ticket State Disagrees

**Symptoms:** The resident portal shows ticket 1008 as `open`. The staff dashboard shows `resolved`. The event feed contains a `ticket_resolved` event.

**Architecture:** PostgreSQL owns ticket state. An outbox relay updates a MongoDB read projection used by the resident portal.

### 14.10.1 Incident Trace

1. Query PostgreSQL ticket 1008 and its transaction/update time.
2. Query the outbox for the stable event ID and publish status.
3. Query consumer logs or checkpoint for that event ID.
4. Query the MongoDB projection's source version and event ID.
5. Check whether later events supersede it.

### 14.10.2 Plausible Finding

PostgreSQL and the outbox committed. The consumer failed after receiving the event but before updating the projection, and its retry queue is paused.

In this incident, the later MongoDB read still shows version 16 while the authoritative row and its event are version 17. The consumer recorded a timeout and then paused because its database credential expired. The timeout by itself is ambiguous; the later record comparison establishes that this particular projection is still behind. The first repair concerns the consumer's approved access and retry path, not reverting the correct authoritative ticket to match the stale page.

### 14.10.3 Response

Resume or replay the idempotent consumer from the recorded event. Do not manually edit both stores without recording the repair. Verify the MongoDB projection now contains the authoritative version, the event is marked processed, lag returns to normal, and no duplicate history was created.

### 14.10.4 Limitation

These observations support one incident path. They do not show that every projection is consistent; run the reconciliation check for the affected partition.

A useful update to a colleague would be: "The resident page is behind the staff record because the projection consumer is not progressing. Ticket 1008 is at authoritative version 17, while the inspected projection remains at 16. Restore the consumer's approved access, resume safe delivery, and verify version 17 or a newer authoritative state. Then inspect the other queued tickets before closing the incident." This explanation separates observed facts from the proposed repair; it does not claim the repair was completed before checking it.

## 14.11 Observability Should Follow a Request Across Systems

Useful signals include:

- request or operation ID;
- authoritative commit result;
- outbox backlog and oldest age;
- consumer success/failure and retry count;
- projection version lag;
- query latency and error by datastore;
- connection-pool exhaustion; and
- backup age and last restore verification.

Logs without stable identifiers make cross-system timelines guesswork. Metrics without user impact can also mislead. Connect technical observations to the failed operation.

## 14.12 A One-Database Design Is Often the Stronger Decision

Use one platform when:

- the workload fits its model and scale;
- the team is small;
- cross-store consistency would add risk;
- operational records are already difficult to correlate; or
- the second system is chosen only to appear advanced.

Use two when the workload benefit is concrete, ownership is explicit, propagation and recovery are designed, and the team can operate both.

## 14.13 Common Misconceptions

### 14.13.1 "The same ID means the records are synchronized"

Identifiers support correlation. They do not prove field equality, ordering, or successful propagation.

### 14.13.2 "Events guarantee consistency"

Events create a mechanism. Delivery, ordering, idempotency, retries, lag, and reconciliation determine behavior.

### 14.13.3 "Two backups produce one consistent recovery point"

Independent artifacts can represent different moments. Ownership and replay order must be part of the plan.

### 14.13.4 "Polyglot means using as many databases as possible"

It means matching technologies to distinct needs while accepting the operating cost.

## 14.14 Study and Practice

The worked incident observes a projection at version 16 after the authoritative record reaches version 17. The event simulation later advances to version 18. Keep those observation times distinct when explaining the failure and repair. Apply the same ownership and reconciliation reasoning to the inventory example below.

Create a responsibility table for a two-store inventory system. For inventory quantity, product description, flexible event details, and user access, name the authoritative store, derived copy if any, propagation path, acceptable staleness, reconciliation check, and recovery order.

## 14.15 Retrieval and Transfer

1. Why does each duplicated fact need one authoritative owner?
2. What partial failure occurs in synchronous dual writes?
3. How does a transactional outbox reduce one failure gap?
4. What makes an event consumer idempotent?
5. Why can two independent backups be application-inconsistent?
6. Which logs and checks would diagnose projection lag?

## 14.16 Further Reading

- Martin Fowler, Polyglot Persistence
- Microsoft Azure Architecture Center, Transactional Outbox pattern
- AWS Prescriptive Guidance, Transactional Outbox pattern
- PostgreSQL logical decoding concepts
- MongoDB change streams
- MongoDB single-document atomicity
- PostgreSQL transaction tutorial

# Part V: Professional Communication

Technical skill becomes career value when another person can inspect the decision, method, tradeoff, and result. Chapter 15 connects the book's technical work to portfolio artifacts, resume bullets, interview explanations, incident stories, and next-skill plans.

The target is not an exaggerated claim of production expertise. It is a precise, credible account of what you built, what you observed, how you verified it, and which boundary you would investigate next in a larger system.

# 15 Professional Communication Makes Technical Skill Visible

## 15.1 Operating Question

How can you explain what you learned so an instructor, teammate, or interviewer can understand your actions, results, reasoning, safety, and tradeoffs rather than only seeing a list of tools?

## 15.2 Learning Outcomes

After this chapter, you can:

- connect practice projects to database, cloud, backend, data, and security roles;
- review a database system through model, query, access, performance, and recovery decisions and results;
- present a database project without exposing credentials;
- write a concise portfolio project that another person can run or inspect;
- answer technical and behavioral interview questions with specific examples; and
- identify a realistic next skill rather than claiming production mastery.

## 15.3 Make Technical Work Understandable

A technical project becomes easier to discuss when another person can follow the problem, your decision, and the result. This applies to an interview, a handoff to another developer, or an explanation to the person who needs a report. It is not a request to turn every practice exercise into a claim of production expertise. Begin with work you actually completed and understand.

Tool names matter less when they are unsupported. Compare:

> Familiar with PostgreSQL, Supabase, MongoDB, Atlas, GitHub, and Python.

with:

> Built and verified a three-table PostgreSQL schema with named integrity constraints; reproduced and diagnosed a two-session blocking incident using `pg_stat_activity` and `pg_blocking_pids`; tested a workload-driven composite index with before/after plans; and restored a logical dump into a separate target with structure, relationship, and behavior checks.

The second statement identifies actions, results, and scope. It does not claim years of production operation.

## 15.4 A Database Skill Matrix

| Workplace need | Relevant technical work |
|---|---|
| understand a data model | relational schema, JSON alternatives, embed/reference decision |
| query and verify | SQL/MQL files, result-grain notes, aggregation checks |
| protect access | role/grant matrix, RLS allow/deny tests, credential-safe notebooks |
| investigate incidents | lock relationship, query plan, polyglot timeline |
| improve performance | unchanged workload, before/after plans, index tradeoff |
| recover data | logical artifact, separate restore, verification runbook |
| use cloud services | Supabase/Atlas shared-responsibility and free-tier decisions |
| automate carefully | small Python connection/import/recovery notebook |
| communicate | decision-result-tradeoff responses and final demonstration |

These skills can support database administration, cloud support, application support, backend development, data engineering, cybersecurity operations, and technical analysis. Job titles differ, but the underlying actions transfer.

For example, O*NET's database-administrator profile includes activities involving database changes, access, performance, and recovery. It is a useful vocabulary reference, not a guarantee that one introductory course meets every employer's requirements. Compare an actual posting with a particular piece of your work rather than adding all of its terms to a resume. O*NET Database Administrators

## 15.5 Review a System Through Six Questions

1. **Purpose:** what workload and user decision does the system support?
2. **Model:** what does one record mean, and how are relationships represented?
3. **Integrity and access:** which bad states and unauthorized actions are refused?
4. **Queries and performance:** which questions matter, and what plans/indexes support them?
5. **Reliability:** what failure is tolerated, what is backed up, and how is restore verified?
6. **Reproducibility:** can another person rebuild, inspect, and explain the result without receiving a secret?

Use these questions for a project walkthrough, a portfolio review, or an interview system-design prompt.

### 15.5.1 Review by Explaining One Result

Here is a small SQL review that runs without loading the Metro Support schema. The two named results inside `WITH` supply the sample rows. It asks for each staff member's active-ticket count, including zero. Explain both the mechanism and the result before considering how to present it to someone else.

Listing 180. SQL

```sql
WITH staff(agent_id, name) AS (
    VALUES (201, 'Agent 201'), (202, 'Agent 202'), (203, 'Agent 203')
),
ticket_sample(ticket_id, assignee_id, status) AS (
    VALUES (1, 201, 'open'), (2, 201, 'resolved'),
           (3, NULL::integer, 'new'), (4, 202, 'open')
)
SELECT s.agent_id, s.name, count(t.ticket_id) AS active_count
FROM staff AS s
LEFT JOIN ticket_sample AS t
    ON t.assignee_id = s.agent_id
   AND t.status IN ('new', 'open', 'in_progress')
GROUP BY s.agent_id, s.name
ORDER BY s.agent_id;
```

The result is Agent 201 1, Agent 202 1, Agent 203 0. There are three active tickets, but the staff counts sum to two because ticket 3 is unassigned. The left join preserves staff, not every ticket. `count(t.ticket_id)` ignores Agent 203's null-extended ticket field; `count(*)` would count that row and incorrectly give her one. Moving the ticket-status condition into WHERE would remove Agent 203 instead of preserving the zero.

That explanation contains several database ideas in a small space: a defined population, a join condition, NULL behavior, result grain, and an independent total. An interviewer can ask "what would change if …?" and you have a way to reason rather than recite a memorized definition.

The same habit transfers beyond SQL. After MongoDB `$unwind`, ask whether one document means a ticket or an event. After an access-policy change, ask which identity ran the query. After an index change, ask whether the same records were returned. After restore, ask which source state and rules were reconstructed. The product commands differ, but the reasoning is connected.

## 15.6 Present a Project as a Decision Story

A focused demonstration shows:

1. problem and user;
2. workload and platform choice;
3. model and one relationship decision;
4. one meaningful query and result;
5. one access, performance, or recovery decision with a concrete result;
6. one tradeoff or limitation; and
7. one next improvement.

Use prepared, redacted project outputs. Do not open a secrets page, paste a connection string, or depend on a live cloud request as the only demonstration. A short static fallback protects the presentation from network failure.

## 15.7 Build a Portfolio Project Around One Concept

A useful repository or folder does not need a complete app. It can teach one concept clearly:

**Listing 181. Text**

```
my-database-concept/
  README.md
  example.sql or example.ipynb   (only if useful)
```

The example may instead live entirely in fenced code blocks in the README. Multiple setup, session, and result files can be useful for a larger project. Choose the smallest package that makes your one idea understandable and reproducible.

The README should include:

- problem and learning objective;
- environment and prerequisites;
- safe run order;
- expected output;
- explanation of the result;
- cleanup;
- limitation; and
- source/license attribution.

Other strong concepts include RLS allow/deny tests, plan-based index design, logical restore verification, CSV-to-JSON model comparison, aggregation grain, or shard-key analysis.

For a Markdown-only concept guide, write the question first, show a small example and its expected result, and explain the important consequence. For a notebook, put a short explanation before each meaningful code step and interpret the result afterward. Running every cell without changing or explaining anything does not make the learning visible to another reader.

## 15.8 Remove Portfolio Risk

Before publishing:

- search for passwords, URIs, API keys, tokens, and private hostnames;
- remove real personal or regulated data;
- use readable text output or a redacted interface capture, depending on what the example needs to show;
- ensure scripts affect only a named disposable schema or collection;
- identify destructive commands prominently;
- test the run order in a clean environment; and
- include licenses and attributions.

Rotating a credential is necessary if it was exposed. Deleting the visible line is not enough because Git history may retain it.

## 15.9 Use STAR-R for Behavioral Questions

- **Situation:** concise context and impact.
- **Task:** your responsibility or decision.
- **Action:** what you personally inspected, changed, and verified.
- **Result:** observable outcome.
- **Reflection:** limitation and next production step.

Example:

> In a controlled PostgreSQL lab, one ticket update waited behind an open transaction. My task was to identify the relationship without terminating the wrong session. I queried `pg_stat_activity` and used `pg_blocking_pids`, which showed the waiting PID and one blocker whose transaction began earlier in my Session A client. I rolled back the controlled blocker, observed Session B complete, and queried the row from a fresh statement to confirm which values remained. The exercise showed the diagnostic method, but in production I would also confirm application ownership and business impact before cancellation and would review transaction timeout and workflow design.

This answer is credible because it narrows the context and names specific observations.

If you studied Chapter 5's written incident example instead of a live connection, change the opening to "I analyzed a provided two-session incident trace." You can still explain the blocking relationship and propose the correct check; you should not claim to have created or resolved a live wait. A precise scope statement lets the reader assess the skill you demonstrated.

### 15.9.1 Respond to a Follow-Up, Not Just the Opening Question

Suppose the interviewer asks, "Why did rollback help?" A useful answer is:

> Session A held the row lock because its transaction had not ended. Rolling it back released that lock and discarded A's uncommitted change. Session B could then continue. I checked the final row because releasing a wait does not tell me which business values were retained.

If asked, "Would you roll back any old transaction?", distinguish this controlled case from a production decision. Determine who owns the transaction, what work it contains, and who can authorize cancellation. Technical capability and authority to use it are separate. You can explain that distinction without having managed a production incident yourself.

## 15.10 Explain Technical Choices With Decision-Result-Tradeoff

Interview question: "Why did you add that index?"

- **Decision:** I kept a composite index on status and descending opening time for the newest-open-ticket queue.
- **Result:** On the 100,000-row fixture, the unchanged query moved from a scan plus sort to an index-supported path, removed the explicit sort, and examined far fewer rows for a 20-row result.
- **Tradeoff:** the index consumes storage and adds work to writes and backup, so I would monitor its actual usage and remove it if the workload changes.

Avoid "indexes make queries faster" without a query, plan, or cost.

Do not invent a speedup percentage from one favorable run. If the available result is a changed scan path and removal of a sort, describe those observations. If you measured repeated timings, state the fixture, query, runs, and limit of that comparison. The result need not be dramatic to demonstrate careful judgment.

## 15.11 Answer Design Questions by Stating Assumptions

Interview question: "Would you use PostgreSQL or MongoDB?"

A strong beginner response:

1. ask about access patterns, relationships, integrity, variation, scale, and recovery;
2. state an assumption;
3. choose the simpler suitable model;
4. show one alternative and tradeoff; and
5. name a test or measurement that could change the decision.

Example:

> If authoritative appointments have stable relationships and scheduling conflicts, I would begin with PostgreSQL constraints and transactions. If the system also stores highly variable, append-heavy form snapshots read as whole documents, MongoDB could own that bounded responsibility. I would not add a second store until query volume or model friction justified its synchronization and recovery cost.

## 15.12 Be Honest About Scope

Practice projects demonstrate foundational skill, not unsupervised production ownership. Useful phrases include:

- "I reproduced this in an isolated cloud lab…"
- "The result showed…"
- "This check did not cover…"
- "In production I would also inspect…"
- "I have not operated that feature at scale, but I can explain the decision and next measurement…"

Honest scope plus concrete reasoning is stronger than inflated certainty.

It is also reasonable not to know an unfamiliar product feature. Separate the known concept from the unverified implementation: "I have tested row-level security in PostgreSQL. I would check how this service maps an authenticated user to a database role and then run allowed and denied tests through that request path." This proposes a technically relevant next step without pretending all platforms work identically.

## 15.13 Plan the Next Skill From a Job Posting

Choose a role family and examine several current postings. Count recurring skills, then connect one gap to a small project.

Examples:

- Linux and shell: automate a safe dump verification script.
- Python: write a small, idempotent data import with logging.
- monitoring: create a local dashboard from query and backup-age metrics.
- cloud networking: document TLS, DNS, IPv4/IPv6, and allow-list diagnostics.
- CI: validate SQL/JSON/notebooks on every repository change.

Do not chase every product. Build depth around one measurable workflow.

## 15.14 Worked Example: Turn a Lab Into a Resume Bullet

Weak:

> Used MongoDB Atlas in class.

Relevant work:

- designed embedded and referenced ticket models;
- wrote MQL filters and an aggregation;
- added validation and a compound index;
- compared `COLLSCAN`/`IXSCAN`, documents examined, and returned;
- restored a logical collection export to a separate target; and
- documented Atlas Free limitations.

Possible bullet:

> Modeled and queried a MongoDB Atlas service-desk dataset, implemented focused BSON schema validation, and evaluated a compound index with execution statistics; documented a free-tier logical recovery and verification procedure.

Adjust wording to exactly match completed work.

## 15.15 Common Misconceptions

### 15.15.1 "A portfolio needs a full web application"

A small, reproducible operations project can demonstrate deeper database skill than a large app whose data decisions are hidden.

### 15.15.2 "The interview wants one correct database"

Many design prompts evaluate assumptions, tradeoffs, failure thinking, and testing.

### 15.15.3 "A certificate proves operational ability"

Training can structure learning. Projects and explanations show how you apply and verify it.

### 15.15.4 "Mentioning every tool sounds more qualified"

A few specific actions with results are easier to understand than a long technology inventory.

## 15.16 Study and Practice

Choose a technical decision you can explain from work you actually completed. Use the following formats to rehearse its purpose, mechanism, observed result, and limits for different audiences.

Choose one completed experiment or project. Write:

1. a 30-second explanation;
2. one resume bullet;
3. one STAR-R interview story;
4. one limitation; and
5. one next production-level measurement or control.

## 15.17 Retrieval and Transfer

1. Which six questions review a database system end to end?
2. What makes a portfolio run order safe and reproducible?
3. How does STAR-R improve a technical incident story?
4. What belongs in a decision-result-tradeoff answer?
5. Why should a presentation have a non-live fallback?
6. How can you describe practice projects without overstating production experience?

## 15.18 Further Reading

- O*NET Database Administrators
- O*NET in-demand DBA technologies
- U.S. Bureau of Labor Statistics, Database Administrators and Architects
- NACE Career Readiness Competencies
- GitHub Skills
- GitHub secret scanning guidance

# Appendix A: Notation and Technical Language

This appendix is a reading reference, not a replacement for the chapters. It collects mathematical symbols, query-language conventions, and operational terms that recur across the book.

## Relational Notation

Assume $T$ is a tickets relation and $U$ is a users relation.

| Operation | Conventional notation | Plain-language job | Common SQL form |
|---|---|---|---|
| selection | $\sigma_{\varphi}(T)$ | keep tuples for which predicate $\varphi$ is true | `WHERE` |
| projection | $\pi_{a_1,\ldots,a_k}(T)$ | keep selected attributes | `SELECT` list |
| rename | $\rho_S(T)$ | give a relation or attribute a usable name | `AS` |
| Cartesian product | $T \times U$ | pair every tuple in one input with every tuple in the other | `CROSS JOIN` |
| theta join | $T \bowtie_{\theta} U$ | keep paired tuples satisfying condition $\theta$ | `JOIN ... ON` |
| union | $T \cup U$ | tuples in either union-compatible relation | `UNION` |
| intersection | $T \cap U$ | tuples in both union-compatible relations | `INTERSECT` |
| difference | $T - U$ | tuples in the first compatible relation but not the second | `EXCEPT` |

The identity

$$T \bowtie_{\theta} U = \sigma_{\theta}(T \times U)$$

explains why an omitted join predicate can multiply rows. SQL is based on relational ideas but is not identical to classical relational algebra: SQL tables can contain duplicate rows, `NULL` introduces three-valued logic, ordering is explicit, and computed expressions and aggregation extend the basic operators.

## Logic and Set Symbols

| Symbol | Read as | Example |
|---|---|---|
| $\wedge$ | and | $p \wedge q$ is true only when both predicates are true |
| $\vee$ | or | $p \vee q$ is true when either predicate is true |
| $\neg$ | not | $\neg p$ negates predicate $p$ |
| $\in$ | is an element of | $v \in V$ means vertex $v$ belongs to set $V$ |

| Symbol | Read as | Example |
|---|---|---|
| $\subseteq$ | is a subset of | $A \subseteq B$ allows $A = B$ |
| $\lvert T \rvert$ | cardinality of $T$ | number of tuples in relation $T$ |
| $\sum$ | sum | $\sum_{i=1}^{n} x_i$ adds indexed values |
| $\sqrt{x}$ | square root | used in Euclidean norm and distance |
| $\approx$ | approximately equal | measured or rounded comparison |

In SQL, a `WHERE` clause keeps rows only when its predicate is `TRUE`. Both `FALSE` and `UNKNOWN` are filtered out. That is why `column = NULL` does not find missing values; use `IS NULL`.

## Graph and Vector Notation

A graph $G = (V, E)$ contains vertices $V$ and edges $E$. A walk follows connected vertices and edges; a simple path has no repeated vertex. In an undirected graph, the degree $deg(v)$ counts edges incident to vertex $v$, with a self-loop counted twice. A directed graph distinguishes incoming degree from outgoing degree.

A vector $x \in \mathbb{R}^n$ is an ordered list of $n$ real numbers. The dot product is $a \cdot b$, the Euclidean norm is $\|a\|_2$, and cosine similarity is

$$s_{\cos}(a, b) = \frac{a \cdot b}{\|a\|_2 \|b\|_2}.$$

Cosine similarity is undefined for a zero vector because its denominator is zero. It measures direction, not semantic truth. The embedding model, data, metric, and evaluation procedure determine whether vector neighbors are useful.

## Code-Language Conventions

Every code block in the second edition declares a language. The publication build adds a numbered listing label automatically.

| Label | What the listing represents |
|---|---|
| SQL | PostgreSQL-compatible SQL unless a note identifies another dialect |
| MongoDB shell / JavaScript | MQL or JavaScript-style values as entered in `mongosh` or Data Explorer |
| JSON | strict JSON: quoted keys/strings, no comments, and no trailing commas |
| Python | Python 3 code, normally a notebook or small client example |

| Label | What the listing represents |
|---|---|
| Shell | a command entered in a POSIX-like terminal; placeholders and environment variables must be replaced safely |
| Text | directory structure, pseudodata, or output that is not executable code |

`mongosh` object notation is not strict JSON. Values such as `ObjectId(...)`, `ISODate(...)`, regular expressions, and unquoted field names belong to shell or BSON representations. Extended JSON provides JSON-compatible encodings for BSON types when data crosses text-only boundaries.

## Operational Glossary

**Access pattern.** A recurring read or write shape, including predicates, returned fields, sort order, frequency, latency expectation, and consistency need.

**Atomicity.** The guarantee that a transaction's changes commit together or do not remain. MongoDB also guarantees atomicity for a single-document write.

**Backend.** A PostgreSQL server process serving a database session. Its PID is visible in `pg_stat_activity` and available through `pg_backend_pid()`.

**Backup.** A retained artifact or managed history intended for recovery. A backup is useful only with a compatible restore path and verification.

**BSON.** MongoDB's binary document representation, which supports more types than strict JSON.

**Cardinality.** Depending on context, the number of rows in a relation, distinct values in a column, or possible values of a key.

**Constraint.** A database-enforced rule such as `NOT NULL`, `UNIQUE`, `CHECK`, primary key, or foreign key.

**Durability.** A DBMS promise about committed data surviving failures within a documented model. It is not indefinite retention.

**Evidence.** An observable artifact that supports or contradicts a technical claim: query text, result, plan, count, checksum, denied action, log, restore test, or recorded limitation.

**Idempotent operation.** An operation designed so that repeating it produces the same intended state rather than accidental duplicates or compounded changes.

**Index.** An access structure that trades storage and write maintenance for faster support of selected predicates, joins, or ordering.

**Isolation.** Rules governing what concurrent transactions can observe and when they must wait, retry, or abort.

**Least privilege.** Giving an actor only the actions and resources required for its current job.

**Managed service.** A service in which the provider operates selected infrastructure and software layers while the customer retains data, access, configuration, workload, recovery, and application responsibilities defined by the plan and contract.

**MVCC.** Multi-version concurrency control, which lets transactions read appropriate row versions while concurrent changes proceed under an isolation model.

**PID.** Process identifier. In PostgreSQL activity diagnostics, the number of a server process. It is not a user ID, ticket ID, or permanent session name.

**Projection.** In relational algebra, choosing attributes; in MongoDB queries, choosing document fields. The similar word does not make the complete semantics identical.

**Read concern.** MongoDB setting that controls visibility guarantees for a read.

**Read preference.** MongoDB setting that chooses which replica-set members are eligible to serve a read.

**Recovery point objective (RPO).** Maximum tolerable data-loss interval. If the last recoverable state is at $t_r$ and failure occurs at $t_f$, realized loss is $t_f - t_r$.

**Recovery time objective (RTO).** Maximum tolerable service-recovery duration. If failure occurs at $t_f$ and acceptable service returns at $t_a$, realized recovery time is $t_a - t_f$.

**Replication.** Copying an active change history to other members for availability and read/write behavior. Replication is not a historical backup.

**Row-level security (RLS).** A PostgreSQL mechanism that applies row predicates according to role and policy context.

**Schema.** The structural and semantic rules for stored data. A flexible document system still has a schema, whether the server, application, convention, or data quality process enforces it.

**Selectivity.** The fraction of input rows expected to match a predicate, $s = \text{matching rows} / \text{input rows}$.

**Session.** A database conversation associated with a connection. A persistent session can execute multiple transactions before disconnecting.

**Shard key.** Fields used to distribute and route a sharded collection. Quality depends on distribution, targeting, growth, and workload rather than cardinality alone.

**Source of truth.** The authoritative owner of a fact when multiple stores hold copies or projections.

**Write concern.** MongoDB setting that defines the acknowledgment required from replica-set members for a write.

# Appendix B: Reusable Technical Templates

These templates turn a lab, project, or workplace exercise into a compact, reviewable record. Replace brackets with observed facts. Do not share secrets, private URLs, addresses, tokens, or unredacted account images.

Use a template when it helps organize the work at hand. Adapt its fields to the question and the environment rather than filling a form without a purpose.

## Companion Materials

The companion notebooks and datasets offer optional ways to explore the book. The complete relational starting data and indexing setup are also printed in Appendix C; no companion download is needed for them. Choose an optional resource by topic in the index below. In every edition, notebook links open the corresponding files in Google Colab, where **Open in GitHub** provides a read-only view of their source. These files are hosted in the companion repository.

The nine core notebooks below are also included in the reusable authoring package's `notebooks/` folder. PDF, Word, HTML and EPUB links all open public browser destinations; none requires that local folder structure. An EPUB reader needs an external browser to run a notebook. Reading the book itself requires neither a Colab account nor a course enrollment.

| Notebook in `notebooks/` | What you will practice |
|---|---|
| `01_relational_sql_review.ipynb` | SQL review and full-fixture practice |
| `02_postgres_transactions_locks.ipynb` | Transaction and blocking experiment |
| `03_postgres_backup_restore.ipynb` | PostgreSQL logical backup and separate restore |
| `04_atlas_mql_modeling.ipynb` | MongoDB CRUD, arrays, and document-model practice |
| `05_mongodb_logical_recovery.ipynb` | BSON-aware export and metadata recovery |
| `06_public_data_capacity_integration.ipynb` | Inspect and import public data through a selected database path |
| `07_aggregation_validation.ipynb` | Fresh four-ticket aggregation and validation case |
| `08_polyglot_incident.ipynb` | Diagnose a stale document copy and retry a failed delivery |
| `09_synthesis_review.ipynb` | Review joins, constraints, rollback, and recovery with SQLite |

Additional explorations include a views and migration notebook and a reporting notebook. The latter develops category summaries and one latest event per ticket.

Other links below open individual files. Their path labels also identify the same resources in the reusable authoring package.

| Other companion file | Use |
|---|---|
| `datasets/metro_support/postgres_setup.sql` | Full relational baseline, also printed in Appendix C |
| `datasets/cisa_kev_sample/kev_sample.json` | Bounded, documented public-data snapshot |
| `datasets/metro_support/users.csv` | User identifiers, names, and roles for relational examples |
| `weeks/week_07/performance_lab_setup.sql` | Index experiment, also printed in Appendix C |
| `datasets/metro_support/tickets.csv` | Ticket facts for relational and document-design examples |

If you downloaded an `.ipynb` file instead of using its direct link, open Google Colab and choose **File > Upload notebook**. Read its Markdown explanations before running the setup, examples, and cleanup cells. Run SQL scripts only in the database environment named by the example; Markdown explanations are text to read, not database commands.

## Query and Index Decision

**Question.** [State the user-visible or operational query in one sentence.]

**Expected grain.** [One row/document per what?]

**Baseline.** [Query text, representative parameters, result count, plan, timing method, and environment.]

**Hypothesis.** [Name the predicate, join, sort, or access path that may explain the observed work.]

**Controlled change.** [One query rewrite, statistic refresh, or index.]

**Verification.** [Same result meaning, comparable plan/run measurements, and write or storage cost.]

**Decision.** [Keep, revise, or remove the change, with one limitation.]

## Access-Control Test Matrix

| Actor | Resource | Action | Expected | Observed result | Decision |
|---|---|---|---|---|---|
| `[role/user]` | `[schema.table or collection]` | `[read/write/admin]` | allow/deny | `[result or error class]` | `[correct/change needed]` |

Test at least one required success and one required denial. A role name or policy definition alone does not establish effective access.

## Recovery Runbook Entry

1. **Failure and scope:** [deleted rows, damaged database, lost collection, account problem, region/service issue, or application corruption].
2. **Target promise:** [RPO and RTO, including assumptions].
3. **Artifact:** [type, creation command/process, timestamp, size, checksum, encryption, retention, and location category].
4. **Separate restore target:** [name and isolation boundary; never overwrite the source during a first test].
5. **Structure checks:** [schemas/collections, constraints/validators, indexes, roles/settings within scope].
6. **Data checks:** [counts, stable identifiers, relationships, aggregates].
7. **Behavior checks:** [one meaningful query and one expected rejection].
8. **Measured recovery:** [realized data-loss window and elapsed recovery time].
9. **Limitations:** [what the artifact and test do not cover].
10. **Cleanup:** [temporary target, credentials, access rules, and retained test record].

## Cloud Connection Diagnosis

Record only nonsecret facts.

| Layer | Safe question | Useful observation |
|---|---|---|
| service | Is the intended project/deployment active? | status label and capture date |
| network | Can this runtime reach the current endpoint? | error class, DNS result category, IPv4/IPv6 route category |
| TLS | Is a current client validating the expected service? | driver/client version and certificate mode; never a secret URI |
| identity | Is this a database credential rather than a website login? | redacted role/user label and authentication result |
| authorization | Does the identity have the required narrow action? | allowed and denied test results |
| object | Do the expected database, schema/collection, and table names exist? | metadata query or redacted tree |
| query | Does the filter use the intended types and values? | query plus known identifier/result |

Stop widening access when the observations point to another layer. Never normalize a TLS error with `tlsInsecure=True`, publish a connection string, or leave a broad temporary access rule in place after the exercise.

## Decision-Result-Tradeoff Paragraph

> **Decision:** [State one technical decision.] **Result:** [Name the observed result, comparison, or failure that supports it.] **Tradeoff:** [State the cost, limitation, or condition under which the decision should be revisited.]

This format is appropriate for a lab conclusion, project explanation, portfolio README, or interview answer. It is stronger than a tool list because it shows reasoning and scope.

# Appendix C: Running the Examples

You can study and use this book without enrolling in a course, opening an instructor's repository, or obtaining a classroom handout. This appendix contains the full relational starting data and the larger indexing workload. The document examples declare their starting data in their chapters. Companion notebooks are optional alternate interfaces, not missing pages of the book.

## Choose an Environment

A database **server** stores data and answers requests. A **client** sends those requests: a browser SQL editor, a terminal program, and a Python driver are different clients. Installing a driver does not start a server. Opening two browser tabs does not prove that they retain two independent database sessions.

| Reading or experiment | Environment needed |
|---|---|
| Relations, algebra, JSON, graph and vector reasoning | The printed examples; paper or a text editor is enough |
| SQL queries, keys, views, schema changes | PostgreSQL; PGlite supports the initial single-session examples |
| Blocking and concurrent transactions | A PostgreSQL server and two persistent client connections |
| Roles and grants | A personal PostgreSQL environment where role creation is permitted |
| Query-plan comparison | PostgreSQL and the indexing workload printed below |
| PostgreSQL logical recovery | A PostgreSQL server, an empty restore database, pg_dump and pg_restore |
| MongoDB querying, modeling, aggregation and validation | MongoDB Community or an Atlas practice deployment, with mongosh |
| Python driver and import examples | Python 3 plus the named driver and a reachable database |
| Replication, sharding and failure reasoning | The stated scenarios; a single local process does not demonstrate a distributed failure |

The worked PostgreSQL checks use version 15; planner choices and interface details can differ on other supported versions. MongoDB examples use operators available in version 8.0. Check tool/server compatibility before restoring a backup. Nothing in these examples requires a paid cloud plan.

## Start a PostgreSQL Session

For the first single-session examples, open the PGlite Playground. It runs a PostgreSQL build inside the browser. At its SQL prompt, enter the statement below as one complete statement. Do not include the printed code-listing label or Markdown backticks.

Listing 182. SQL

```sql
SELECT 2 + 3 AS sum, current_database() AS database_name;
```

The first value should be 5. The database name identifies the environment that answered; it need not match an example screenshot. Browser data is local to that browser profile. Keep important scripts outside the page; clearing site data can remove the practice database. This is not a connection to Supabase. See PGlite's architecture for its execution model.

If using Supabase instead, open a personal project on the Free plan, select **SQL Editor**, create a query, and run the same statement. The editor sends SQL to the project's remote PostgreSQL server. This dashboard path does not require a Python driver or a database URL. If no free project is available, use the browser or local option; do not upgrade just to follow the book. Supabase's database overview distinguishes its SQL and table editors.

For server-specific experiments, install PostgreSQL from the official operating-system downloads. Use its installer instructions to start a local server and retain the database administrator login securely. The examples below assume the client programs are on your PATH and that your local login can create a practice database:

**Listing 183. Shell**

```
createdb operating_cloud_practice
psql --dbname=operating_cloud_practice
```

These are operating-system terminal commands, not SQL. An installation may require an explicit host, port and database user; obtain those from your own installation rather than copying somebody else's credentials. A command-not-found error concerns the client installation or PATH. A connection error concerns the server address, startup state or authentication. Within psql, enter SQL and end each statement with a semicolon; use `\q` to leave the client.

For the two-session experiment, open a second terminal and run the same psql command there. Each client has its own server connection. In each session run `SELECT pg_backend_pid();`: different process IDs establish different connections. Keep the first session open until the chapter releases its transaction. A fresh browser-editor request may not preserve this behavior. PostgreSQL's installation tutorial explains the client/server prerequisites.

## The Complete Relational Baseline

Run the following five SQL blocks in order, in a dedicated practice database. They create all data needed for the full Metro Support examples: eight users, twelve tickets and twenty-one events. No CSV import, repository checkout or notebook execution is required.

**Reset warning:** the first block drops `metro_support` with `CASCADE`. That removes its existing objects and can remove dependent objects outside that schema. Never run it against shared or valuable data. Run it when deliberately starting a fresh experiment, not between steps that depend on a view, index or constraint you just created. Keep a script or backup of changes you want to retain.

### Create the Relations

This first block resets the practice schema and defines the three relations; `CASCADE` can also remove objects that depend on that schema. Foreign keys require users before tickets and tickets before events.

Listing 184. SQL

```
DROP SCHEMA IF EXISTS metro_support CASCADE;
CREATE SCHEMA metro_support;

CREATE TABLE metro_support.users (
    user_id integer PRIMARY KEY,
    display_name text NOT NULL,
    email text NOT NULL UNIQUE,
    role text NOT NULL,
    neighborhood text NOT NULL,
    created_at timestamptz NOT NULL
);

CREATE TABLE metro_support.tickets (
    ticket_id integer PRIMARY KEY,
    requester_id integer NOT NULL REFERENCES metro_support.users(user_id),
    assignee_id integer REFERENCES metro_support.users(user_id),
    category text NOT NULL,
    priority text NOT NULL,
    status text NOT NULL,
    subject text NOT NULL,
    opened_at timestamptz NOT NULL,
    closed_at timestamptz,
    CHECK (closed_at IS NULL OR closed_at >= opened_at)
);

CREATE TABLE metro_support.ticket_events (
    event_id integer PRIMARY KEY,
    ticket_id integer NOT NULL REFERENCES metro_support.tickets(ticket_id),
    actor_id integer NOT NULL REFERENCES metro_support.users(user_id),
    event_type text NOT NULL,
    old_status text,
    new_status text,
    note text,
    event_at timestamptz NOT NULL
);
```

## Insert the Eight Users

Identifiers distinguish records even when names or addresses change. The names and reserved example.test email addresses below are synthetic.

Listing 185. SQL

```
INSERT INTO metro_support.users
    (user_id, display_name, email, role, neighborhood, created_at)
VALUES
    (101, 'Requester 101' , 'requester101@example.test' , 'resident' , 'Harbor' ,
        '2026-01-08T14:20:00Z' ),
    (102, 'Requester 102' , 'requester102@example.test' , 'resident' , 'Northside'
        , '2026-01-10T09:15:00Z' ),
    (103, 'Requester 103' , 'requester103@example.test' , 'resident' , 'Central' ,
        '2026-01-12T18:05:00Z' ),
    (104, 'Requester 104' , 'requester104@example.test' , 'resident' , 'Harbor' ,
        '2026-01-18T11:40:00Z' ),
    (201, 'Agent 201' , 'agent201@example.test' , 'agent' , 'Central' ,
        '2025-11-03T13:00:00Z' ),
    (202, 'Agent 202' , 'agent202@example.test' , 'agent' , 'Northside'
        , '2025-11-05T13:00:00Z' ),
    (203, 'Agent 203' , 'agent203@example.test' , 'supervisor' ,
        'Central' , '2025-09-14T13:00:00Z' ),
    (204, 'Sam Okafor' , 'sam.okafor@example.test' , 'analyst' , 'Harbor' ,
        '2025-12-01T13:00:00Z' );
```

### Insert the Twelve Tickets

The two null assignees are intentional. Ticket 1002 intentionally starts with in_progress; Chapter 3 changes a copy to demonstrate a repair rather than relying on a hidden broken starter file.

Listing 186. SQL

```
INSERT INTO metro_support.tickets
    (ticket_id, requester_id, assignee_id, category, priority, status, subject,
        opened_at, closed_at)
VALUES
    (1001, 101, 201, 'streetlight' , 'high' , 'open' ,
        'Streetlight dark near bus stop' , '2026-02-01T23:10:00Z' , NULL),
    (1002, 102, 202, 'sanitation' , 'medium' , 'in_progress' ,
        'Missed recycling pickup' , '2026-02-02T15:45:00Z' , NULL),
    (1003, 103, 201, 'water' , 'urgent' , 'resolved' , 'Low water pressure' ,
        '2026-02-03T12:05:00Z' , '2026-02-03T19:40:00Z' ),
    (1004, 104, NULL, 'parks' , 'low' , 'new' , 'Broken bench slat' ,
        '2026-02-04T17:20:00Z' , NULL),
    (1005, 101, 202, 'sanitation' , 'high' , 'resolved' ,
        'Overflowing corner bin' , '2026-02-05T14:00:00Z' ,
        '2026-02-05T20:15:00Z' ),
    (1006, 102, 201, 'streetlight' , 'medium' , 'in_progress' ,
        'Flickering lamp outside library' , '2026-02-06T01:30:00Z' , NULL),
    (1007, 103, 202, 'parks' , 'medium' , 'open' ,
        'Playground gate will not latch' , '2026-02-07T16:10:00Z' , NULL),
    (1008, 104, 201, 'water' , 'high' , 'resolved' , 'Hydrant leaking slowly' ,
        '2026-02-08T10:25:00Z' , '2026-02-09T09:05:00Z' ),
    (1009, 101, NULL, 'transportation' , 'medium' , 'new' ,
        'Bus shelter panel cracked' , '2026-02-09T22:15:00Z' , NULL),
    (1010, 102, 202, 'sanitation' , 'low' , 'closed' , 'Replacement bin request'
        , '2026-02-10T13:50:00Z' , '2026-02-12T16:30:00Z' ),
    (1011, 103, 201, 'transportation' , 'high' , 'open' ,
        'Crosswalk signal delayed' , '2026-02-11T08:35:00Z' , NULL),
    (1012, 104, 202, 'streetlight' , 'low' , 'resolved' ,
        'Lamp stays on during daytime' , '2026-02-12T14:45:00Z' ,
        '2026-02-14T18:10:00Z' );
```

### Insert the Twenty-One Events

These rows record selected history, not a complete audit of every possible application action. An event timestamp and a ticket's current status answer different questions.

Listing 187. SQL

```sql
INSERT INTO metro_support.ticket_events
    (event_id, ticket_id, actor_id, event_type, old_status, new_status, note,
        event_at)
VALUES
    (5001, 1001, 101, 'created' , NULL, 'open' , 'Reported through mobile form'
        , '2026-02-01T23:10:00Z' ),
    (5002, 1001, 201, 'assigned' , 'open' , 'open' , 'Electrical crew notified'
        , '2026-02-02T14:05:00Z' ),
    (5003, 1002, 102, 'created' , NULL, 'open' ,
        'Pickup was scheduled for Monday' , '2026-02-02T15:45:00Z' ),
    (5004, 1002, 202, 'status_changed' , 'open' , 'in_progress' ,
        'Route supervisor checking vehicle log' , '2026-02-03T13:30:00Z' ),
    (5005, 1003, 103, 'created' , NULL, 'open' , 'Pressure lower on two floors'
        , '2026-02-03T12:05:00Z' ),
    (5006, 1003, 201, 'status_changed' , 'open' , 'in_progress' ,
        'Crew dispatched' , '2026-02-03T14:25:00Z' ),
    (5007, 1003, 201, 'status_changed' , 'in_progress' , 'resolved' ,
        'Valve adjustment restored pressure' , '2026-02-03T19:40:00Z' ),
    (5008, 1004, 104, 'created' , NULL, 'new' ,
        'Photo attached in original report' , '2026-02-04T17:20:00Z' ),
    (5009, 1005, 101, 'created' , NULL, 'open' , 'Bin blocks part of sidewalk' ,
        '2026-02-05T14:00:00Z' ),
    (5010, 1005, 202, 'status_changed' , 'open' , 'resolved' ,
        'Extra collection completed' , '2026-02-05T20:15:00Z' ),
    (5011, 1006, 102, 'created' , NULL, 'open' ,
        'Flicker repeats every few seconds' , '2026-02-06T01:30:00Z' ),
    (5012, 1006, 201, 'status_changed' , 'open' , 'in_progress' ,
        'Ballast inspection scheduled' , '2026-02-06T15:10:00Z' ),
    (5013, 1007, 103, 'created' , NULL, 'open' , 'Gate opens toward play area' ,
        '2026-02-07T16:10:00Z' ),
    (5014, 1008, 104, 'created' , NULL, 'open' , 'Small stream along curb' ,
        '2026-02-08T10:25:00Z' ),
    (5015, 1008, 201, 'status_changed' , 'open' , 'resolved' ,
        'Gasket replaced and area checked' , '2026-02-09T09:05:00Z' ),
    (5016, 1009, 101, 'created' , NULL, 'new' , 'No sharp edge visible' ,
        '2026-02-09T22:15:00Z' ),
    (5017, 1010, 102, 'created' , NULL, 'open' , 'Current bin lid is missing' ,
        '2026-02-10T13:50:00Z' ),
    (5018, 1010, 202, 'status_changed' , 'open' , 'closed' ,
        'Replacement delivered' , '2026-02-12T16:30:00Z' ),
    (5019, 1011, 103, 'created' , NULL, 'open' ,
        'Wait exceeds one full light cycle' , '2026-02-11T08:35:00Z' ),
    (5020, 1012, 104, 'created' , NULL, 'open' , 'Possible photocell issue' ,
        '2026-02-12T14:45:00Z' ),
    (5021, 1012, 202, 'status_changed' , 'open' , 'resolved' ,
        'Photocell cleaned and tested' , '2026-02-14T18:10:00Z' );
```

## Check the Loaded State

Expect users = 8, tickets = 12, and ticket_events = 21. SQL does not guarantee the display order without ORDER BY; compare each name with its count.

Listing 188. SQL

```sql
SELECT 'users' AS table_name, count(*) AS row_count FROM metro_support.users
UNION ALL
SELECT 'tickets', count(*) FROM metro_support.tickets
UNION ALL
SELECT 'ticket_events', count(*) FROM metro_support.ticket_events;
```

If a block fails, read the first error before proceeding. An error about a missing referenced row often means an earlier insert did not complete. Inside a failed transaction, run `ROLLBACK;` before retrying. Do not hide a failed setup by continuing until a later query happens to return rows.

For a known-record check, the following query must return ticket 1004, status new, a null assignee, and requester Requester 104:

Listing 189. SQL

```sql
SELECT t.ticket_id, t.status, t.assignee_id, u.display_name AS requester
FROM metro_support.tickets AS t
JOIN metro_support.users AS u ON u.user_id = t.requester_id
WHERE t.ticket_id = 1004;
```

Every setup block and chapter query uses qualified table names such as `metro_support.tickets`. Run the blocks in order; they also work as separate SQL Editor requests without relying on a session's `search_path`. In Supabase Table Editor, select the `metro_support` schema rather than assuming every table is in `public`.

## Start a Chapter From the Intended State

Chapters 2 and 3 begin with the relational baseline above. Chapter 4 creates separately named views and an identity example; retain those objects while following that chapter. Chapter 5 should start with a fresh baseline so ticket 1004 is new and unassigned. Chapter 6 names its Chapter 4 view prerequisite and also provides a complete alternative limited-view example. Chapter 7 uses the separate large workload below. Chapter 8 verifies whichever source you actually backed up; a changed source cannot be verified against unchanged baseline counts alone.

The complete MongoDB insert blocks in Chapters 10 and 11 create separate small fixtures. Run the opening block once and then continue in that same mongosh session. Chapter 12 creates its own reliability example. These smaller document fixtures are not automatically the twelve-ticket relational database converted to BSON. Expected counts belong to a particular fixture and starting state.

## The Indexing Workload

The twelve-ticket dataset teaches correctness, but it is too small to make a useful index comparison. This complete generator creates the deterministic 100,000-row workload used in Chapter 7, including 2,000 open tickets.

Run it only in a disposable database. It resets `performance_lab` and its dependent objects; it does not reset `metro_support`. The generated assignee numbers model a larger synthetic population and are not foreign keys into the eight-user baseline. Run the baseline EXPLAIN before creating a candidate index.

Listing 190. SQL

```sql
DROP SCHEMA IF EXISTS performance_lab CASCADE;
CREATE SCHEMA performance_lab;

CREATE TABLE performance_lab.tickets (
    ticket_id bigint PRIMARY KEY,
    assignee_id integer,
    category text NOT NULL,
    priority text NOT NULL,
    status text NOT NULL,
    subject text NOT NULL,
    opened_at timestamptz NOT NULL
);

INSERT INTO performance_lab.tickets
SELECT
    n AS ticket_id,
    CASE WHEN n % 13 = 0 THEN NULL ELSE 200 + (n % 25) END AS assignee_id,
    (ARRAY['streetlight', 'sanitation' , 'water' , 'parks' , 'transportation'
        ])[(n % 5) + 1] AS category,
    (ARRAY['low', 'medium', 'high', 'urgent'])[(n % 4) + 1] AS priority,
    CASE
        WHEN n % 100 < 2 THEN 'open'
        WHEN n % 100 < 7 THEN 'in_progress'
        WHEN n % 100 < 10 THEN 'new'
        ELSE 'closed'
    END AS status,
    'Synthetic ticket ' || n AS subject,
    timestamptz '2025-01-01 00:00:00+00' + n * interval '2 minutes' AS opened_at
FROM generate_series(1, 100000) AS g(n);

ANALYZE performance_lab.tickets;

SELECT count(*) AS row_count,
       count(*) FILTER (WHERE status = 'open') AS open_count
FROM performance_lab.tickets;
```

The final row should be `100000, 2000`. The remainder of Chapter 7 supplies the query, candidate index, measurement method and interpretation. Retain the same data and query when comparing plans. A different runtime need not reproduce the printed timing.

## Start a MongoDB Shell

MongoDB Community runs a local server; mongosh is a separate shell client. Install both using the Community installation guide and mongosh installation guide. The following terminal example starts an isolated loopback-only server on port 27018. Use a new empty directory and a personal trusted machine; this local demonstration has no authentication and is not a production deployment.

Listing 191. Shell

```
mkdir book_mongodb_data
mongod --dbpath book_mongodb_data --bind_ip 127.0.0.1 --port 27018
```

Leave that terminal running. In a second terminal, start the client:

Listing 192. Shell

```
mongosh "mongodb://127.0.0.1:27018/?directConnection=true"
```

Within mongosh, test the connection:

**Listing 193. MongoDB shell / JavaScript**

```
db.adminCommand({ ping: 1 })
```

Expect an `ok` value of 1. Then use the chapter's opening database-selection and insert block. Do not change the server bind address to expose this unauthenticated process to a network. Other local users may still reach a loopback service. If the chosen port is occupied, do not stop an unfamiliar process: use another unused port consistently in both server and client.

Chapter 10 explains the Atlas alternative: a database user, allowed client IP, and a shell or driver connection. Website login is not database authentication. An Atlas replica set can execute the ordinary document examples, but its free plan does not make every failover, backup or sharding exercise available.

When finished, run the chapter's targeted database cleanup, leave mongosh with `exit`, and stop only the foreground mongod you started using Control-C in its terminal. Closing mongosh alone does not stop the server. A stopped server's data directory remains on disk. Stopping a local process does not pause or delete an Atlas deployment.

## Python Clients and Code Fragments

A Python listing runs in Python, not in mongosh or SQL Editor. Standard-library modules such as `json`, `sqlite3`, `datetime` and `pathlib` need no separate package installation. PyMongo and Psycopg are database drivers.

With Python 3.11 or later installed, create a project-local virtual environment in an operating-system terminal:

**Listing 194. Shell**

```
python3 -m venv .venv
```

On macOS/Linux, run `source .venv/bin/activate`. On Windows, use `.venv\Scripts\activate.bat` in Command Prompt; installations that name their launcher `py` can use `py -3 -m venv .venv`. Then install the drivers:

**Listing 195. Shell**

```
python -m pip install "psycopg[binary]>=3.2,<4" "pymongo>=4.11,<5"
```

The Python executable inside this environment runs your `.py` files. Colab is another Python runtime on a remote machine. In Colab, use `%pip install "psycopg[binary]>=3.2,<4" "pymongo>=4.11,<5"` in a code cell. A percent-prefixed notebook command is not syntax for an ordinary Python file.

Complete examples define their inputs, connections and cleanup. Short fragments isolate an operation: for example, a parameterized-query fragment belongs inside an already open cursor block, before that cursor closes. A named variable is not created merely because it appears in an explanation. Use the complete setup associated with the example; do not paste a fragment into an unrelated empty session.

For a remote client, separate network reachability, TLS, authentication and authorization. Adding your laptop's address to Atlas does not authorize a Colab runtime with a different outgoing address. Chapter 13 explains that connection boundary. Keep certificate validation enabled and remove temporary access rules when practice ends.

## When a Result Differs

| Symptom | First useful check |
|---|---|
| Table or collection missing | Confirm the selected database/schema and run the intended setup, not just the query |
| Zero rows but no exception | Check the filter, record types and known identifier; an empty result can be correct |
| Different counts | Identify the exact fixture and any committed changes before comparing results |
| Constraint or validator rejection | Compare the proposed value with the named rule; some examples intentionally fail |
| A transaction remains aborted | Run ROLLBACK, correct the first error, then restart the intended transaction |
| Import works once but fails twice | Inspect unique keys and rerun semantics rather than deleting unrelated data |
| Remote connection fails | Diagnose the layer named by the error; do not solve every problem by widening access |
| A simulated result differs from a live server | Check which behavior the simulation actually implements |

Use the chapter's question and expected result to decide what to investigate. A green success indicator is not a substitute for checking the resulting data.

# License, Attribution, and Reuse

Unless a credit states otherwise, original instructional prose and media in *Operating Cloud Databases* are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

You may copy and adapt that material for noncommercial purposes when you credit Atilio Barreda, identify the source, link to the license, indicate changes, and license the adaptation under the same terms. Original code is available under the MIT License, with copyright (c) 2026 Atilio Barreda. Original synthetic datasets are released under CC0. Source-specific notices govern public or adapted data.

## Suggested Attribution

> Adapted from *Operating Cloud Databases: PostgreSQL, MongoDB, and Reliable Data Systems* by Atilio Barreda, second-edition draft, licensed CC BY-NC-SA 4.0. Changes: [describe changes].

## Vendor and Free-Access Boundary

MongoDB University activities, MongoDB-supplied educator presentations, MongoDB and PostgreSQL documentation, Atlas, Supabase, GitHub, and Colab may be useful and free without being original material authored for this book. Their inclusion in a reading list does not place them under the book's license. Product names and screenshots are used for instruction and identification; their owners retain their respective rights.

## Editions and Companion Resources

Word, HTML, EPUB, and PDF editions are generated from the same editable source. The explanations, full relational starting data, and core worked examples are contained in this book. Appendix B links optional executable companions by topic; Appendix C supplies setup, data, and troubleshooting instructions.

The public CST4714 class repository hosts a PDF copy and companion teaching materials. Course schedules and assignments belong to that separate teaching collection, not to the textbook's independent-study sequence.

An earlier textbook and OER course package are available in the separate CST4714 OER repository. Those two repositories may contain different editions. Identify the edition when citing or adapting an example. This second edition is a draft, not an institutionally approved release.

## Code License

The MIT License permits use, copying, modification, distribution, sublicensing, and sale of the original code with the copyright and permission notice retained. Consult its linked full text when redistributing code. The code is supplied without warranty. External libraries and third-party data retain their own licenses; the book's code license does not relicense those dependencies.